Deep Reinforcement Learning Notes

Yixuan Wang & Zhixiao Xiong

Weiyang College, Tsinghua University
{wangyixu21,xiong-zx21}@mails.tsinghua.edu.cn

July 17, 2024

Chapter 1 Reinforcement Learning Basics

Outline

In this chapter, we will introduce some important prerequisite knowledge and basics ideas in reinforcement learning. Based on these ideas, we will present several specific reinforcement learning algorithms, which will later serve as an important foundation to understand *deep* reinforcement learning .

In section 1.1, we introduce some basic concepts in sequential decision-making model, including the *state-value function* and the *Bellman equation*. Based on this, we can do *iterative value evaluation*, which later give rise to *value iteration*.

In section 1.2, we add actions to MRP and therefore formulates a policy. Then we want to evaluate how good a policy is, so we have *policy evaluation*. We also want to polish up the policy, so we have *policy improvement* and *policy iteration*.

Up to now, the model (typically the reward functions and the transition probabilities) are assumed known for decision making. However, in real life we have to figure out the model on our own. Therefore we introduce several estimation methods in section 1.3, including *Monte-Carlo evaluation* which sample the entire episode, and *temporal difference evaluation* which updates on a per-action basis, with the idea of *dragging the value toward what Bellman equation requires*

But both TD and MC deal with value function and we still need the transition functions in order to do policy improvement. We solve this problem by estimating Q function which helps us get rid of the transition probability and goes directly into the action part.

1.1 Markov Reward Process (MRP)

A Markov Reward Process (MRP) is a sequential decision-making model that describes the interaction between an agent and an environment in a series of states, where the agent receives rewards based on the states it transitions through. In an MRP, the next state of the agent solely depends on the current state and there is no notion of actions or decision-making. It addresses questions like "What is the value of being in a certain state?"

A typical MRP model consists of the following elements:

- State $\mathcal{S}$
- Dynamics/transition model p
- Reward function r (return at time t is R_t)
- Discount factor $\gamma \in [0, 1]$
- Return

$$G_t = r_t + \gamma r_{t+1} + \cdots + \gamma^{H-1} r_{t+H-1}$$

在 MRP 中，智能体的下一个状态仅仅依赖于当前状态，不存在动作或决策的概念

Definition (State-value function). $V(s)$ is the expected return starting from state s:

$$V(s) = \mathbb{E}\left[G_t | s_t = s\right] = \mathbb{E}\left[\sum_{k=0}^{H-1} \gamma^k r_{t+k} | s_t = s\right] \tag{1.1.1}$$

If $H = \infty$, then $V(s) = \mathbb{E}\left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} | s_t = s\right]$.

Theorem 1.1. The MRP value function V satisfies the **Bellman equation**:

$$V(s) = r(s) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s)V(s') \tag{1.1.2}$$

And the following equation must hold for a policy to be optimal:

$$V^{\star}(s) = \max_{a \in A(s)} \sum_{s' \in S} P_a\left(s' \mid s\right)\left[r\left(s, a, s'\right) + \gamma V\left(s'\right)\right] \tag{1.1.3}$$

We can break this equation down:

$$V^{\star}(s) = \overbrace{\max_{a \in A(s)}}^{\text{best action from } s} \overbrace{\underbrace{\sum}_{\text{for every state}} P_a\left(s' \mid s\right)\left[\underbrace{r\left(s, a, s'\right)}_{\text{immediate reward}} + \underbrace{\gamma}_{\text{discount factor}} \cdot \underbrace{V\left(s'\right)}_{\text{value of } s'}\right]}^{\text{expected reward of executing action } a \text{ in state } s} \tag{1.1.4}$$

First, we calculate the expected reward for each action. The reward of an action is: the sum of the immediate reward for all states possibly resulting from that action plus the discounted future reward of those states.Second, the optimal value $V^{\star}(s)$ is the value of the action with the maximum the expected reward.

If states are **finite**, then in matrix form:

$$\begin{aligned}
\begin{pmatrix} V(s_1) \\ \vdots \\ V(s_N) \end{pmatrix} &= \begin{pmatrix} r(s_1) \\ \vdots \\ r(s_N) \end{pmatrix} + \gamma \begin{pmatrix} p(s_1|s_1) & \cdots & p(s_N|s_1) \\ p(s_1|s_2) & \cdots & p(s_N|s_2) \\ \vdots & \ddots & \vdots \\ p(s_1|s_N) & \cdots & p(s_N|s_N) \end{pmatrix} \begin{pmatrix} V(s_1) \\ \vdots \\ V(s_N) \end{pmatrix}, \\
V &= R + \gamma PV, \\
V - \gamma PV &= R, \\
V &= (I - \gamma P)^{-1} R.
\end{aligned} \tag{1.1.5}$$

Iterative algorithm for computing value of a MRP is shown in Alg. 1. Computational complexity is $O(|\mathcal{S}|^2)$ for each iteration.

Algorithm 1 Iterative value evaluation for MRP

Require: Initialize $V(s) = 0, \forall s \in \mathcal{S}$

1: **repeat**
2: **for** $s \in \mathcal{S}$ **do**
3: $V_k(s) = r(s) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s) V_{k-1}(s')$
4: **until** Convergence

MRP 是 MDP（马尔可夫决策过程）的一个特例，它相当于一个已经确定了策略的 MDP 。由于没有动作选择的环节，状态之间的转移是根据环境固有的概率 p(s'|s) 自动发生的，所以其定义和价值方程中不涉及策略 π。

1.2 Markov Decision Process (MDP)

A Markov Decision Process (MDP) extends the concept of an MRP by introducing the notion of actions and decision-making. It is a mathematical framework that models sequential decision-making problems in a controlled environment. In an MDP, the agent takes actions at each state, which influences the transition to the next state, the rewards received, and ultimately affects the agent's long-term goals. It addresses questions like "What is the optimal policy for the agent to follow to maximize its long-term rewards?"

Similar to MRP, a MDP model have the following elements:

- State space $\mathcal{S}$
- Action space $\mathcal{A}$
- Dynamics/transition model p

$$p(s'|s,a) = \Pr(s_{t+1} = s'|s_t = s, a_t = a) \tag{1.2.1}$$

- Reward function r

$$r(s_t = s, a_t = a) = \mathbb{E}[r_t|s_t = s, a_t = a] \tag{1.2.2}$$

- Discount factor $\gamma \in [0,1]$
- Policy π

$$\pi(a|s) = \Pr(a_t = a|s_t = s) \tag{1.2.3}$$

定义奖励的方式不同，从对state奖励变为对(s,a)奖励

MDP + π $\Longrightarrow$ MRP. In other words, MRP is $(\mathcal{S}, R^\pi, P^\pi, \gamma)$ where

$$\begin{aligned} r^\pi(s) &= \sum_{a\in\mathcal{A}} \pi(a|s) r(s,a), \\ p^\pi(s'|s) &= \sum_{a\in\mathcal{A}} \pi(a|s) p(s'|s,a). \end{aligned} \tag{1.2.4}$$

1.2.1 Policy Evaluation

How to evaluate a policy π? The following iterative algorithm in Alg. 2 provides a solution. The computational complexity is $O(|\mathcal{A}||\mathcal{S}|^2)$for each iteration. The value function V_k^π converges to V^π as $k \to \infty$, which is the value function of policy π.

Algorithm 2 Iterative Policy Evaluation

Require: Initialize $V_0^\pi(s) = 0, \forall s \in \mathcal{S}$
1: **repeat**
2: **for** $s \in \mathcal{S}$ **do**
3: Compute $V_k^\pi(s) = \sum_{a\in\mathcal{A}} \pi(a|s) \left[R(s,a) + \gamma \sum_{s'\in\mathcal{S}} P(s'|s,a) V_{k-1}^\pi(s')\right]$
4: **until** V_k^π converge
5: **return** Value function V^π

Note. When the policy π is deterministic, the above algorithm can be simplified as

$$V_k^\pi(s) = r(s,\pi(s)) + \gamma \sum_{s'\in\mathcal{S}} p(s'|s,\pi(s)) V_{k-1}^\pi(s'). \tag{1.2.5}$$

Question. Can we initialize the value function $V_0^\pi(s)$ to be anything other than 0? Will the V_k^π converge to V^π?

1.2.5 节和 1.4 节详细论证了贝尔曼算子是一个收缩映射 (Contraction Mapping) 。收缩映射的一个关键数学特性是，无论从哪个点开始，反复应用这个映射，最终都会收敛到一个唯一的不动点 (Fixed Point)，所以无论初始值如何，最终都会收敛到策略 π 对应的真实价值函数

Optimal Policy

- The optimal policy
$$\pi^*(s) = \arg\max_\pi V^\pi(s)$$
- There exists a **unique** optimal value function
- The optimal policy for a **infinite** MDP is
  - **Deterministic**: given any state, the optimal policy prescribes exactly one action to perform, rather than a probability distribution over multiple actions.
  - **Stationary**:the optimal policy remains the same over time and is not dependent on the current time step.
  - **Not necessarily unique**: there may be multiple optimal policies that achieve the same maximum expected cumulative reward

Remark. Number of deterministic policies is $|\mathcal{A}|^{|\mathcal{S}|}$.

1.2.2 Partial Observable MDP (POMDP)

Things could get more complicated when we don't have access to the full state s_t at time t. Instead, we have access to an observation o_t at time t, which is a function of the state s_t at time t.

A POMDP has the following components:

- State space $\mathcal{S}$
- Action space $\mathcal{A}$
- Observation space $\mathcal{O}$
- Dynamics/transition model p
- Emission probability $\mathcal{E}$
- Reward function $r : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$
- Discount factor $\gamma \in [0, 1]$

比如现实任务中，只能获取agent身边的准确信息(state)

$$\mathcal{E}(o|s) = \Pr(o_t = o|s_t = s) \tag{1.2.6}$$

Note. For the sake of simplicity, we assume that the model is fully observable in the following sections.

1.2.3 Policy Iteration

Definition (State-Action Value). $Q^\pi(s, a)$ is the expected return starting from state s, taking action a, and then following policy π.

$$\begin{aligned} Q^\pi(s,a) &= \mathbb{E}_\pi\left[G_t|s_t = s, a_t = a\right] \\ &= r(s,a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s,a) V^\pi(s') \\ &= r(s,a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s,a) \sum_{a' \in \mathcal{A}} \pi(a'|s') Q^\pi(s',a') \end{aligned} \tag{1.2.7}$$

Note that $Q^\pi(s, a)$ differs from $V^\pi(s)$ in that it specify a perticular action a to take at state s, so the following equations would hold:

$$V^\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) Q^\pi(s,a) \tag{1.2.8}$$

$$V^\star(s) = \max_{a \in A(s)} Q(s,a) \tag{1.2.9}$$

The following algorithm in Alg. 3 provides an iterative method to find the optimal policy.

The idea of policy evaluation is to estimate how good the policy is (i.e., the value function V^π of a policy π). And the idea of policy improvement is to find a better policy π' based on the value function V^π.

Algorithm 3 Policy Iteration

Require: Initialize π_0 arbitrarily
1: **repeat**
2: Policy evaluation: compute $V^{\pi_k}(s)$ for all $s \in \mathcal{S}$
3: Policy improvement: $\pi_{k+1}(s) = \arg\max_a Q^{\pi_k}(s,a)$ for all $s \in \mathcal{S}$
4: $k \longleftarrow k+1$
5: **until** $\|\pi_k - \pi_{k-1}\|_1 = 0$
6: **return** Optimal policy π^*

In the policy evaluation step, what we do is essentially iterate over k until the value for each state converges. Note that this is quite similar to value iteration, only without taking the arg max of a:

$$V_{k+1}^{\pi_i}(s) \leftarrow \sum_{s'} T\left(s, \pi_i(s), s'\right)\left[R\left(s, \pi_i(s), s'\right) + \gamma V_k^{\pi_i}\left(s'\right)\right] \tag{1.2.10}$$

In the policy improvement step, we perform a one-step look-ahead to get a better policy using **policy extraction** (select the action with the highest expected reward):

$$\pi_{i+1}(s) = \arg\max_a \sum_{s'} T\left(s, a, s'\right)\left[R\left(s, a, s'\right) + \gamma V^{\pi_i}\left(s'\right)\right] \tag{1.2.11}$$

Remark. Evaluating a policy π is time-consuming, which has the complexity of $O(|\mathcal{A}||\mathcal{S}|^2)$ for each iteration. This have to be done for several iterations until the value function of policy V^π converges.

Policy improvement is also time-consuming, which has the complexity of $O(|\mathcal{A}||\mathcal{S}|^2)$ for each iteration. Why is this? Because evaluating each $Q(s,a)$ requires computing for $O(|\mathcal{S}||\mathcal{A}|)$ times, and to to improve the policy π, we need to compute $Q^\pi(s,a)$ for each state s.

When the policy doesn't change for any state, the policy is optimal.

Definition. A policy π_1 is better than a policy π_2 if

$$V^{\pi_1} \geq V^{\pi_2} : V^{\pi_1}(s) \geq V^{\pi_2}(s), \forall s \in \mathcal{S}. \tag{1.2.12}$$

Theorem 1.2 (Monotonic Improvement in Policy).

$$V^{\pi_{i+1}} \geq V^{\pi_i}, \forall i \geq 0. \tag{1.2.13}$$

with strict inequality unless π_i is suboptimal, where π_{i+1} is the policy obtained by Alg. 3 from π_i.

Proof

$$\begin{aligned}
V^{\pi_i}(s) &\leq \max_a Q^{\pi_i}(s,a) \\
&= \max_a \left[r(s,a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s,a) V^{\pi_i}(s') \right] \\
&= r(s, \pi_{i+1}(s)) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, \pi_{i+1}(s)) V^{\pi_i}(s') \\
&\leq r(s, \pi_{i+1}(s)) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, \pi_{i+1}(s)) V^{\pi_{i+1}}(s) \\
&= V^{\pi_{i+1}}(s).
\end{aligned} \tag{1.2.14}$$

Note. The value function V^{π_i} is a sequence of monotonically increasing functions which converges to V^{π^*}, the optimal value.

1.2.4 Value Iteration

Policy iteration computes infinite horizon value of a policy and then improves the policy. Value iteration computes the optimal value function (i.e., the value function under the optimal policy π^*) directly by

- Maintain optimal value of starting in a state s if have a finite number of steps k left in the episode
- Iterate to consider longer and longer episodes

Algorithm 4 Value Iteration

Require: Initialize $V_0(s) = 0, \forall s \in \mathcal{S}$
1: **repeat**
2: **for** $s \in \mathcal{S}$ **do**
3: $V_k(s) = \max_a [r(s,a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s,a) V_{k-1}(s')]$
4: **until** Convergence, e.g., $\|V_{k+1} - V_k\| < \epsilon$

Note that in value iteration, we perform a max over action space. That is to say we always maintain the state value to be optimal. However in the evaluation step in policy iteration, we employ the transition probability instead of the argmax, where the optimal assumption does not hold.

Remark. For each iteration in Alg. 4, we need to compute the value of each state s, which has the complexity of $O(|\mathcal{A}||\mathcal{S}|^2)$. This seems to be much more efficient than policy iteration. However, in reality, the policy iteration usually converges much faster, which explains why both methods are used in practice.

1.2.5 Bellman Backup Operator

In this section, we will introduce the **Bellman backup operator**, which contains some math and hence is quite elegant.

Theorem 1.3 (Bellman Equation). Value function of a policy π satisfies the Bellman equation

$$V^\pi(s) = r(s, \pi(s)) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, \pi(s)) V^\pi(s'). \tag{1.2.15}$$

Definition (Bellman Backup Operator). The Bellman backup operator for policy π is defined as

$$BV(s) = \max_a [r(s,a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s,a) V(s')]. \tag{1.2.16}$$

It is easy to seet that the Bellman backup operator

- applies to a value function
- returns a new value function
- improves the value if possible

We can denote the Bellman operator applied to value function at all states as BV. Furthermore, we can define the Bellman operator for policy π as B_π.

$$\begin{aligned} B^\pi V(s) &= r^\pi(s) + \gamma \sum_{s' \in \mathcal{S}} p^\pi(s'|s) V(s') \\ &= \sum_{a \in \mathcal{A}} \pi(a|s) \left[r(s,a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s,a) V(s') \right] \end{aligned} \tag{1.2.17}$$

Then the policy evaluation is equivalent to

$$V^\pi = B^\pi B^\pi \dots B^\pi V.$$

We might want to ask, then, does value iteration converge in theory? Does it converge to a unique optimal value function? Fortunately, the answer is yes and is guaranteed by the fact that the Bellman equation is a **contraction mapping**.

Definition (Contraction Mapping). Let $T : M \to N$ be a mapping. T is a contraction if there exists a constant $\gamma \in [0, 1)$ such that for all $x, y \in M$, the following holds:

$$\|T(x) - T(y)\|_M \leq \gamma \|x - y\|_N$$

where $\|\cdot\|_M$ and $\|\cdot\|_N$ denote proper norms on M and N, respectively.

Theorem 1.4. The Bellman operator $\mathcal{B}$ is a contraction mapping for value function V w.r.t. the ∞-norm.

无穷范数就是向量中绝对值最大的那个元素的值

Proof. Let $|\cdot|$ denote the absolute value and $\|\cdot\|$ the infinity norm. $\forall s \in \mathcal{S}$ and value functions V_1, V_2:

$$\begin{aligned}
|\mathcal{B}V_1(s) - \mathcal{B}V_2(s)| &= \left| \left(\max_a \left[r(s,a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s,a) V_1(s') \right] \right) - \left(\max_a \left[r(s,a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s,a) V_2(s') \right] \right) \right| \\
&\leq \max_a \left| \left[r(s,a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s,a) V_1(s') \right] - \left[r(s,a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s,a) V_2(s') \right] \right| \\
&= \gamma \left| \max_a \sum_{s' \in \mathcal{S}} p(s'|s,a) |V_1(s') - V_2(s')| \right| \\
&\leq \gamma \max_{s'} |V_1(s') - V_2(s')| \cdot \max_a \sum_{s' \in \mathcal{S}} p(s'|s,a) \\
&= \gamma \max_{s'} |V_1(s') - V_2(s')| \cdot 1 \\
&= \gamma \max_{s'} |V_1(s') - V_2(s')| \\
&= \gamma \|V_1 - V_2\|
\end{aligned}$$

Therefore, with $0 < \gamma < 1$, we have:

$$\|\mathcal{B}V_1 - \mathcal{B}V_2\| \leq \gamma \|V_1 - V_2\|$$

which proves that $\mathcal{B}$ is a contraction mapping. □

1.3 Model-Free Value Methods

In previous sections we introduced MRP and MDP. The key idea of MRP is to understand the value of states in terms of the expected cumulative rewards the agent can achieve starting from those states. MDP builds on this idea by introducing actions and characterizing the state-action transitions and rewards to determine the optimal policy.

However in previous discussions, we assume that we have access to the dynamics P and reward function R. (Recall that we need to know dynamics in order to do eigher value or policy iteration.)

In this section, let's consider a more general yet more difficult case: we don't have access to the dynamics P and reward function R. We try to figure these out by estimation methods.

1.3.1 Monte Carlo (MC) Policy Evaluation

Monte Carlo policy evaluation is a rather straightforward estimation method that rely merely on experience gained through interaction with the environment.

In Monte-Carlo Policy Evaluation, an agent interacts with the environment, following a specific policy, and collects a sequence of state-action-reward trajectories. The value of a state is then estimated by averaging the returns obtained from all visits to that state. This process is repeated over multiple episodes, and the state values are updated after each episode.

The algorithm is also quite simple:

Algorithm 5 Monte-Carlo evaluation

Require: Initialize $V_0(s) = 0, \forall s \in \mathcal{S}$

repeat
  for $s \in \mathcal{S}$ **do**
    $N(S_t) \leftarrow N(S_t) + 1$
    $V(S_t) \leftarrow V(S_t) + \frac{1}{N(S_t)}(G_t - V(S_t))$
until Convergence

MC policy evaluation is good because it's super simple, easy to implement, and it learns only from experience. However, it also has some limitations: every single episodes must terminate in the sampling process; it requires a large number of samples to converge, and it does not exploit the Markov property through the Bellman equation.

1.3.2 Temporal Difference (TD) Evaluation

In temporal difference learning, an agent interacts with the environment and receives immediate rewards after each step. It updates the value estimates of states based on the difference between the current estimate and the estimated value of the subsequent state. The iteration equation should be:

$$V(S_t) \leftarrow V(S_t) + \alpha(R_{t+1} + \gamma V(S_{t+1}) - V(S_t)) \tag{1.3.1}$$

This is intuitively logical because for a true $V(S_t)$, it should satisfy the Bellman equation $V(S_t) = R_{t+1} + \gamma V(S_{t+1})$. Therefore, in TD evaluation, after a single step, we update $V(S_t)$ with a learning rate α, gradually pulling it towards satisfying the Bellman equation.

Specifically, we have the following definitions:

- $R_{t+1} + \gamma V(S_{t+1})$ is called **TD target**
- $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ is called **TD error**

Bias and Variance

Now we would revisit the idea of MC estimation and TD estimation from the view of bias. Recall the definition of *return* and state-value function.

$$G_t = r_t + \gamma r_{t+1} + \cdots + \gamma^{H-1} r_{t+H-1} \tag{1.3.2}$$

$$V(s) = \mathbb{E}[G_t | s_t = s] = \mathbb{E}\left[\sum_{k=0}^{H-1} \gamma^k r_{t+k} | s_t = s\right] \tag{1.3.3}$$

Then let's draw a comparison between MC estimation, TD estimation, and policy evaluation:

$$\begin{aligned}
\text{policy evaluation:} &\quad V(S_t) \leftarrow \mathbb{E}_\pi[R_{t+1} + \gamma V(S_{t+1})] \\
\text{MC estimation:} &\quad V(S_t) \leftarrow V(S_t) + \alpha(G_t - V(S_t)) \\
\text{TD estimation:} &\quad V(S_t) \leftarrow V(S_t) + \alpha(R_{t+1} + \gamma V(S_{t+1}) - V(S_t))
\end{aligned}$$

In MC estimation, the information source used to update $V(S_t)$ is G_t, which refers to the sampled return of the entire episode.

While in TD estimation, the information source used to update $V(S_t)$ is TD error, which is the information obtained after one-step forward sampling. This information contains the information of the entire episode since $V(S_{t+1})$ itself encompasses the expected reward of the entire trajectory. However, during the sampling process, we only rely on the next timestep's information (R_{t+1} and $V(S_{t+1})$).

In other word, return $G_t = r_t + \gamma r_{t+1} + \cdots + \gamma^{H-1} r_{t+H-1}$ is unbiased estimate of $v_\pi(s_t)$; true TD target $R_{t+1} + \gamma v_\pi(S_{t+1})$ is also unbiased estimate of $v_\pi(s_t)$; but the TD target $R_{t+1} + \gamma v(S_{t+1})$ is biased estimate of $v_\pi(s_t)$.

It also holds true that TD has lower variance than MC. This is because TD estimation updates its value estimates by using a bootstrapping approach that incorporates information from subsequent time steps, whereas MC estimation relies solely on the sampled return of the entire episode. This dependence on the full episode for each update leads to higher variance.

To sum up, MC has high variance, zero bias, while TD has low variance and some bias.

N-Step Return and Lamda-Return

We could also find something in the middle between TD (which only samples one step further) and MC (which samples the episode till the end), which is called n-step return.

$$\begin{aligned} n=1 \quad & G_t^{(1)} = R_{t+1} + \gamma V\left(S_{t+1}\right) \\ n=2 \quad & G_t^{(2)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 V\left(S_{t+2}\right) \\ & \vdots \\ n=\infty \quad & G_t^{(\infty)} = R_{t+1} + \gamma R_{t+2} + \ldots + \gamma^{T-1} R_T \end{aligned} \tag{1.3.4}$$

Similarly we have the definition of n-step return and n-step TD:

$$\begin{aligned} \text{n-step return} \quad & G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \ldots + \gamma^{n-1} R_{t+n} + \gamma^n V\left(S_{t+n}\right) \\ \text{n-step TD} \quad & V\left(S_t\right) \leftarrow V\left(S_t\right) + \alpha\left(G_t^{(n)} - V\left(S_t\right)\right) \end{aligned} \tag{1.3.5}$$

N-step return partially find a balance between TD and MC, but it still cannot provide a reasonable trade-off of variance and bias: if the initialization is too bad, then there is too much bias so I might want to do MC; while if value is already somewhat good, I'd prefer TD because it has less variance.

One possible solution is λ-return. The λ-return G_t^λ combines all n -step returns $G_t^{(n)}$ using weight $(1-\lambda)\lambda^{n-1}$

$$G_t^\lambda = (1-\lambda)\sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)} \tag{1.3.6}$$

And the corresponding time difference evaluation method is called forward-view TD(λ).

$$V\left(S_t\right) \leftarrow V\left(S_t\right) + \alpha\left(G_t^\lambda - V\left(S_t\right)\right) \tag{1.3.7}$$

This is reasonable because the sampling data from farther future time steps is given less weight, while we are more concerned with the sampling data from the nearer time steps.

Eligibility Trace

前向视角 TD(λ)需要直到episode结束才能获得所有n步回报，这使得它变成了一个离线算法，违背了我们希望在线更新的初衷

Actually there is something wrong with $TD(\lambda)$. We introduce the idea of time difference evaluation because we want to update with incomplete sequences without sampling the entire episode. But as λ-return G_t^λ combines all n -step returns $G_t^{(n)}$ using weight $(1-\lambda)\lambda^{n-1}$, we are dragged back to offline policy with episodic updates. To solve this problem we present a contrary view called *backward-view* $TD(\lambda)$

Note that there are two things that we want to take into account while making a policy:

- **frequency heuristics:** it prioritizes actions that have been chosen frequently in the past. This strategy assumes that actions that have been successful in the past are more likely to lead to positive outcomes in the future.
- **recency heuristics:** it prioritizes actions that have been successful recently. This strategy assumes that recent experiences are more indicative of the current state of the environment and that the agent should focus on exploiting recent successful actions to maximize its reward.

Recency 通过两个机制共同实现：
当一个状态被访问时，其资格迹立即增加1。
当一个状态未被访问时，其资格迹随时间指数级衰减。

Based on this, we have eligibility traces which combine both heuristics:

用于给出首项Ek(s)=1

$$\begin{aligned} E_0(s) &= 0 \\ E_t(s) &= \gamma\lambda E_{t-1}(s) + \mathbf{1}\left(S_t = s\right) \end{aligned} \tag{1.3.8}$$

Frequency 主要通过以下机制实现：
每次访问状态时，其资格迹都会累加1，使得频繁访问的状态保持较高的资格值。

$\mathbf{1}\left(S_t = s\right)$ here denotes an indicator which is set to 1 while $(S_t = s)$ and 0 otherwise.

It tells us that every past state has an influence on the current TD error by:

$$\delta_t = R_{t+1} + \gamma V\left(S_{t+1}\right) - V\left(S_t\right) \tag{1.3.9}$$

后向视角 TD(λ) 的更新：在每一步 t，计算当前的 TD 误差: δt，用个 δt更新 [过去所有状态 s] 的价值函数。
资格迹 (Et(s))是一个记忆机制，记录了状态 s 在过去被访问的频率和新近度，如果一个过去的状态 s 最近且频繁被访问，它的 Et (s) 值就很高，那么它就会在当前 δt 的更新中占有很大权重。

and therefore we can propagate the TD error now to all the past states (namely the backward-view) by:

$$V(s) \leftarrow V(s) + \alpha \delta_t E_t(s) \tag{1.3.10}$$

Next, we will show that in offline update, forward view and backward view are actually two different interpretation of the same mathematical result: Consider this case: state s is only visited at time k, then the eligibility trace would be:

$$E_t(s) = \gamma\lambda E_{t-1}(s) + \mathbf{1}(S_t = s) = \begin{cases} 0 & \text{if } t < k \\ (\gamma\lambda)^{t-k} & \text{if } t \geq k \end{cases} \tag{1.3.11}$$

Consequently for the entire episode, we have:

$$\sum_{t=1}^{T} \alpha \delta_t E_t(s) = \alpha \sum_{t=k}^{T} (\gamma\lambda)^{t-k} \delta_t = \alpha \left(G_k^{\lambda} - V(S_k) \right) \tag{1.3.12}$$

For general λ, TD errors also telescope to λ-error:

这是 λ-error，即前向视角 TD(λ) 的更新目标与当前价值的差距 。它包含了从 t 时刻开始，未来所有路径的综合信息

$$\begin{aligned}
G_t^{\lambda} - V(S_t) &= -V(S_t) + (1-\lambda)\lambda^0 \left(R_{t+1} + \gamma V(S_{t+1}) \right) \\
&\quad + (1-\lambda)\lambda^1 \left(R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2}) \right) \\
&\quad + (1-\lambda)\lambda^2 \left(R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 V(S_{t+3}) \right) \\
&\quad + \dots \\
&= -V(S_t)(\gamma\lambda)^0 \left(R_{t+1} + \gamma V(S_{t+1}) - \gamma\lambda V(S_{t+1}) \right) \\
&\quad + (\gamma\lambda)^1 \left(R_{t+2} + \gamma V(S_{t+2}) - \gamma\lambda V(S_{t+2}) \right) \\
&\quad + (\gamma\lambda)^2 \left(R_{t+3} + \gamma V(S_{t+3}) - \gamma\lambda V(S_{t+3}) \right) \\
&\quad + \dots \\
&\quad (\gamma\lambda)^0 \left(R_{t+1} + \gamma V(S_{t+1}) - V(S_t) \right) \\
&\quad + (\gamma\lambda)^1 \left(R_{t+2} + \gamma V(S_{t+2}) - V(S_{t+1}) \right) \\
&\quad + (\gamma\lambda)^2 \left(R_{t+3} + \gamma V(S_{t+3}) - V(S_{t+2}) \right) \\
&\quad + \dots \\
&= \dots \\
&= \delta_t + \gamma\lambda\delta_{t+1} + (\gamma\lambda)^2 \delta_{t+2} + \dots
\end{aligned} \tag{1.3.13}$$

它证明了，后向视角中“用当前误差 δt去更新过去所有状态”的方法，在累积一个完整的 episode 后，其效果与前向视角中“直接使用 λ-return 进行更新”是完全一样的 。而前者是一种离线 (offline) 算法 ； 资格迹通过[向后传播]单步的 TD 误差，即[在 当前 t 时刻计算出的误差 δt，会被用来更新 过去 所有访问过的状态的价值]，实现了在线 (online) 更新，节省内存、支持即时更新价值函数。

代表一连串单步 TD 误差的和

This shows that after one whole episode, forward and backward would give the same results.

1.3.3 Q-Learning: Off-Policy TD Learning

In MC and TD methods, we solve the problem of not knowing value function by estimation based on experiences. However, in policy improvement:$\pi_{i+1}(s) = \arg\max_a \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma V^{\pi_i}(s') \right]$ we still need the transition model to decide on the actions. Q learning solve this problem by estimating the Q-function instead of value function, which directly take action into consideration.

Q-learning is a foundational method for reinforcement learning. It is a TD method that estimates the future reward $V(s')$ using the *Q-function* itself, assuming that from state s', the *best* action (according to the Q function) will be executed at each state.

Need to explain more about Q learning and sarsa (why we should set value function to be the argmax of Q function, why iterating over the error term would help improve the policy, the highlevel idea of SARSA and Q-learning).

Therefore, we have the Bellman equation expressed with Q function:

$$\begin{aligned}
Q(s,a) &= R(s) + \gamma \sum_{s'} T(s,a,s') V(s') \\
&= R(s) + \gamma \sum_{s'} T(s,a,s') \max_{a'} Q(s',a')
\end{aligned} \tag{1.3.14}$$

TD, MC, 和 TD(λ) 都属于 Model-Free 学习，它们被设计出来的初衷，就是为了解决我们 不知道 环境模型 T(s,a,s′) 和奖励函数 R(s,a) 的情况。它们产出的是 状态价值函数 V(s)。但在不知道环境模型的情况下，仅有 V(s) 不足以让我们判断哪个 动作 (action) 是更好的，因此无法直接进行策略改进。要找到一个更好的策略，我们需要对每个动作 a 进行评估，看看它通过 T(s,a,s′)能把我们带到哪个后续状态 s'，然后结合后续状态的价值 V(s′) 来做出最优选择。

Algorithm 6 Q-learning

Require: Initialize Q function (e.g. $Q(s,a)=0, \forall s \in \mathcal{S}, \forall a \in \mathcal{A}$).

1: **repeat**
2: Take action $a$according to some exploitation/exploration policy given state s
3: Observe reward r and new state s'
4: Update $Q(s,a)$ using the Bellman equation:

$$Q'(s,a) \leftarrow Q(s,a) + \alpha(r + \gamma \max_{a'} Q(s',a') - Q(s,a))$$

5: **until** Convergence

Q-learning (以及 SARSA) 的核心思想是，绕开对 V(s) 和环境模型的依赖，直接学习一个不同的价值函数——动作价值函数 (Action-Value Function) Q(s,a)，直接包含了对动作 a 的评估，隐式地学习了环境的动态特性，不再需要明确的状态转移概率 T(s,a,s′)

Note that we estimate the future value using $\max_a Q(s',a)$, which means it ignores the actual next action that will be executed, and updates based on the estimated best action. This is known as *off-policy* learning.

Definition (On-Policy and Off-Policy). If an RL algorithm does not depend on the actual next action(s), it is *off-policy*. Otherwise, it is *on-policy*.

区别在于：在更新价值函数时，我们用来计算“未来期望回报”的那个“下一步动作”是我们实际要走的，还是我们想象中最好的？

Theorem 1.5 (Convergence of Q-Learning). Q-learning converges to the optimal Q-value function in the limit with probability 1, if every state-action pair is visited infinitely often (Jaakkola et al., 1993).

Proof.

Proof of convergence of Q-learning

□

1.3.4 SARSA: On-Policy TD Learning

SARSA (State-action-reward-state-action) is an on-policy reinforcement learning algorithm. It is very similar to Q-learning, except that in its update rule, it actually selects the next action that it will execute, and updates accordingly. Taking this approach is known as *on-policy* reinforcement learning.

Algorithm 7 SARSA

Require: Initialize Q function (e.g. $Q(s,a)=0, \forall s \in \mathcal{S}, \forall a \in \mathcal{A}$).

1: **repeat**
2: Take action a_n according to some exploitation/exploration policy given state s_n
3: Observe reward $r(s_n,a_n)$ and next state s_{n+1}
4: Select action a_{n+1}according to some policy and update Q_{n+1} using:

$$Q'(s_n,a_n) = Q(s_n,a_n) + \alpha\left[r(s_n,a_n) + \gamma Q(s_{n+1},a_{n+1}) - Q(s_n,a_n)\right]$$

5: **until** Convergence

1.4 Exploration strategies

In previous discussions we always use the greedy policy, where we take

$$\pi\left(\mathbf{a}_t \mid \mathbf{s}_t\right) = \begin{cases} 1 \text{ if } \mathbf{a}_t = \arg\max_{\mathbf{a}_t} Q_\phi\left(\mathbf{s}_t, \mathbf{a}_t\right) \\ 0 \text{ otherwise} \end{cases} \tag{1.4.1}$$

However, while learning is progressing, using the greedy policy might not be the best solution. Part of why we might not want to use greedy policy in step one of online Q iteration is that this argmax policy is deterministic. And if our intial Q function is quite bad, we might be stuck in taking a bad action, failing to discover any other probably better actions. In practice, we introduce randomness to avoid this situation. One common choice is called ϵ **- greedy**:

$$\pi\left(\mathbf{a}_t \mid \mathbf{s}_t\right) = \begin{cases} 1-\epsilon \text{ if } \mathbf{a}_t = \arg\max_{\mathbf{a}_t} Q_\phi\left(\mathbf{s}_t, \mathbf{a}_t\right) \\ \epsilon/(|\mathcal{A}|-1) \text{ otherwise} \end{cases} \tag{1.4.2}$$

It's a very simple exploration rule and very useful in practice. Another way of doing this is matching the probability of taking an action with the goodness of the value obtained by calculating that action. This is what **Boltzmann exploration** (also called the softmax rule) do:

$$\pi\left(\mathbf{a}_t \mid \mathbf{s}_t\right) \propto \exp\left(Q_\phi\left(\mathbf{s}_t, \mathbf{a}_t\right)\right) \tag{1.4.3}$$

Boltzmann exploration is to some extent smarter than ϵ - greedy, since if two action are almost as good, in ϵ - greedy you hardly have chance to explore the second best one, while in Boltzmann exploration they share similar probability of being explored. Also in Boltzmann exploration you don't waste time on actions that are pretty bad.

Note that exploration in reinforcement learning is still an active research area. And we will touch a little bit deeper later.

1.5 Exercises

1.5.1 Reward Choices

This problem is adapted from Assignment 1 of CS234 (2023 Spring).

Consider a tabular MDP with $0 < \gamma < 1$ and no terminal states. The agent will act forever. The original optimal value function for this problem is V_1^* and the optimal policy is π_1^*.

(a) Now someone decides to add a small reward bonus c to all transitions in the MDP. This results in a new reward function $\hat{r}(s,a) = r(s,a) + c; \forall s, a$ where $r(s,a)$ is the original reward function. What is an expression for the new optimal value function? Can the optimal policy in this new setting change? Why or why not?

(b) Instead of adding a small reward bonus, someone decides to multiply all rewards by an arbitrary constant $c \in \mathbb{R}$. This results in a new reward function $\hat{r}(s,a) = c \times r(s,a); \forall s, a$ where $r(s,a)$ is the original reward function. Are there cases in which the new optimal policy is still π_1^* and the resulting value function can be expressed as a function of c and V_1^* (if yes, give the resulting expression and explain for what value(s) of c and why? If not, explain why not)? Are there cases where the optimal policy would change (if yes, provide a value of c and a description of why the optimal policy would change. If not, explain why not)? Is there a choice of c such that all policies are optimal?

(c) If the MDP instead has terminal states that can end an episode, does that change your answer to part (a)? If yes, provide an example MDP where your answers to part (a) and this part would differ.

1.5.2 Markov Decision Process

This problem is adapted from Problem Session 1 of CS234 (2023 Spring).

For all the three questions below, define $V^\pi(s)$ as the expected discounted reward starting from state s and following policy π.

$$V^\pi(s) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t \mathcal{R}\left(s_t, a_t\right) \mid s_0 = s, \pi\right]$$

Q1

For an arbitrary $Z \in \mathbb{N}$, consider learning with $Z+1$ distinct discount factors $\gamma_0, \gamma_1, \ldots, \gamma_Z$ where the final discount factor matches that of the MDP $\mathcal{M}, \gamma_Z = \gamma$. Letting $[Z] \triangleq \{1, 2, \ldots, Z\}$ denote the index set, we define the following functions for any policy π :

$$V_{\gamma_z}^\pi = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma_z^t \mathcal{R}\left(s_t, a_t\right) \mid s_0 = s, \pi\right] \quad W_z^\pi = V_{\gamma_z}^\pi - V_{\gamma_{z-1}}^\pi, \quad \forall z \in [Z]$$

where $W_0 = V_{\gamma_0}^\pi$.

(a) For any $z \in [Z]$; any policy $\pi : \mathcal{S} \to \Delta(\mathcal{A})$; and any $s \in \mathcal{S}$, write an expression for $V_{\gamma_z}^\pi(s)$ exclusively in terms of $\{W_0^\pi, W_1^\pi, \ldots, W_Z^\pi\}$.

(b) Show that W_z^π obeys the following Bellman equation for any $z \in [Z]$ and $s \in \mathcal{S}$:

$$W_z^\pi(s) = \mathbb{E}_{\substack{a\sim\pi(\cdot|s)\\ s'\sim\mathcal{T}(\cdot|s,a)}}\left[(\gamma_z - \gamma_{z-1})\, V_{\gamma_{z-1}}^\pi(s') + \gamma_z W_z(s')\right]$$

Q2

Let $\gamma, \beta \in [0,1)$ be two discount factors such that $\beta \leq \gamma$. Let $\pi : \mathcal{S} \to \Delta(\mathcal{A})$ be an arbitrary policy that induces value functions V_γ^π and V_β^π under the two discount factors, respectively. Similarly, define the Bellman operators

$$\mathcal{B}_\gamma^\pi V(s) = \mathbb{E}_{a\sim\pi(\cdot|s)}\left[\mathcal{R}(s,a) + \gamma \mathbb{E}_{s'\sim\mathcal{T}(\cdot|s,a)}\left[V(s')\right]\right]$$
$$\mathcal{B}_\beta^\pi V(s) = \mathbb{E}_{a\sim\pi(\cdot|s)}\left[\mathcal{R}(s,a) + \beta \mathbb{E}_{s'\sim\mathcal{T}(\cdot|s,a)}\left[V(s')\right]\right].$$

With the reward upper bound $R_{\text{MAX}} = \max_{(s,a)\in\mathcal{S}\times\mathcal{A}} \mathcal{R}(s,a)$, prove that

$$\left\|V_\gamma^\pi - V_\beta^\pi\right\|_\infty \leq \frac{(\gamma-\beta)R_{\text{MAX}}}{(1-\gamma)(1-\beta)}.$$

Q3

Let $\alpha, \gamma \in [0,1)$ be two discount factors such that $\gamma \leq \alpha$. Consider a new MDP $\mathcal{M}' = \langle \mathcal{S}, \mathcal{A}, \mathcal{T}', \mathcal{R}, \alpha \rangle$ with a different transition function $\mathcal{T}' : \mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{S})$ defined for $\lambda \in [0,1]$ as

$$\mathcal{T}'(s' \mid s,a) = (1-\lambda)\mathcal{T}(s' \mid s,a) + \lambda \mathbf{1}(s = s'), \quad \forall (s,a,s') \in \mathcal{S}\times\mathcal{A}\times\mathcal{S}.$$

In words, the new transition function $\mathcal{T}'$ follows the transitions of the original MDP $\mathcal{T}$ with probability $(1-\lambda)$ and takes a self-looping transition with probability λ. We will use subscripts to distinguish between value functions of $\mathcal{M}$ versus those of $\mathcal{M}'$.Assuming that both $\mathcal{M}$ and $\mathcal{M}'$ are tabular, recall the matrix form of the Bellman equations for any policy π:

$$V_{\mathcal{M}}^\pi = (I - \gamma\mathcal{T}^\pi)^{-1}\mathcal{R}^\pi \quad V_{\mathcal{M}'}^\pi = (I - \alpha\mathcal{T}'^\pi)^{-1}\mathcal{R}^\pi,$$

where

$$\mathcal{R}^\pi(s) = \mathbb{E}_{a\sim\pi(\cdot|s)}[\mathcal{R}(s,a)] \quad \mathcal{T}^\pi(s' \mid s) = \mathbb{E}_{a\sim\pi(\cdot|s)}\left[\mathcal{T}(s' \mid s,a)\right] \quad \mathcal{T}'^\pi(s' \mid s) = \mathbb{E}_{a\sim\pi(\cdot|s)}\left[\mathcal{T}'(s' \mid s,a)\right]$$

(a) Give a value of λ such that, for any policy π,

$$V_{\mathcal{M}'}^\pi = \frac{1-\gamma}{1-\alpha} \cdot V_{\mathcal{M}}^\pi$$

(b) If $\pi^\star$ is the optimal policy of MDP $\mathcal{M}$, prove that $\pi^\star$ is also optimal in $\mathcal{M}'$.

1.5.3 Bellman Residuals and Performance Bounds

This problem is adapted from Assignment 1 of CS234 (2023 Spring).

In this problem, we will study **Bellman residuals**, which we will define below, and how it helps us bound the performance of a policy. But first, some recap and definitions.

Recall that the Bellman backup operator B defined below is a contraction with the fixed point as V^*, the optimal value function of the MDP. The symbols have their usual meanings. γ is the discount factor and $0 \leq \gamma < 1$. In all parts, $\|v\|$ is the infinity norm of the vector.

$$(BV)(s) = \max_a \left[r(s,a) + \gamma \sum_{s'\in S} p(s' \mid s,a)\, V(s')\right]$$

We also saw the contraction operator B^π with the fixed point V^π, which is the Bellman backup operator for a particular policy given below:

$$\left(B^{\pi}V\right)(s) = \mathbb{E}_{a\sim\pi}\left[r(s,a) + \gamma \sum_{s'\in S} p\left(s' \mid s,a\right) V\left(s'\right)\right]$$

Previously, we showed that $\|BV - BV'\| \leq \gamma \|V - V'\|$ for two arbitrary value functions V and V'. For the rest of this question, you can also assume that $\|B^{\pi}V - B^{\pi}V'\| \leq \gamma \|V - V'\|$. The proof of this is very similar to the proof we saw in class for B.

(a) Prove that the fixed point for B^{π} is unique.

We can recover a greedy policy π from an arbitrary value function V using the equation below.

$$\pi(s) = \arg\max_{a}\left[r(s,a) + \gamma \sum_{s'\in S} p\left(s' \mid s,a\right) V\left(s'\right)\right]$$

(b) When π is the greedy policy, what is the relationship between B and B^{π} ?

Insight. The *motivation* is that it is often helpful to know what the performance will be if we extract a greedy policy from a value function. In the rest of this problem, we will prove a bound on this performance.

Recall that a value function is a $|S|$-dimensional vector where $|S|$ is the number of states of the MDP. When we use the term V in these expressions as an "arbitrary value function", we mean that V is an arbitrary $|S|$-dimensional vector which need not be aligned with the definition of the MDP at all. On the other hand, V^{π} is a value function that is achieved by some policy π in the MDP. For example, say the MDP has 2 states and only negative immediate rewards. $V = [1, 1]$ would be a valid choice for V even though this value function can never be achieved by any policy π, but we can never have a $V^{\pi} = [1, 1]$. This distinction between V and V^{π} is important for this question and more broadly in reinforcement learning.

Why do we care about setting V to vectors than can never be achieved in the MDP? Sometimes algorithms, such as Deep Q-networks, return such vectors. In such situations we may still want to extract a greedy policy π from a provided V and bound the performance of the policy we extracted, aka V^{π}.

Definition (Bellman Residual). The Bellman residual is defined as $BV - V$ and the Bellman error magnitude is defined as $\|BV - V\|$.

Insight. Bellman residual is an important component of several popular RL algorithms such as the Deep Q-networks, referenced above. Intuitively, you can think of it as the difference between what the value function V specifies at a state and the one-step look-ahead along the seemingly best action at the state using the given value function V to evaluate all future states (the BV term).

(c) For what V does the Bellman error magnitude equal 0?

(d) Prove the following statements for an arbitrary value function V and any policy π. [Hint: Try leveraging the triangle inequality by inserting a zero term.]

$$\|V - V^{\pi}\| \leq \frac{\|V - B^{\pi}V\|}{1-\gamma}$$
$$\|V - V^{*}\| \leq \frac{\|V - BV\|}{1-\gamma}$$

The result you proved in part (d) will be useful in proving a bound on the policy performance in the next few parts. Given the Bellman residual, we will now try to derive a bound on the policy performance, V^{π}.

(e) Let V be an arbitrary value function and π be the greedy policy extracted from V. Let $\varepsilon = \|BV - V\|$ be the Bellman error magnitude for V. Prove the following for any state s. [Hint: Try to use the results from part (d).]

$$V^{\pi}(s) \geq V^{*}(s) - \frac{2\varepsilon}{1-\gamma}$$

A little bit more notation: define $V \leq V'$ if $\forall s, V(s) \leq V'(s)$. What if our algorithm returns a V that satisfies $V^* \leq V$, i.e., it returns a value function that is better than the optimal value function of the MDP. Once again, remember that V can be any vector, not necessarily achievable in the MDP but we would still like to bound the performance of V^π where π is extracted from said V. We will show that if this condition is met, then we can achieve an even tighter bound on policy performance.

(f) Using the same notation and setup as part (e), if $V^* \leq V$, show the following holds for any state s. [Hint: Recall that $\forall \pi, V^\pi \leq V^*$.]

$$V^\pi(s) \geq V^*(s) - \frac{\varepsilon}{1-\gamma}$$

(g) It's not easy to show that the condition $V^* \leq V$ holds because we often don't know V^* of the MDP. Show that if $BV \leq V$ then $V^* \leq V$. Note that this sufficient condition is much easier to check and does not require knowledge of V^*. [Hint: Try to apply induction. What is $\lim_{n\to\infty} B^n V$?]

Insight. A useful way to interpret the results from parts (e) (and (f)) is based on the observation that a constant immediate reward of r at every time-step leads to an overall discounted reward of $r + \gamma r + \gamma^2 r + \ldots = \frac{r}{1-\gamma}$. Thus, the above results say that a state value function V with Bellman error magnitude ε yields a greedy policy whose reward per step (on average), differs from optimal by at most 2ε. So, if we develop an algorithm that reduces the Bellman residual, we're also able to bound the performance of the policy extracted from the value function outputted by that algorithm, which is very useful!

(h) (**Challenge**) It is possible to make the bounds from parts (e) and (f) tighter. Try to prove the following if you're interested.

Let V be an arbitrary value function and π be the greedy policy extracted from V. Let $\varepsilon = \|BV - V\|$ be the Bellman error magnitude for V. Prove the following for any state s :

$$V^\pi(s) \geq V^*(s) - \frac{2\gamma\varepsilon}{1-\gamma}$$

Further, if $V^* \leq V$, prove for any state s

$$V^\pi(s) \geq V^*(s) - \frac{\gamma\varepsilon}{1-\gamma}$$

1.5.4 The Error Bound

This problem is adapted from hw3 of CS285.

Consider the problem of imitation learning within a discrete MDP with horizon T and an expert policy π^*. We gather expert demonstrations from π^* and fit an imitation policy π_θ to these trajectories so that

$$\mathbb{E}_{p_{\pi^*}(s)} \pi_\theta \left(a \neq \pi^*(s) \mid s\right) = \frac{1}{T} \sum_{t=1}^{T} \mathbb{E}_{p_{\pi^*}(s_t)} \pi_\theta \left(a_t \neq \pi^*\left(s_t\right) \mid s_t\right) \leq \varepsilon,$$

i.e., the expected likelihood that the learned policy π_θ disagrees with the expert π^* within the training distribution p_{π^*} of states drawn from random expert trajectories is at most ε.

For convenience, the notation $p_\pi\left(s_t\right)$ indicates the state distribution under π at time step t while $p(s)$ indicates the state marginal of π across time steps, unless indicated otherwise.

Q1

Show that $\sum_{s_t} \left|p_{\pi_\theta}\left(s_t\right) - p_{\pi^*}\left(s_t\right)\right| \leq 2T\varepsilon$. Hints:

- In lecture, we showed a similar inequality under the stronger assumption $\pi_\theta\left(s_t \neq \pi^*\left(s_t\right) \mid s_t\right) \leq \varepsilon$ for every $s_t \in \operatorname{supp}\left(p_{\pi^*}\right)$. Try converting the inequality above into an expectation over p_{π^*}.
- Use the union bound inequality: for a set of events $E_i, \Pr\left[\bigcup_i E_i\right] \leq \sum_i \Pr\left[E_i\right]$.

Q2

Consider the expected return of the learned policy π_θ for a state-dependent reward $r(s_t)$, where we assume the reward is bounded with $|r(s_t)| \leq R_{\max}$:

$$J(\pi) = \sum_{t=1}^{T} \mathbb{E}_{p_\pi(s_t)} r(s_t)$$

(a) Show that $J(\pi^*) - J(\pi_\theta) = \mathcal{O}(T\varepsilon)$ when the reward only depends on the last state, i.e., $r(s_t) = 0$ for all $t < T$.

(b) Show that $J(\pi^*) - J(\pi_\theta) = \mathcal{O}\left(T^2\varepsilon\right)$ for an arbitrary reward.

在所有转移中增加一个常数奖励 c

对于任意一个给定的策略 π，其状态价值函数是未来折扣奖励的期望值。在新的奖励函数 $\hat{r}$ 下，价值函数 $\hat{V}^{\pi}(s)$ 为：

$\hat{V}^{\pi}(s) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t \hat{r}(s_t, a_t) \middle| s_0 = s, \pi\right]$

将 $\hat{r}(s,a) = r(s,a) + c$ 代入：

$\hat{V}^{\pi}(s) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t (r(s_t, a_t) + c) \middle| s_0 = s, \pi\right]$

$= \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \middle| s_0 = s, \pi\right] + \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t c \middle| s_0 = s, \pi\right]$

第一项正是原始奖励函数下的价值函数 $V^{\pi}(s)$。第二项是一个常数 c 的几何级数求和：

$c \sum_{t=0}^{\infty} \gamma^t = c \cdot \frac{1}{1-\gamma}$

所以，对于任意策略 π，我们有 $\hat{V}^{\pi}(s) = V^{\pi}(s) + \frac{c}{1-\gamma}$。

最优价值函数是所有策略中能达到的最大价值函数，因此：

$\hat{V}^{*}(s) = \max_{\pi} \hat{V}^{\pi}(s) = \max_{\pi} \left(V^{\pi}(s) + \frac{c}{1-\gamma} \right) = (\max_{\pi} V^{\pi}(s)) + \frac{c}{1-\gamma}$

最终我们得到：

$\hat{V}^{*}(s) = V_1^{*}(s) + \frac{c}{1-\gamma}$

最优策略 **不会** 改变。

最优策略是对于最优动作价值函数 $Q^{*}(s,a)$ 采取贪心策略的结果，即 $\pi^{*}(s) = \arg\max_a Q^{*}(s,a)$。我们来分析新的最优Q函数 $\hat{Q}^{*}(s,a)$。

根据贝尔曼最优方程：

$\hat{Q}^{*}(s,a) = \hat{r}(s,a) + \gamma \sum_{s'} p(s'|s,a) \hat{V}^{*}(s')$

将我们上面得到的结果代入：

$\hat{Q}^{*}(s,a) = (r(s,a) + c) + \gamma \sum_{s'} p(s'|s,a) \left(V_1^{*}(s') + \frac{c}{1-\gamma} \right)$

$= r(s,a) + c + \gamma \sum_{s'} p(s'|s,a) V_1^{*}(s') + \gamma \sum_{s'} p(s'|s,a) \frac{c}{1-\gamma}$

因为 $\sum_{s'} p(s'|s,a) = 1$：

$\hat{Q}^{*}(s,a) = (r(s,a) + \gamma \sum_{s'} p(s'|s,a) V_1^{*}(s')) + c + \frac{\gamma c}{1-\gamma}$

第一项正是原始的最优Q函数 $Q_1^{*}(s,a)$。所以：

$\hat{Q}^{*}(s,a) = Q_1^{*}(s,a) + c\left(1 + \frac{\gamma}{1-\gamma}\right) = Q_1^{*}(s,a) + \frac{c}{1-\gamma}$

在任何状态 s 下，新的Q函数 $\hat{Q}^{*}(s,a)$ 只是在旧的Q函数 $Q_1^{*}(s,a)$ 的基础上，为**所有**动作 a 增加了**同一个常数** $\frac{c}{1-\gamma}$。这个常数不改变动作之间的相对优劣顺序。因此，能使 $Q_1^{*}(s,a)$ 最大化的动作，同样也能使 $\hat{Q}^{*}(s,a)$ 最大化。

$\arg\max_a \hat{Q}^{*}(s,a) = \arg\max_a \left(Q_1^{*}(s,a) + \frac{c}{1-\gamma} \right) = \arg\max_a Q_1^{*}(s,a)$

所以，最优策略 π_1^{*} 保持不变。

将所有奖励乘以一个常数 c

1. 当 $c > 0$ 时，最优策略不变，且 $\hat{V}^*(s) = c \cdot V_1^*(s)$。
   - **推导**: 类似于(a)的推导，对于任意策略 π，$\hat{V}^\pi(s) = c \cdot V^\pi(s)$。新的Q函数为 $\hat{Q}^*(s,a) = c \cdot r(s,a) + \gamma \sum_{s'} p(s'|s,a)(c \cdot V^*(s')) = c \cdot Q_1^*(s,a)$。当 $c > 0$ 时，$\arg\max_a (c \cdot Q_1^*(s,a)) = \arg\max_a Q_1^*(s,a)$，所以策略不变。
2. 当 $c < 0$ 时，最优策略通常会改变。
   - **原因**: 当 c 为负数时，最大化 $\hat{Q}^*(s,a)$ 等同于最大化 $c \cdot Q_1^*(s,a)$，这相当于**最小化** $Q_1^*(s,a)$（因为c是负数）。原来的最优动作会变成最差的动作，而原来的最差动作可能会变成最优的。例如，如果原有两个动作的Q值分别为10和5，最优动作是前者；乘以-1后，Q值变为-10和-5，最优动作就变成了后者。
3. 当 $c = 0$ 时，所有策略都是最优策略。
   - **原因**: 当 $c = 0$ 时，所有奖励 $\hat{r}(s,a)$ 都变为0。因此，对于任何策略，所有状态的价值都是0。任何状态下的任何动作的Q值也都是0。既然所有动作的价值都相同，那么无论选择哪个动作都是最优的。因此，所有策略都成为了最优策略。

存在终止状态时的影响

在有终止状态的情况下，给所有转移增加一个常数奖励 c **可以改变最优策略**。

在没有终止状态的无限回合模型中，每个策略下的未来步数期望都是无限的，所以增加的累积奖励 $\frac{c}{1-\gamma}$ 对所有策略都是一个固定的值。

但在有终止状态的情况下，**不同策略可能导致不同的期望回合长度**。一个倾向于“规避风险”并尽可能延长回合的策略，会比一个“激进冒险”并可能快速导致回合终止的策略，经历更多的状态转移。

当我们给每个转移都增加一个正的奖励 $c > 0$ 时，这就相当于给了“存活”这个行为一个额外的奖励。那些能够让智能体“活得更久”（即推迟进入终止状态）的策略，会累积更多的额外奖励 c。如果这个累积的额外奖励足够大，就可能超过一个高风险高回报策略所带来的原始奖励，从而使得原本次优的“求稳”策略变为最优策略。

示例 MDP:

- **状态**: S_0 (起始状态), S_T (终止状态, 价值为0)。
- **动作 (在S_0)**:
  - a_1 (“安全”): 获得低奖励 $r(S_0, a_1) = 1$，有90%的概率留在 S_0，10%的概率进入 S_T。
  - a_2 (“冒险”): 获得高奖励 $r(S_0, a_2) = 10$，有90%的概率进入 S_T，10%的概率留在 S_0。
- **折扣因子**: $\gamma = 0.9$。

原始情况 (c=0):
通过计算可以发现，从长远看，“冒险”动作 a_2 的期望回报更高，是初始的最优策略 π_1^*。

- $V^{\pi_2}(S_0) = 10 + 0.9 \cdot (0.1 V^{\pi_2}(S_0)) \implies 0.91 V^{\pi_2}(S_0) = 10 \implies V^{\pi_2}(S_0) \approx 10.99$
- $V^{\pi_1}(S_0) = 1 + 0.9 \cdot (0.9 V^{\pi_1}(S_0)) \implies 0.19 V^{\pi_1}(S_0) = 1 \implies V^{\pi_1}(S_0) \approx 5.26$
  最优策略是选择 a_2。

增加奖励 (c=10):
现在 $\hat{r}(S_0, a_1) = 11$，$\hat{r}(S_0, a_2) = 20$。

- 选择 a_2 的新价值: $\hat{V}^{\pi_2}(S_0) = 20 + 0.09 \hat{V}^{\pi_2}(S_0) \implies 0.91 \hat{V}^{\pi_2}(S_0) = 20 \implies \hat{V}^{\pi_2}(S_0) \approx 21.98$

- 选择 a_1 的新价值: $\hat{V}^{\pi_1}(S_0) = 11 + 0.81\hat{V}^{\pi_1}(S_0) \implies 0.19\hat{V}^{\pi_1}(S_0) = 11 \implies \hat{V}^{\pi_1}(S_0) \approx 57.89$

现在，$\hat{V}^{\pi_1}(S_0) > \hat{V}^{\pi_2}(S_0)$。最优策略变成了选择“安全”动作 a_1。策略发生了改变。原因是“安全”动作 a_1 让智能体有更大概率停留在游戏中，从而不断地累积那个值为10的额外奖励 c，这个长期收益最终超过了“冒险”动作带来的单次高额奖励。

Chapter 2 Value-based Methods

In the Sec. 1.3, we talked about Q-learning and SARSA, which are both value-based methods (i.e., try to learn the value function in the first place, and then derive a corresponding policy based on the value function). Why do we start a new chapter here on value-based methods?

Think about the game of go. There are about 10^{170} states. If we want to use Q-learning and SARSA, we have to maintain a Q-table for $|\mathcal{S}| \times |\mathcal{A}|$ size, which is impractical. In addition, in previous discussions, we have been using dynamic programming in tabular cases as an example, and have only dealt with scenarios where the state space and action space are discrete. What if we need to handle continuous cases? The solution here is instead of maintaining a table storing Q values, we want to approximate the value function $V^{\pi}(\mathbf{s})$ or state-action function $Q^{\pi}(\mathbf{s}, \mathbf{a})$. Then, to represent functions, neural networks come into play.

2.1 Value Function Approximation

Using a Q-table has two main limitations:

1. It requires that we visit every reachable state many times and apply every action many times to get a good estimate of $Q(\mathbf{s}, \mathbf{a})$.Thus, if we never visit a state $\mathbf{s}$, we have no estimate of $Q(\mathbf{s}, \mathbf{a})$, even if we have visited states that are very similar to $\mathbf{s}$.
2. It requires us to maintain a table of size $|\mathcal{S}| \times |\mathcal{A}|$ size, which is prohibitively large for any non-trivial problem.

To get around these we will look at how to use machine learning to approximate Q-functions. Instead of calculating an exact Q-function, we approximate it using simple methods that both eliminate the need for a large Q-table (therefore the methods scale better), and also allowing use to provide reasonable estimates of $Q(\mathbf{s}, \mathbf{a})$, even if we have not applied action $\mathbf{a}$ in state $\mathbf{s}$ previously.

There are a variety of methods other than neural nets to approximate a function, e.g. linear combinations of features, decision trees, nearest neighbor, Fourier / wavelet bases, etc.. They have their own unique properties, for example Fourier transformation would be particularly useful if you care about frequency information. But here we would like to care more about the differentiable ones. So in particular, we will look at linear function approximation and approximation using deep learning (deep Q-learning).

Linear Function Approximation

In linear Q-learning, we store features and weights, not states. What we need to learn is how important each feature is (its weight) for each action.

将Q值表示为一组特征 (features) 的线性加权和，线性函数近似法中，我们学习的目标，就是找到最优的权重向量 w

To represent this, we have two vectors:

1. A *feature vector* $f(s, a)$, which is a vector of $n \cdot |A|$ different functions, where n is the number of state features and $|A|$ the number of actions. Each function extracts the value of a feature for state-action pair (s, a). We say $f_i(s, a)$ extracts the i-th feature from the state-action pair (s, a) :

$$f(s,a) = \begin{pmatrix} f_1(s,a) \\ f_2(s,a) \\ \dots \\ f_{n\times|A|}(s,a) \end{pmatrix}$$

每个特征 fi(s,a) 代表了 (s, a) 的某一种属性或特性。例如，在吃豆人游戏中，一个特征可以是“执行‘向右’动作后离最近的豆子有多远”

2. A *weight vector* w of size n $\times|A|$: one weight for each feature-action pair. w_i^a defines the weight of a feature i for action a.

一个状态-动作对的价值，等于它的所有特征值与其对应重要性的乘积之和。

Defining State-Action Features

Often it is easier to just define features for states, rather than state-action pairs. The features are just a vector of n functions of the form $f_i(s)$. It is straightforward to construct $n \times |A|$ state-pair features from just n state features:

$$f_{i,k}(s,a) = \begin{cases} f_i(s) & \text{if } a = a_k \\ 0 & \text{otherwise } 1 \le i \le n, 1 \le k \le |A| \end{cases} \tag{2.1.1}$$

This effectively results in $|A|$ different weight vectors:

当动作是 a_1 时，最终的特征向量 $f(s,a_1)$ 是：

$$f(s,a_1) = \begin{pmatrix} f_1(s) \\ f_2(s) \\ \vdots \\ f_n(s) \\ 0 \\ 0 \\ \vdots \end{pmatrix}$$

（状态特征 $f(s)$ 出现在第一块，其余部分为0）

当动作是 a_2 时，最终的特征向量 $f(s,a_2)$ 是：

$$f(s,a_2) = \begin{pmatrix} 0 \\ \vdots \\ 0 \\ f_1(s) \\ \vdots \\ f_n(s) \\ 0 \\ \vdots \end{pmatrix}$$

（状态特征 $f(s)$ 出现在第二块，其余部分为0）

(2.1.2)

这种转化方法最直接和重要的效果是，**它等价于为每一个动作学习一套独立的权重** (This effectively results in |A| different weight vectors) 。

让我们回顾一下Q值的计算公式：$Q(s,a) = \sum f_i(s,a)w_i$。

- 当我们计算 $Q(s,a_1)$ 时，只有新特征向量的前 `n` 个元素（第一块）有值，后面都是0。所以，只有权重向量 `w` 的前 `n` 个权重 $w_1, \ldots, w_n$ 会参与计算。
- 当我们计算 $Q(s,a_2)$ 时，只有新特征向量的第 `n+1` 到 `2n` 个元素（第二块）有值。所以，只有权重向量 `w` 的第 `n+1` 到 `2n` 个权重会参与计算。

这样一来，虽然我们表面上只维护一个长长的权重向量 `w`，但实际上，我们为动作 a_1 学习了一套权重（w的前n个），为动作 a_2 学习了另一套独立的权重（w的中间n个），以此类推。这使得算法可以学习到，**同样的状态特征（比如“离敌人很近”）对于不同的动作（比如“攻击”和“逃跑”）具有完全不同的重要性**。

Linear Q-function Update

To use approximate Q-functions in reinforcement learning, there are two steps we need to change from the standard algorithsm: (1) initialisation; and (2) update.

For initialisation, initialise all weights to 0. Alternatively, you can try Q-function initialisation and assign weights that you think will be “good” weights.

For update, we now need to update the weights instead of the Q-table values. The update rule is now:

(TD-error)项代表了我们当前Q值预测的“错误程度”，权重的更新量与特征值本身的大小成正比

$$\text{For each state-action feature,} \quad w_i^a \leftarrow w_i^a + \alpha \cdot \delta \cdot f_i(s,a) \tag{2.1.4}$$

where δ depends on which algorithm we are using; e.g. Q-learning, SARSA. As this is linear, it is therefore convex, so the weights will converge. In the next section we will show how to merge this value approximation method into Q learning algorithm.

There might be something wrong with the algorithm here. What's the difference between Q iteration and Q learning? Besides, let's standardize the notation used in the algorithms (compared to the way used in Q learning and SARSA in previous chapter).

2.2 Value Functions in Theory

This part would be kind of aside the main discussion and a little bit *maths*. In previous discussions we mentioned that value functions does not have any convergence guarantees that it will ultimately find the optimal solution. Here we will find out more about this theoretically.

2.2.1 Value iteration theory

Let's begin with value iteration. In value iteration we have step 1, set $Q(\mathbf{s},\mathbf{a}) \leftarrow r(\mathbf{s},\mathbf{a}) + \gamma E\left[V\left(\mathbf{s}'\right)\right]$ and step 2, set $V(\mathbf{s}) \leftarrow \max_{\mathbf{a}} Q(\mathbf{s},\mathbf{a})$. What we care about here is does this algorithm actually converge? And if so, what does it converge to?

Here we introduce an operator called Bellman operator $\mathcal{B}$ (in a more elegant matrix form):

$$\mathcal{B}V = \max_{\mathbf{a}} r_{\mathbf{a}} + \gamma \mathcal{T}_{\mathbf{a}} V \tag{2.2.1}$$

where V stands for the values of states, which is a vector that has the same dimension $\mathcal{S}$ as the state space, $r_{\mathbf{a}}$ is a stacked vector of reward at all states for an action $\mathbf{a}$ and $\mathcal{T}_{\mathbf{a}}$ is a $\mathcal{S} \times \mathcal{S}$ matrix, a matrix of transition for action $\mathbf{a}$ such that $\mathcal{T}_{\mathbf{a},i,j} = p\left(\mathbf{s}' = i \mid \mathbf{s} = j, \mathbf{a}\right)$.

One interesting property we can show is that $V^\star$ is *fixed point* of $\mathcal{B}$, which means that:

$$V^\star(\mathbf{s}) = \max_{\mathbf{a}} r(\mathbf{s}, \mathbf{a}) + \gamma E\left[V^\star\left(\mathbf{s}'\right)\right], \text{ so } V^\star = \mathcal{B}V^\star \tag{2.2.2}$$

Furthermore, $V^\star$ always exists, always unique, and it always corresponds to the optimal policy, the policy that maximize total rewards. The only problem is if we can actually reach this fixed point. Here we only show a high level sketch of proving why it converges here. We can prove that value iteration converges to $V^\star$ because $\mathcal{B}$ is a *contraction* (see the previous Sec 1.2.5 for proof). Recall that by contraction we mean that:

$$\text{for any } V \text{ and } \bar{V}, \text{ we have } \|\mathcal{B}V - \mathcal{B}\bar{V}\|_\infty \leq \gamma \|V - \bar{V}\|_\infty \tag{2.2.3}$$

So if we choose $V^\star$ as $\bar{V}$, given $\mathcal{B}V^\star = V^\star$ we have:

$$\left\|\mathcal{B}V - V^\star\right\|_\infty \leq \gamma \left\|V - V^\star\right\|_\infty \tag{2.2.4}$$

In the rest of the notes, we will use $V^\star$ to represent the value function under the optimal policy $\pi^\star$.

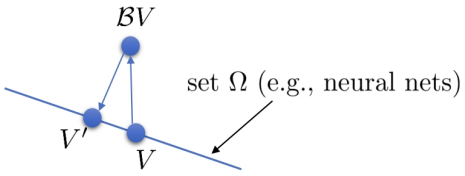
2.2.2 Fitted value iteration theory

In tabular value iteration we have one single step: $V \leftarrow \mathcal{B}V$, which represents setting $Q(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \gamma E\left[V\left(\mathbf{s}'\right)\right]$ and setting $V(\mathbf{s}) \leftarrow \max_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a})$. Now we move on to non-tabular cases, which is the fitted value iteration. We will find out how to deal with the second step, which is the supervised learning part: $\phi \leftarrow \arg\min_\phi \frac{1}{2} \sum_i \left\|V_\phi\left(\mathbf{s}_i\right) - \mathbf{y}_i\right\|^2$

If we look at what step two actually does mathematically, one way to think of supervised learning is that: you have a set of value functions Ω by going through all possible weights of each neuron in the neural network. In supervised learning we sometimes refer to this as hypothesis set. Here the target value, or the updated value function is $\mathcal{B}V$ which we get from the first step. Therefore we can think of the entire fitted value iteration as:

$$V' \leftarrow \arg\min_{V' \in \Omega} \frac{1}{2} \sum \left\|V'(\mathbf{s}) - (\mathcal{B}V)(\mathbf{s})\right\|^2 \tag{2.2.5}$$

We can intuitively think of this procedure as projection of $\mathcal{B}V$, to Ω



To mathematically describe this operation we define a new operator Π, which is a projection onto Ω (in terms of ℓ_2 norm)

$$\Pi V = \arg\min_{V' \in \Omega} \frac{1}{2} \sum \left\|V'(\mathbf{s}) - V(\mathbf{s})\right\|^2 \tag{2.2.6}$$

It's easy to see that $\mathcal{B}$ is a contraction w.r.t. ∞-norm (max norm), and that Π is a contraction w.r.t. ℓ_2 norm (Euclidean distance):

$$\begin{aligned} \|\mathcal{B}V - \mathcal{B}\bar{V}\|_\infty &\leq \gamma \|V - \bar{V}\|_\infty \\ \|\Pi V - \Pi\bar{V}\|^2 &\leq \|V - \bar{V}\|^2 \end{aligned} \tag{2.2.7}$$

But unfortunately $\Pi\mathcal{B}$ is not a contraction of any kind.

This means that value iteration (tabular case) always converges, but in fitted value iteration it does not converge in general, and often not in practice.

2.2.3 Fitted Q-iteration theory

In Q-iteration it's actually the same. All we have to do is to make a tiny adjustment to operator $\mathcal{B}$.

$$\mathcal{B}Q = r + \gamma \mathcal{T} \max_{\mathbf{a}} Q \tag{2.2.8}$$

1. **Value iteration theory** (表格型价值迭代理论) 处理的是**离散的、表格型 (tabular) 的情况**。它在数学上有一个关键算子——贝尔曼算子 $\mathcal{B}$，并且该算子是一个**压缩映射 (Contraction)**。这**保证了算法一定能收敛**到唯一的最优解 V^* 。
2. **Fitted value iteration theory** (拟合价值迭代理论) 处理的是**非表格型 (non-tabular) 的情况**，即我们使用函数近似（如神经网络）。这个过程被分解为两步，引入了一个新的**投影算子 (Projection Operator)** Π 。而这两个算子结合后的新算子 $\Pi\mathcal{B}$ **不再是压缩映射**，因此算法**失去了收敛的保证** 。

> “在一般情况下它不会收敛，在实践中也经常不收敛”。这就是为什么DQN（它本质上是一种 Fitted Q-iteration）的训练会如此不稳定，需要各种技巧（如Replay Buffer和Target Network）来缓解这种不稳定性。

Online vs. Offline: 这个维度回答的是“**我们如何处理和使用经验数据？**”

- **Online (在线)**: 算法在与环境交互的过程中，**每产生一条或一小批新数据，就立刻用它来学习和更新模型，然后通常会丢弃这条数据** 。它强调学习的即时性。
- **Offline (离线)**: 算法先收集一个**固定的大数据集**，然后在这个数据集上进行学习，学习过程中不再与环境产生新的交互 。像 `Fitted Q-learning` 就是一个典型的例子，它会先收集一个数据集，然后在这个数据集上反复迭代直至收敛 。

On-Policy vs. Off-Policy: 这个维度回答的是“**我们学习的策略和我们收集数据的策略是同一个吗？**”

- **On-Policy (同策略)**: 用来学习和改进的策略 (target policy)，与当前用来与环境交互并产生数据的策略 (behavior policy) **是同一个**。简单来说，就是“用自己正在执行的策略所产生的数据来学习”。
- **Off-Policy (离策略)**: 用来学习和改进的策略，可以与产生数据的策略**不同** 。这意味着算法可以利用过去旧策略产生的数据，甚至是其他智能体或人类专家产生的数据来进行学习 。

Note that this time max comes after the transition operator.

The following stuff are all the same: $\mathcal{B}$ is a contraction w.r.t. ∞-norm (max norm), Π is a contraction w.r.t. ℓ_2 norm (Euclidean distance), $\Pi\mathcal{B}$ is not a contraction of any kind.

This also applies to online Q-learning and basically any algorithms of this sort. Specifically, the actor-critic algorithm also does not ensures to converge for the same reason. We will try to solve the convergence problem in next chapter.

2.3 DQN: Deep Learning with Q-Functions

Before diving into Deep Q-Network (DQN), let's take a moment to look back on what we've done.

In Sec 1.3.3, we've shown that Q learning and SARSA both work under the same principle:

$$Q(s,a) = R(s,a) + \gamma \mathbb{E}[V(s')] = R(s) + \gamma \sum_{s'} T(s,a,s') V(s') \tag{2.3.1}$$

And in Q learning we use $\max_{a'} Q(s',a')$ to approximate $\mathbb{E}[V(s')]$, while in SARSA $\mathbb{E}[V(s')]$ is approximated online by $Q(s_{n+1}, a_{n+1})$, which is given by the *next actual state and action*.

2.3.1 Fitted Q-learning and online Q-learning

Now move on to Q-learning with value approximation, we are going to substitute Q by Q_ϕ, where we use a certain method with parameters ϕ to approximate the Q function, given by the following:

$$Q_{\phi'}(s,a) \leftarrow Q_\phi(s,a) + \alpha \left[R(s,a) + \gamma \max_{a'} Q(s',a') - Q_\phi(s,a) \right] \tag{2.3.2}$$

Or it can be rewritten as:

$$Q_{\phi'}(s,a) \leftarrow (1-\alpha) Q_\phi(s,a) + \alpha \underline{\left[R(s,a) + \gamma \max_{a'} Q(s',a') \right]} \tag{2.3.3}$$

And then we use a network that takes in s and a, and outputs $Q_\phi(s,a)$ to do the approximation: for a bunch of pre-collected data $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)\}$, set $\mathbf{y}_i \leftarrow r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'_i} Q_\phi(\mathbf{s}'_i, \mathbf{a}'_i)$ and then fit the following:

$$\phi \leftarrow \arg\min_\phi \frac{1}{2} \sum_i \left\| Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{y}_i \right\|^2 \tag{2.3.4}$$

This is actually the fitted Q learning algorithm.

Fitted Q-learning

Algorithm 8 Fitted Q learning algorithm

1: **repeat**
2: collect dataset $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)\}$ using some policy
3: **repeat**
4: set $\mathbf{y}_i \leftarrow r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'_i} Q_\phi(\mathbf{s}'_i, \mathbf{a}'_i)$
5: set $\phi \leftarrow \arg\min_\phi \frac{1}{2} \sum_i \left\| Q_\phi(\mathbf{s}_i) - \mathbf{y}_i \right\|^2$
6: **until** convergence
7: **until** convergence

Off-Policy：有一个装满了各种经验（即 (s,a,s',r) 转换数据）的大桶 。无论这些经验是哪个版本的"你"（旧的策略）收集的，现在的"你"（当前的策略）都可以从这些旧经验中学习 。你不必亲自去经历一遍来收集新的数据。

Insight. Note that fitted Q-learning algorithm is an *off-policy* algorithm, which means that you could reuse samples from previous iterations. The policy affects the iteration step only in $\max_{\mathbf{a}'_i} Q_\phi(\mathbf{s}'_i, \mathbf{a}'_i)$. This is because intuitively $\max_{\mathbf{a}'_i} Q_\phi(\mathbf{s}'_i, \mathbf{a}'_i) = Q_\phi\left(\mathbf{s}'_i, \arg\max_{\mathbf{a}'_i} Q_\phi\right)$, and $\arg\max_{\mathbf{a}'_i} Q_\phi$ is actually the policy itself since we are following a greedy policy:

$$\pi'(\mathbf{a}_t \mid \mathbf{s}_t) = \begin{cases} 1 \text{ if } \mathbf{a}_t = \arg\max_{\mathbf{a}_t} Q^\pi(\mathbf{s}_t, \mathbf{a}_t) \\ 0 \text{ otherwise} \end{cases} \tag{2.3.5}$$

So one way to think of fitted Q learning is that you have a big bucket of different transitions $((\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i))$. And this is off-policy because transitions are independent of policy π.

The underlined part of taking max in line 3 of Alg. 8 is also why fitted Q learning can indeed improve the policy. While in step three it is actually the fitting error. So you could think of fitted Q learning as optimizing an error:

$$\mathcal{E} = \frac{1}{2} E_{(\mathbf{s},\mathbf{a})\sim\beta}\left[\left(Q_\phi(\mathbf{s},\mathbf{a}) - \left[r(\mathbf{s},\mathbf{a}) + \gamma \max_{\mathbf{a}'} Q_\phi\left(\mathbf{s}',\mathbf{a}'\right)\right]\right)^2\right] \tag{2.3.6}$$

But of course the error itself does not reflect the goodness of your policy. It's just the accuracy with which we are able to copy the target values. If the error is zero, then

$$Q_\phi(\mathbf{s},\mathbf{a}) = r(\mathbf{s},\mathbf{a}) + \gamma \max_{\mathbf{a}'} Q_\phi\left(\mathbf{s}',\mathbf{a}'\right) \tag{2.3.7}$$

This is an optimal Q-function, corresponding to the optimal policy π'. However most guarantees of convergence are lost when we leave the tabular case (e.g., use neural networks).

Online Q-learning

We can make some adjustment and get a different version of Q-learning algorithm that looks more similar to what we've encountered in Sec. 1.3.3:

Algorithm 9 Online Q learning algorithm

1: **repeat**
2: take one action $\mathbf{a}_i$ and observe one transition $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}_i', r_i)$
3: set $\mathbf{y}_i = r\left(\mathbf{s}_i,\mathbf{a}_i\right) + \gamma \max_{\mathbf{a}'} Q_\phi\left(\mathbf{s}_i',\mathbf{a}_i'\right)$
4: set $\phi \leftarrow \phi - \alpha \frac{dQ_\phi}{d\phi}\left(\mathbf{s}_i,\mathbf{a}_i\right)\left(Q_\phi\left(\mathbf{s}_i,\mathbf{a}_i\right) - \mathbf{y}_i\right)$
5: **until** convergence

Insight. This is *online* because it sample data while iterating. And it's also an *off-policy* algorithm since it does not rely on the latest policy to sample data.

Line 4 in Alg. 9 might look like policy gradient (to be explained later in Sec. 3.1), but actually if we substitute $\mathbf{y}_i$ with what it really means we would see the difference:

$$\phi \leftarrow \phi - \alpha \frac{dQ_\phi}{d\phi}\left(\mathbf{s}_i,\mathbf{a}_i\right)\left(Q_\phi\left(\mathbf{s}_i,\mathbf{a}_i\right) - \left[r\left(\mathbf{s}_i,\mathbf{a}_i\right) + \gamma \max_{\mathbf{a}'} Q_\phi\left(\mathbf{s}_i',\mathbf{a}_i'\right)\right]\right) \tag{2.3.8}$$

The problem is that Q_ϕ shows up in two places, and there's no gradient in the second term. So that online Q-learning algorithm *does not guarantee to converge*.

There is actually another problem with online Q-learning algorithm, which is the *correlated samples* (the underlined part). In reality, states in time step $t+1$ are quite similar to states at time step t because changes in states are generally continuous, which means that sequential states are strongly correlated. Besides, the target value is always changing, so that one sample is really hard to train the network.

2.3.2 Q-Learning with replay buffers

Now we will try to solve the aforementioned two problems (the sample correlation problem and the convergence problem), during which process we actually move from *fitted Q-learning* to *deep Q-learning*.

We'll start from the dataset problem first. Other than using parallel workers, another way of solving it is to use **replay buffers**. A replay buffer is literally a buffer that stores the dataset of transitions. Here is the algorithm of Q-learning with a replay buffer.

Algorithm 10 Q-learning with replay buffers

1: **repeat**
2: collect dataset $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}_i', r_i)\}$ using some policy, add it to $\mathcal{B}$
3: **repeat**
4: sample a batch $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}_i', r_i)$ from $\mathcal{B}$
5: set $\phi \leftarrow \phi - \alpha \sum_i \frac{dQ_\phi}{d\phi}\left(\mathbf{s}_i,\mathbf{a}_i\right)\left(Q_\phi\left(\mathbf{s}_i,\mathbf{a}_i\right) - \left[r\left(\mathbf{s}_i,\mathbf{a}_i\right) + \gamma \max_{\mathbf{a}'} Q_\phi\left(\mathbf{s}_i',\mathbf{a}_i'\right)\right]\right)$
6: **until** convergence
7: **until** convergence

1. 问题一：样本相关性 (Correlated Samples)

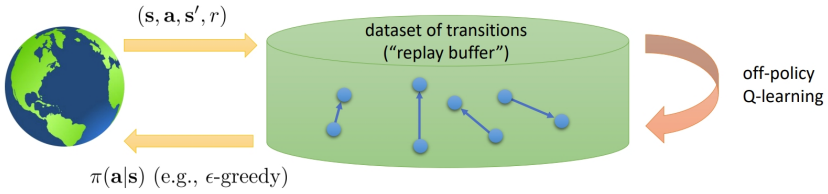
- **Online Q-learning**: 该算法每与环境交互一步，就立即使用这个新产生的样本 (s_i, a_i, s'_i, r_i) 进行训练 。由于在真实环境中，连续的状态之间通常高度相关（例如，游戏中角色在下一帧的位置与上一帧非常接近），这导致训练样本之间失去了独立性，使得神经网络训练非常不稳定 。
- **DQN的解决方案：经验回放 (Replay Buffer)**: DQN 不直接使用刚产生的样本进行训练。相反，它将新样本存入一个叫做“经验回放池 (replay buffer)”的大容量存储器中 。在训练时，它从这个池子里**随机采样一个小批量 (mini-batch)** 的样本来进行学习 。通过这种方式，DQN 打破了样本之间的时间相关性，使样本近似于独立同分布 (i.i.d.)，从而让训练过程更加稳定 。

2. 问题二：目标值不稳定 (Moving Target)

- **Online Q-learning**: 在其更新公式中，用于计算目标值 `y` 的Q网络和用于更新参数 ϕ 的Q网络是**同一个** 。
  - 更新公式: $\phi \leftarrow \phi - \alpha \frac{dQ_\phi}{d\phi}(Q_\phi - [r + \gamma \max_{a'} Q_\phi])$
  - 这意味着，每更新一次参数 ϕ，目标值 $[r + \gamma \max_{a'} Q_\phi]$ 也会跟着改变 。这就好比追逐一个不断移动的目标，使得训练过程非常难以收敛 。
- **DQN的解决方案：目标网络 (Target Network)**: DQN 使用**两个**独立的神经网络。
  - **当前网络 (Q_ϕ)**: 用于评估当前状态-动作对的Q值，并且它的参数 ϕ 在每一步都会通过梯度下降进行更新 。
  - **目标网络 ($Q_{\phi'}$)**: **专门用于计算目标值** `y` 。这个网络的参数 ϕ' **不会**在每一步都更新，而是被**冻结**一段时间，然后定期（例如每N步）从当前网络 Q_ϕ 那里**直接复制**参数 。
  - 通过这种方式，DQN在一段时间内（N步以内）将更新目标固定下来，将Q函数更新问题转化为了一个稳定的监督学习问题，极大地提升了收敛性 。

Instead of collect dataset from real world, it simply samples a batch from the replay buffer $\mathcal{B}$. Therefore the samples are no longer correlated, but i.i.d.. We still do not have a real gradient yet, but at least we solved the correlated problem.

But where does the data come from? Actually what we need to do is to *periodically feed the replay buffer*. Otherwise if we use a fixed replay buffer produced by an initial policy, chances are that the policy is bad and does not cover any interesting regions of states. The entire workflow would look like the figure below.



By adding a replay buffer, the samples are no longer correlated, and we also keep updating the replay buffer by adding new real-world transitions. Note that in this algorithm (Alg. 10), we would commonly repeat the inner loop for only one time, even though repeating for more times would be more efficient.

2.3.3 Q-Learning with target networks

Now solve the remaining problem: Q-learning is not gradient descent, and in particular, there's no gradient through target value, which make it hard to converge.

We can make a comparison between the iteration step of Q-learning with replay buffer and fitted Q-learning:

$$\phi \leftarrow \phi - \alpha \sum_i \frac{dQ_\phi}{d\phi}(\mathbf{s}_i, \mathbf{a}_i)\left(Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - \left[r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\phi(\mathbf{s}'_i, \mathbf{a}'_i)\right]\right) \quad \text{Q-learning with replay buffer} \tag{2.3.9}$$

$$\phi \leftarrow \arg\min_\phi \frac{1}{2} \sum_i \left\| Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - \left[r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\phi(\mathbf{s}'_i, \mathbf{a}'_i)\right] \right\|^2 \quad \text{full fitted Q-learning} \tag{2.3.10}$$

In fitted Q-learning it performs what looks like supervised learning, which is a perfectly well-defined and stable regression. While in Q-learning with replay buffer it has a *moving target*. A target network would be something in between.

Algorithm 11 Q-learning with replay buffer and target network

1: **repeat**
2: save target network parameters $\phi' \leftarrow \phi$
3: **for** N times **do**
4: collect dataset $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)\}$ using some policy, add it to $\mathcal{B}$
5: **for** K times **do**
6: sample a batch $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)$ from $\mathcal{B}$
7: set $\phi \leftarrow \phi - \alpha \sum_i \frac{dQ_\phi}{d\phi}(\mathbf{s}_i, \mathbf{a}_i)\left(Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - \left[r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_{\phi'}(\mathbf{s}'_i, \mathbf{a}'_i)\right]\right)$
8: **until** convergence

Note that *targets don't change in the inner loop!* This (from line 3 to line 9) is purely a supervised learning problem. Parameter K would typically be $1-4$, while N would be about $10,000$.

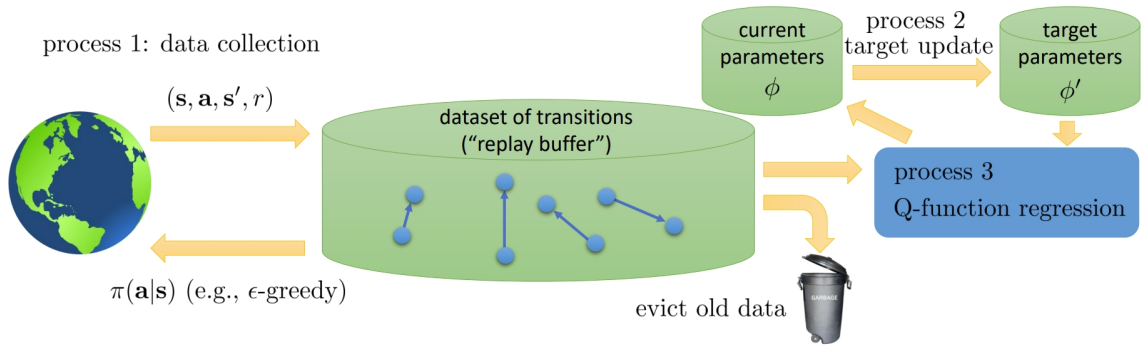
The classic Deep Q-Learning algorithm looks almost the same, simply choosing $K = 1$:

Algorithm 12 Deep Q-Learning

1: **repeat**
2: take some action $\mathbf{a}_i$ and observe $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)$, add it to $\mathcal{B}$
3: sample mini-batch $\{\mathbf{s}_j, \mathbf{a}_j, \mathbf{s}'_j, r_j\}$ from $\mathcal{B}$ uniformly
4: compute $y_j = r_j + \gamma \max_{\mathbf{a}'_j} Q_{\phi'}(\mathbf{s}'_j, \mathbf{a}'_j)$ using target network $Q_{\phi'}$
5: $\phi \leftarrow \phi - \alpha \sum_j \frac{dQ_\phi}{d\phi}(\mathbf{s}_j, \mathbf{a}_j)(Q_\phi(\mathbf{s}_j, \mathbf{a}_j) - y_j)$
6: update ϕ': copy ϕ every $N(\approx 10000)$ steps
7: **until** Convergence

2.3.4 Comparison

We could view the three different Q-learning algorithms (fitted Q-iteration (the purely regression-based one), online Q-learning, deep Q-learning) in a more general way.



As is shown in the figure above, There are three different steps in the algorithm, and several important concepts:

- **replay buffer**: it is actually the dataset of transition, dipicting the transition model of the real world interaction. It is also the foundation of how we improve our policy.
- **data collection**: the data in replay buffer comes from data collection process, where we interact with the real world following some policy (e.g. $\epsilon-$greedy) and bring back one or more transtions. It can be viewed as a continuous process that is always running, and we call it **process 1**.
- **evict old data**: a replay buffer has a finite size, and we have to periodically discard old data. A simple and reasonable way is to structure the replay buffer as a circular buffer. When the buffer is full, the oldest data is overwritten with the newest data.
- **parameters** ϕ: these are the parameters you use to compute the target value. Current parameters ϕ are the parameters that you are going to give to process 1 to construct the $\epsilon-$greedy policy to collect more data.
- **parameters update**: parameters update is typically a very slow process which we call **process 2**. We typically run it very infrequently.
- **Q-function regression**: in this process we load a batch of transitions from the replay buffer, and the target parameters ϕ'. Then we use the target parameters to calculate target values for each of the transitions, regress onto the Q-function, and update the current parameters. We call this procedure **process 3**.

In fitted Q-iteration algorithm, process 3 is in the inner loop of process 2, which is in the inner loop of process 1. In online Q-learning, we evict the old transitions immediately, and process 1, 2, and 3 run at the same speed. In DQN, process 1 and 3 run at the same speed (they are fairly decoupled), but process 2 runs slower. And the replay buffer is quite big.

1. Fitted Q-iteration (纯回归式)

这是一种**分阶段的、类似批处理 (batch)** 的方法。

- **工作流程**:
  1. **过程 1 (运行一次)**: 使用某个策略收集一个庞大的数据集，然后暂停交互 。
  2. **过程 2 & 3 (内循环)**: 在这个固定的数据集上，反复执行Q函数回归（过程3），直到网络参数 ϕ 对这个数据集上的目标值完全拟合（收敛） 。
- **过程关系解读**: 它的流程是“收集一大批 -> 训练到收敛 -> 再收集一大批...”，因此过程3（回归）在过程2（目标参数在此阶段不变）的内循环中，而整个训练过程又在过程1（数据收集）的内循环中 。
- **优点**: 由于目标值在内循环中是固定的，其回归过程（过程3）是一个标准的监督学习问题，非常稳定 。
- **缺点**: 数据利用率极低，且流程非常缓慢。

2. Online Q-learning (在线学习)

这是最直接的**在线 (online)** 算法，试图将函数近似与传统Q-learning结合。

- **工作流程**:
  1. **过程 1, 2, 3 (同步运行)**: 与环境交互一步，获得一个样本。立即用这个样本计算目标值并更新Q网络。然后这个样本就被丢弃了 。
- **过程关系解读**: 数据收集、目标计算和网络更新是同步的 。由于没有目标网络，可以认为过程2（目标参数更新）和过程3（当前网络更新）是同一个过程。
- **优点**: 算法简单直接。
- **缺点**: 存在两个致命问题：
  - **样本相关性 (Correlated Samples)**: 连续的样本之间高度相关，违反了机器学习中“独立同分布”的假设，导致训练不稳定 。
  - **移动目标 (Moving Target)**: 用于计算目标值的网络和正在被更新的网络是同一个 (Q_ϕ)，导致目标值本身在不断变化，网络难以收敛 。

3. Deep Q-learning (DQN)

DQN可以看作是**对 Online Q-learning 两个核心缺陷的修正**，是一种高效且稳定的混合方法。

- **工作流程**:
  - **过程 1 (持续运行)**: 持续与环境交互，并将新数据存入一个大的**经验回放池 (Replay Buffer)** 。
  - **过程 3 (持续运行)**: 每次更新时，从Replay Buffer中**随机采样一个小批量**数据，用目标网络 $Q_{\phi'}$ 计算目标值，然后更新当前网络 Q_ϕ 。
  - **过程 2 (缓慢运行)**: **每隔N步**（例如10000步），才将当前网络 Q_ϕ 的参数**复制**给目标网络 $Q_{\phi'}$ 。
- **过程关系解读**: 数据收集（过程1）和网络回归（过程3）是解耦的、同时高速运行的 。而目标更新（过程2）是**非常缓慢的** 。
- **优点**:
  - **Replay Buffer**通过随机采样打破了样本相关性 。
  - **Target Network**通过冻结参数，将目标值在一段时间内固定下来，解决了移动目标问题，使训练更稳定 。

2.4 Improving DQN

2.4.1 Double DQN(DDQN)

In this section we are going to discuss some practical considerations we need to take while implementing Q-learning algorithm. To start with, as a prediction of total reward from taking $\mathbf{a}_t$ in $\mathbf{s}_t$, Q values are actually inaccurate. Specifically, the estimated Q value is systematically *larger* than the true value, which is often referred to as **overestimation** in Q-learning. The problem lies in the target value:

DQN用单一样本 sj′上的最大Q值，来作为对理论上未来价值期望γE[V(s ′)] 的估计，本身是一个带噪声的估计。max 操作放大了正向噪声的影响，从而导致了系统性的偏差。

$$\text{target value:} \quad y_j = r_j + \gamma \underline{\max_{\mathbf{a}'_j} Q_{\phi'}\left(\mathbf{s}'_j, \mathbf{a}'_j\right)} \tag{2.4.1}$$

The underlined part of taking max is the problem. Imagine we have two random variables: X_1 and X_2. Then $E\left[\max\left(X_1, X_2\right)\right] \geq \max\left(E\left[X_1\right], E\left[X_2\right]\right)$ (Jensen's inequality). This is because while taking max you are actually picking those with larger noise. Similarly $Q_{\phi'}\left(\mathbf{s}'_j, \mathbf{a}'_j\right)$ can be viewed as a true value plus a noise, hence $\max_{\mathbf{a}'_j} Q_{\phi'}\left(\mathbf{s}'_j, \mathbf{a}'_j\right)$ overestimates the next value by always selecting the noisy in positive direction.

One way of fixing this problem is called **double Q-learning**.

Note that:

$$\max_{\mathbf{a}'} Q_{\phi'}\left(\mathbf{s}', \mathbf{a}'\right) = Q_{\phi'}\left(\mathbf{s}', \arg\max_{\mathbf{a}'} Q_{\phi'}\left(\mathbf{s}', \mathbf{a}'\right)\right) \tag{2.4.2}$$

We are *selecting action* according to $Q_{\phi'}$, while also *calculating values* using the same function $Q_{\phi'}$. If the noise in these two procedures are de-correlated, then the problem could be alleviated (but not completely eliminated). The idea here is not to use the same network of choosing actions and estimating values. Double Q-learning uses two networks:

$$\begin{aligned} Q_{\phi_A}(\mathbf{s}, \mathbf{a}) &\leftarrow r + \gamma Q_{\phi_B}\left(\mathbf{s}', \arg\max_{\mathbf{a}'} Q_{\phi_A}\left(\mathbf{s}', \mathbf{a}'\right)\right) \\ Q_{\phi_B}(\mathbf{s}, \mathbf{a}) &\leftarrow r + \gamma Q_{\phi_A}\left(\mathbf{s}', \arg\max_{\mathbf{a}'} Q_{\phi_B}\left(\mathbf{s}', \mathbf{a}'\right)\right) \end{aligned} \tag{2.4.3}$$

In practice, we simply use the current and target network to implement these two networks. In standard Q-learning, we have $y = r + \gamma Q_{\phi'}\left(\mathbf{s}', \arg\max_{\mathbf{a}'} Q_{\phi'}\left(\mathbf{s}', \mathbf{a}'\right)\right)$. While in double Q-learning, we just use the *current* network (not the target network) to *select actions* and still use the *target* network to *evaluate values*, namely $y = r + \gamma Q_{\phi'}\left(\mathbf{s}', \arg\max_{\mathbf{a}'} Q_{\phi}\left(\mathbf{s}', \mathbf{a}'\right)\right)$. Here is the algorithm:

Algorithm 13 DDQN

Require: Q_ϕ is the current network, $Q_{\phi'}$ is the target network, $\mathcal{B}$ is the replay buffer

1: **repeat**
2: take some action $\mathbf{a}_i$ and observe $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)$, add it to $\mathcal{B}$
3: sample mini-batch $\left\{\mathbf{s}_j, \mathbf{a}_j, \mathbf{s}'_j, r_j\right\}$ from $\mathcal{B}$
4: compute $y_j = r_j + \gamma Q_{\phi'}\left(\mathbf{s}', \arg\max_{\mathbf{a}'} Q_{\phi}\left(\mathbf{s}', \mathbf{a}'\right)\right)$
5: $\phi \leftarrow \phi - \alpha \sum_j \frac{dQ_\phi}{d\phi}\left(\mathbf{s}_j, \mathbf{a}_j\right)\left(Q_\phi\left(\mathbf{s}_j, \mathbf{a}_j\right) - y_j\right)$
6: update ϕ': copy ϕ to ϕ' every $N(\approx 10000)$ steps
7: **until** Convergence

Take a look at Van Hasselt et al. (2016) for more details.

2.4.2 Using Multi-step Returns

There is another trick we can use called **Multi-step returns**, which is quite similar to N-step return when we first learn about time difference evaluation in section 1.3.2

The Q-learning target value consists of two parts, one is the reward of current time-step, the other is the estimated reward of next time-step given Q'_ϕ. At the beginning, Q'_ϕ is bad and the first term is the only values that matters (so we are not learning much about the Q-function), while when the policy becomes better, the second term would be more important. This is somehow similar to the actor-critic:

$$\text{Actor-critic:} \quad \nabla_\theta J(\theta) \approx \frac{1}{N}\sum_{i=1}^{N}\sum_{t=1}^{T} \nabla_\theta \log \pi_\theta\left(\mathbf{a}_{i,t} \mid \mathbf{s}_{i,t}\right)\left(\underline{r\left(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}\right) + \gamma \hat{V}^\pi_\phi\left(\mathbf{s}_{i,t+1}\right)} - \hat{V}^\pi_\phi\left(\mathbf{s}_{i,t}\right)\right) \tag{2.4.4}$$

因此，多步回报的目标值 y 强依赖于生成这条轨迹的数据收集策略。如果我们想用这些数据来更新我们的目标策略（贪婪策略），但这些数据本身又是另一个策略（比如 ε-greedy策略）产生的，两者之间就出现了不匹配。为了保证学习的正确性，数据收集策略和目标策略必须是同一个，这就是同策略 (on-policy) 的定义。

In actor-critic algorithm, to leverage the bias and variance trade-off in policy gradient, we can end the trajectory earlier, and only count the reward summed up to N steps from now. The way we construct multi-step return (N-step return estimator) is similar to that in actor-critic:

$$y_{j,t} = \sum_{t'=t}^{t+N-1} \gamma^{t-t'} r_{j,t'} + \gamma^N \max_{\mathbf{a}_{j,t+N}} Q_{\phi'}\left(\mathbf{s}_{j,t+N}, \mathbf{a}_{j,t+N}\right) \tag{2.4.5}$$

This adjustment would make the target value *less biased* when the Q function is inaccurate, and allows for typically faster learning especially at early on stage.

One subtle problem with this solution is that the learning process suddenly becomes *on-policy*, so we cannot efficiently make use of the off-policy data. Why is it on-policy? If we look at the summation of the rewards, we are collecting the rewards data using a certain trajectory, which is generated by a specific policy.

Add reference for *Safe and efficient off-policy reinforcement learning.*

To fix this problem, here are three possible solutions:

1. You can simply *ignore* this mismatch, which somehow works very well in practice.
2. You can cut the trace by dynamically adapting N to get only on-policy data. (This only works well when data is mostly on-policy, and that action space is small.)
3. You can use **importance sampling** (more on this later).

2.4.3 Prioritized Experience Replay

Note that the samples saved in the replay buffer are used randomly, and this is intuitively inefficient, since we don't want to waste computation on the samples that are not informative. A method called **prioritized experience replay (PER)** (Schaul et al., 2015) solves this issue by assigning weights to the samples so that "important" ones are drawn more frequently for training.

Based on this idea, we would sample with respect to some mysterious function $f\left((s_t, a_t, r_t, s_{t+1})\right)$ that exactly tells us the *"usefulness"* of samples, for fastest learning to get maximum reward. An ideal option for this magic function is TD error. As a recap, in Q learning the magnitude of the TD error (squared) is what we want to minimize according to the Bellman equation (See sec.1.3.2 if you forget what is TD error).

The TD error for vanilla DQN is:

$$\delta_i = r_t + \gamma \max_{a \in \mathcal{A}} Q_{\theta^-}\left(s_{t+1}, a\right) - Q_\theta\left(s_t, a_t\right) \tag{2.4.6}$$

And for Double DQN it would be:

$$\delta_i = r_t + \gamma Q_{\theta^-}\left(s_{t+1}, \text{argmax}_{a \in \mathcal{A}} Q_\theta\left(s_{t+1}, a\right)\right) - Q_\theta\left(s_t, a_t\right) \tag{2.4.7}$$

Then naturally we link between TD error and priority:

$$p_i = |\delta_i| + \epsilon \tag{2.4.8}$$

where ϵ is a small constant ensuring that the sample has some non-zero probability of being drawn. Then the probability distribution would be:

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha} \tag{2.4.9}$$

where α determines the level of prioritization. If $\alpha \to 0$, then there is no prioritization, because all$p_i^\alpha = 1$. If $\alpha \to 1$, then we get to, in some sense, "full" prioritization, where sampling data points is more heavily dependent on the actual TD error values.

Note that there is one more technical issue to be solved: Prioritized replay introduce bias because it does not sample experiences uniformly at random due to the sampling proportion corresponding to TD error. We need to correct this bias by using importance sampling weights:

$$w_i = \left(\frac{1}{N}\frac{1}{P(i)}\right)^\beta \tag{2.4.10}$$

优先经验回放TD误差 (TD-error) 越大的样本，其被采样的优先级 (priority) 就越高，打破了原先均匀随机 (uniformly at random) 采样（无偏的）

α [0, 1]决定了我们要在多大程度上使用优先级进行采样，α=0 时PER退化为均匀随机采样，当 α=1 时，采样概率与样本的优先级成正比，这被称为"完全的"优先级采样；

由优先级 pi和 α 决定哪个样本更容易被选中，对于已经被选中的样本，
β [0, 1]通过重要性采样权重 (Importance Sampling Weights) 控制着我们在多大程度上对采样偏差进行修正，β=1 时，权重 wi会完全补偿非均匀采样带来的偏差[即对于TD误差δi，乘上系数wi]

where N is the size of the replay buffer and β is a hyperparameter that controls the degree of importance sampling. w_i fully compensates for the non-uniform probability if $\beta = 1$. For stability, we always normalize weights by $max(w_i)$.

$$w_i = \frac{w_i}{\max(w_i)} \tag{2.4.11}$$

In practice, we don't update the priorities of the transitions in the entire replay buffer. Instead, we store a new transition to the transition buffer with the maximum priority and update the priorities of the sampled transitions in the replay buffer after each update.

2.4.4 Dueling Networks

The goal of Dueling DQN is to have a network that separately computes the advantage and value functions, and combines them back into a single Q-function only at the final layer:

- Action-independent value function: $V(s;\theta,\beta)$ "当前这个局面/状态本身的价值如何？"
- Action-dependent advantage function: $A(s,a;\theta,\alpha)$ "在这个状态下，执行动作a比执行平均水平的动作好多少？"

神经网络的目标是最小化最终Q值的预测误差。它只关心 V 和 A 加起来的结果对不对，而不关心 V 和 A 各自的值是否具有真实的物理意义。

Combining these two together, we have:

$$Q(s,a;\theta,\alpha,\beta) = V(s;\theta,\beta) + A(s,a;\theta,\alpha) \tag{2.4.12}$$

However, naively adding these two values together can be problematic in two aspects:

1. It is problematic to assume that V and A gives reasonable estimates of the state-value and action advantages, respectively. The estimates given by these components, despite being parts of the parameterized Q-function, may not accurately represent the true state-value or advantage function.
2. The naive sum of the two is "unidentifiable," in that given the Q value, we cannot recover the V and A uniquely. It is empirically shown that this lack of identifiability leads to poor practical performance.

Therefore, the last module of the neural network implements forward mapping shown below:

$$Q(s,a;\theta,\alpha,\beta) = V(s;\theta,\beta) + \left(A(s,a;\theta,\alpha) - \max_{a' \in |\mathcal{A}|} A(s,a;\theta,\alpha)\right) \tag{2.4.13}$$

which will force the Q value for the maximizing action to equal V, solving the identifiability issue. The network structure is visualized in figure 2.1:

在结构上保证了优势最大的那个动作，其最终的Q值就等于网络输出的V值 。通过这种方式给V一个明确的含义：当前状态下最优动作的Q值，消除了 Q=V+A 分解时的无穷多可能性

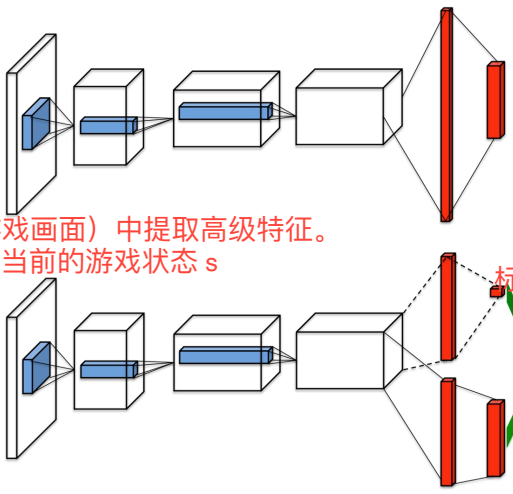


Figure 2.1: Dueling Networks

It may seem somewhat pointless to do this at first glance. Why decompose a function that we will just put back together?

流向V-stream的梯度:

- 从公式中我们可以看到，$V(s)$ 对**所有动作**的 $Q(s,a)$ 都有贡献。
- 因此，传递给 $V(s)$ 的梯度，是**所有**从 $Q(s,a)$ 传回来的梯度的**总和**。

$$\nabla_V L = \sum_a \frac{\partial Q(s,a)}{\partial V(s)} \nabla_Q L = \sum_a \nabla_Q L$$

- **直观理解**：因为V代表的是整体的状态价值，所以它会汇集所有动作Q值上的误差信号，来判断对当前状态的整体估计是偏高了还是偏低了。

流向A-stream的梯度:

- 对于一个特定的动作 a_i，它的优势函数 $A(s,a_i)$ 对最终Q值的影响来自两部分：直接的 $A(s,a_i)$ 项和间接的 $-\max_{a'} A(s,a')$ 项。
- 我们假设优势最大的动作是 a^* (即 $A(s,a^*) = \max_{a'} A(s,a')$)。
  - **对于 $a_i \neq a^*$ 的动作**：$A(s,a_i)$ 只对 $Q(s,a_i)$ 有贡献。所以梯度直接从 $Q(s,a_i)$ 流回。

$$\nabla_{A_i} L = \frac{\partial Q(s,a_i)}{\partial A(s,a_i)} \nabla_{Q_i} L = \nabla_{Q_i} L$$

  - **对于最优动作 a^***：$A(s,a^*)$ 不仅对 $Q(s,a^*)$ 有贡献，它还作为最大值，对**所有其他**动作的Q值（通过 $-\max$ 项）都有贡献。因此，它的梯度是：

$$\nabla_{A^*} L = \nabla_{Q^*} L - \sum_{a \neq a^*} \nabla_{Q_a} L$$

    (注意：这里减去其他梯度的总和，是因为 $\max$ 函数的导数特性，梯度只会流向那个值最大的元素。)

- **直观理解**：
  - 对于非最优动作，网络只学习如何调整它自己的优势值。
  - 对于最优动作，它的优势值 $A(s,a^*)$ 成为了所有其他优势值的“基准”。因此，它不仅要接收来自自身Q值的误差信号，还要接收来自所有其他Q值的误差信号（以负值的形式），以调整这个“基-准”本身。

Actually the advantage of this approach lies in the fact that it helps the network to generalize learning across different actions more effectively. By decoupling the value and advantage, the network can learn which states are valuable without being influenced by the action advantages, and at the same time, it can learn which actions are beneficial without being affected by the differences in value between states.

The more prosaic explanation is that the function decomposition is always technically correct (for an MDP). Coding the network like this incorporates *known structure of the problem* into the network, which otherwise it may have to spend resources on learning. So it's a way of injecting the designer's knowledge of reinforcement learning problems into the architecture of the network.

2.5 Q-Learning with Continuous Actions

In this section, we will introduce two methods for Q-learning with continuous action space.

2.5.1 Deep Deterministic Policy Gradients

To begin with, let's see why we need DDPG for continuous actions planning. Here is a concrete example: we are trying to solve the classic Inverted Pendulum control problem. In this setting, we can take only two actions: swing left or swing right. What make this problem challenging for Q-Learning Algorithms is that actions are continuous instead of being discrete. That is, instead of using two discrete actions like -1 or +1, we have to select from infinite actions in a given range. In DQN we have the greedy policy assumption, so that in target network you have to do the 'max-trick':

$$\pi\left(\mathbf{a}_t \mid \mathbf{s}_t\right)=\begin{cases} 1 \text{ if } \mathbf{a}_t=\arg\max_{\mathbf{a}_t} Q_\phi\left(\mathbf{s}_t, \mathbf{a}_t\right) \\ 0 \text{ otherwise} \end{cases} \tag{2.5.1}$$

$$y_j=r_j+\gamma \max_{\mathbf{a}_j'} Q_{\phi'}\left(\mathbf{s}_j', \mathbf{a}_j'\right) \tag{2.5.2}$$

But turning to continuous action space, it's no longer possible to perform the arg max.

There are several solutions to performing the max:

Option 1: sample and optimize

The first option is to use optimization techniques. Gradient based optimization (e.g., SGD) involves multiple steps of optimization and would be a bit slow in the inner loop. It turns out that stochastic optimization would be a better option since action space here is typically low-dimensional.

In stead of giving a close form solution of optimization on continuous action space, a deadly simple way is to sample $(\mathbf{a}_1, \ldots, \mathbf{a}_N)$ from some distribution:

$$\max_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a}) \approx \max\left\{Q\left(\mathbf{s}, \mathbf{a}_1\right), \ldots, Q\left(\mathbf{s}, \mathbf{a}_N\right)\right\} \tag{2.5.3}$$

However this does not work well in practice majorly because it is generally non-trivial to sample from an arbitrary distribution. Some more accurate solutions, like **cross-entropy method (CEM)** (simple iterative stochastic optimization) or **CMA-ES** (substantially less simple iterative stochastic optimization) works OK for up to about 40 dimensions.

Option 2: Better Q function structure

由于NAF的Q函数被设计成一个开口向下的二次函数，其最大值点就是顶点，可以直接由网络输出得到

The second option is to use function class that is inherently *easy to optimize*. One example is to use **quadratic function**:

$$Q_\phi(\mathbf{s}, \mathbf{a})=-\frac{1}{2}\left(\mathbf{a}-\mu_\phi(\mathbf{s})\right)^T P_\phi(\mathbf{s})\left(\mathbf{a}-\mu_\phi(\mathbf{s})\right)+V_\phi(\mathbf{s}) \tag{2.5.4}$$

This approach is called **Normalized Advantage Functions (NAF)**. In this case the object to be maximized is given directly by the networks' output:

$$\arg\max_{\mathbf{a}} Q_\phi(\mathbf{s}, \mathbf{a})=\mu_\phi(\mathbf{s}) \quad \max_{\mathbf{a}} Q_\phi(\mathbf{s}, \mathbf{a})=V_\phi(\mathbf{s}) \tag{2.5.5}$$

Using NAF does not change the algorithm itself, so that it's just as efficient as Q-learning, but it sacrifices the representational power and is less expressive.

一个通用的神经网络原则上可以拟合任意复杂的函数。但是，NAF强行规定Q函数必须是一个二次函数

Option 3: learn an approximate maximizer

Remember we have this equation always hold:

$$\max_{\mathbf{a}} Q_\phi(\mathbf{s}, \mathbf{a}) = Q_\phi\left(\mathbf{s}, \arg\max_{\mathbf{a}} Q_\phi(\mathbf{s}, \mathbf{a})\right) \tag{2.5.6}$$

So a big idea based on this is to train a neural network $\mu_\theta(s)$ so that $\mu_\theta(\mathbf{s}) \approx \arg\max_{\mathbf{a}} Q_\phi(\mathbf{s}, \mathbf{a})$. In practice, this is equivlant to finding a set of parameters θ that maximize $Q_\phi(\mathbf{s}, \mu_\theta(\mathbf{s}))$, i.e., $\theta \leftarrow \arg\max_\theta Q_\phi(\mathbf{s}, \mu_\theta(\mathbf{s}))$.

Given the chain rule:

$$\frac{dQ_\phi}{d\theta} = \frac{d\mathbf{a}}{d\theta}\frac{dQ_\phi}{d\mathbf{a}} \tag{2.5.7}$$

The update rule for theta should be:

$$\theta \leftarrow \theta + \beta \sum_j \frac{d\mu}{d\theta}(\mathbf{s}_j)\frac{dQ_\phi}{d\mathbf{a}}(\mathbf{s}_j, \mu(\mathbf{s}_j)) \tag{2.5.8}$$

The network $\mu_\theta(\mathbf{s})$ is sometimes referred to as *deterministic network*.

Now merging this idea to DQN, we have Deep Deterministic Policy Gradient (DDPG) (Lillicrap et al., 2015)

Algorithm 14 DDPG algorithm

1: **repeat**
2: take some action $\mathbf{a}_i$ and observe $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)$, add it to $\mathcal{B}$
3: sample mini-batch $\{\mathbf{s}_j, \mathbf{a}_j, \mathbf{s}'_j, r_j\}$ from $\mathcal{B}$ uniformly
4: compute $y_j = r_j + \gamma \max_{\mathbf{a}'_j} Q_{\phi'}(\mathbf{s}'_j, \mu_{\theta'}(\mathbf{s}'_j))$ using target network $Q_{\phi'}$ and $\mu_{\theta'}$
5: $\phi \leftarrow \phi - \alpha \sum_j \frac{dQ_\phi}{d\phi}(\mathbf{s}_j, \mathbf{a}_j)(Q_\phi(\mathbf{s}_j, \mathbf{a}_j) - y_j)$
6: $\theta \leftarrow \theta + \beta \sum_j \frac{d\mu}{d\theta}(\mathbf{s}_j)\frac{dQ_\phi}{d\mathbf{a}}(\mathbf{s}_j, \mu(\mathbf{s}_j))$
7: update ϕ' and θ' after every N steps using ϕ and θ respectively
8: **until** Convergence

Note that though it has 'policy gradient' in its name, DDPG is essentially a Q-learning in disguise. And similar to DQN, DDPG is also a model-free off-policy algorithm.

2.5.2 Twin Delayed DDPG (TD3)

DDPG actually suffers from similar problems as in DQN, for example overestimation. Besides, the critices might be unstable, making convergence difficult. The critic also weirdly prefers some actions but not their neighbor actions. Twin Delayed DDPG introduced three solution to improve DDPG.

Solution 1: Clipped Double Q Learning

Solution 2: Delayed Policy Updates

Solution 3: Target Policy smoothing

2.6 Implementation Tips

In this section we would provide some practical tips for implementing Q-learning algorithm.

- Q-learning takes some care to stabilize, so you should test on easy, reliable tasks first, to make sure your implementation is correct.
- Large replay buffers help improve stability.
- It takes time to train, so be patient because the performance might look no better than random for a while
- Start with high exploration (epsilon) and gradually reduce.
- Double Q-learning helps a lot in practice. It's really simple and basically has no downsides.

$$\text{Bellman Error} = \underbrace{(r + \gamma \max_{a'} Q_w(s', a'))}_{\text{目标Q值 (Target)}} - \underbrace{Q_w(s, a)}_{\text{当前Q值预测}}$$

Q(s, a)是在当前状态 s 下，做动作 a 有多好。也就是在这种情况下，我预期能获得的所有未来奖励的总和

- Bellman error gradients can be big, so instead you can use clip gradients or use Huber loss $L(x)$.

- 当误差较小时 ($|x| \leq \delta$)：Huber损失等同于**均方误差 (MSE)**。它的梯度与误差成正比，这有助于模型在接近最优解时进行精细调整。
- 当误差较大时 ($|x| > \delta$)：Huber损失变为**绝对值误差 (MAE) 的线性形式**。重要的是，此时它的梯度是一个常数（δ 或 $-\delta$），而**不是**随着误差的增大而无限增大。

$$L(x) = \begin{cases} x^2/2 & \text{if } |x| \leq \delta \\ \delta|x| - \delta^2/2 & \text{otherwise} \end{cases}$$

- N-step returns also help a lot, but have some downsides.
- Schedule exploration (high to low) and learning rates (high to low), Adam optimizer can help too.
- Run multiple random seeds, it's very inconsistent between runs.

2.7 Mathy Stuff

理想化的场景：在状态 s 下，所有动作的"真实价值"都是完全一样的。Q值估计函数（比如一个神经网络）会有误差。这里假设误差是iid、无偏的

Theorem 2.1. Consider a state s in which all the true optimal action values are equal at $Q_*(s, a) = V_*(s)$. Suppose that the estimation errors $Q_t(s, a) - Q_*(s, a)$ are independently distributed uniformly randomly in $[-1, 1]$. Then,

即使我们对每个动作的Q值估计是无偏的（误差期望为0），但对这些估计值取max操作之后的结果却不再是无偏的

$$\mathbb{E}\left[\max_a Q_t(s, a) - V_*(s)\right] = \frac{m-1}{m+1} \tag{2.7.1}$$

Theorem 2.2. Consider a state s in which all the true optimal action values are equal at $Q_*(s, a) = V_*(s)$ for some $V_*(s)$. Let Q_t be arbitrary value estimates that are on the whole unbiased in the sense that

$$\sum_a (Q_t(s, a) - V_*(s)) = 0,$$

这里的无偏假设比定理2.1更宽松。它不要求误差是均匀分布的，只要求所有动作的误差加起来等于0。

but that are not all zero, such that

$$\frac{1}{m} \sum_a (Q_t(s, a) - V_*(s))^2 = C$$

只要Q值估计存在误差（C>0），那么估计出的最大Q值必然会比真实的最大值高

过高估计的程度与误差的方差 C 成正比（估计越不稳定，高估越严重），与动作数量 m 成反比（动作越多，单个动作误差的影响被摊薄，高估的下界反而变小）。高

for some $C > 0$, where $m \geq 2$ is the number of actions in s.

Under these conditions,

$$\max_a Q_t(s, a) \geq V_*(s) + \sqrt{\frac{C}{m-1}}. \tag{2.7.2}$$

This lower bound is tight. Under the same conditions, the lower bound on the absolute error of the Double Q-learning estimate is zero.

Double Q-learning从机制上避免了这种必然的过高估计问题。它通过将"选择哪个是最佳动作"和"评估这个最佳动作的价值"这两个步骤解耦，从而打破了max操作带来的正偏差循环。

2.8 Exercises

2.8.1 Linear Approximation

This problem is adapted from Assignment 2 of CS234 (2023 Spring).

Recall that the update rule for Q-learning is

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left(r + \gamma \max_{a' \in \mathcal{A}} Q(s', a') - Q(s, a) \right) \tag{2.8.1}$$

Due to the scale of Atari environments, we cannot reasonably learn and store a Q value for each state-action tuple. We will instead represent our Q values as a parametric function $Q_{\mathbf{w}}(s, a)$ where $\mathbf{w} \in \mathbb{R}^p$ are the parameters of the function (typically the weights and biases of a linear function or a neural network). In this approximation setting, the update rule becomes

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \left(r + \gamma \max_{a' \in \mathcal{A}} Q_{\mathbf{w}}(s', a') - Q_{\mathbf{w}}(s, a) \right) \nabla_{\mathbf{w}} Q_{\mathbf{w}}(s, a) \tag{2.8.2}$$

where (s, a, r, s') is a transition from the MDP. Let $Q_{\mathbf{w}}(s, a) = \mathbf{w}^\top \delta(s, a)$ be a linear approximation for the Q function, where $\mathbf{w} \in \mathbb{R}^{|\mathcal{S}||\mathcal{A}|}$ and $\delta : \mathcal{S} \times \mathcal{A} \to \mathbb{R}^{|\mathcal{S}||\mathcal{A}|}$ with

$$[\delta(s, a)]_{s',a'} = \begin{cases} 1 & \text{if } s' = s, a' = a \\ 0 & \text{otherwise} \end{cases}$$

In other words, δ is a function which maps state-action pairs to one-hot encoded vectors. Compute $\nabla_{\mathbf{w}} Q_{\mathbf{w}}(s, a)$ and write the update rule for $\mathbf{w}$. Show that Eq. 2.8.1 and 2.8.2 are are equal when this linear approximation is used.

2.8.2 N-step Estimators

This problem is adapted from Assignment 2 of CS234 (2020 Winter).

Add N-step Estimators

Chapter 3 Policy-based Methods

- τ **(tau)**：代表一个**轨迹 (trajectory)** 或 **回合 (episode)**。它是一个完整的状态-动作序列，例如 $\tau = (s_1, a_1, s_2, a_2, ..., s_T, a_T)$。它记录了智能体从开始到结束的完整路径。
- $r(\tau)$：代表轨迹 τ 的**总回报 (total reward)**。它是该轨迹中所有单步奖励的总和，即 $r(\tau) = \sum_t r(s_t, a_t)$。
- $p_\theta(\tau)$：代表当智能体遵循策略 π_θ 时，**轨迹 τ 发生的概率**。这个概率取决于初始状态的分布、策略 $\pi_\theta(a_t|s_t)$ 和环境的动态 $p(s_{t+1}|s_t, a_t)$。策略参数 θ 的改变会直接影响动作的选择，从而改变每条轨迹发生的概率。

3.1 Policy Gradient

3.1.1 Direct Policy Differentiation

During previous discussions, we learned that trying to find $\theta^\star$ would help us get more reward.

$$\theta^\star = \arg\max_\theta E_{\tau\sim p_\theta(\tau)}\left[\sum_t r\left(\mathbf{s}_t, \mathbf{a}_t\right)\right] \tag{3.1.1}$$

$$p_\theta\left(\mathbf{s}_1, \mathbf{a}_1, \ldots, \mathbf{s}_T, \mathbf{a}_T\right) = p\left(\mathbf{s}_1\right)\prod_{t=1}\pi_\theta\left(\mathbf{a}_t \mid \mathbf{s}_t\right) p\left(\mathbf{s}_{t+1} \mid \mathbf{s}_t, \mathbf{a}_t\right) \tag{3.1.2}$$

这就得到了期望的定义式：$J(\theta) = E_{\tau\sim p_\theta(\tau)}[r(\tau)]$。这个表达式的含义是："当轨迹 τ 是从概率分布 $p_\theta(\tau)$ 中采样时，回报 $r(\tau)$ 的期望值"。

对于连续（或高维）的变量空间，期望的求和操作就用**积分**来表示。于是，我们就得到了图片中的积分形式：$J(\theta) = \int p_\theta(\tau) r(\tau) d\tau$。

We can rewrite the objective function as:

$$J(\theta) = E_{\tau\sim p_\theta(\tau)}\left[\sum_t r\left(\mathbf{s}_t, \mathbf{a}_t\right)\right] \approx \frac{1}{N}\sum_i\sum_t r\left(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}\right) \tag{3.1.3}$$

where $\sum_i$ means the sum over samples from π_θ. If we abbreviate $\sum_t r\left(\mathbf{s}_t, \mathbf{a}_t\right)$ as $r\left(\tau\right)$, then we have:

$$J(\theta) = E_{\tau\sim p_\theta(\tau)}[r(\tau)] = \int p_\theta(\tau) r(\tau) d\tau \tag{3.1.4}$$

这里的积分是对轨迹 τ 在所有可能轨迹组成的巨大轨迹空间(每一个点都代表一条完整的轨迹)中进行积分

To find the optimal θ, we calculate the policy differentiation:

$$\nabla_\theta J(\theta) = \int \nabla_\theta p_\theta(\tau) r(\tau) d\tau = \int p_\theta(\tau)\nabla_\theta \log p_\theta(\tau) r(\tau) d\tau = E_{\tau\sim p_\theta(\tau)}\left[\nabla_\theta \log p_\theta(\tau) r(\tau)\right] \tag{3.1.5}$$

The derivation from the second term to the third term of the equation might be a bit confusing. Here we use a convenient identity:

$$p_\theta(\tau)\nabla_\theta \log p_\theta(\tau) = p_\theta(\tau)\frac{\nabla_\theta p_\theta(\tau)}{p_\theta(\tau)} = \nabla_\theta p_\theta(\tau) \tag{3.1.6}$$

Also, note that $p_\theta(\tau)$ is given in (4.2) , and take *log* on both sides, then we have:

$$\log p_\theta(\tau) = \log p\left(\mathbf{s}_1\right) + \sum_{t=1}^{T}\log \pi_\theta\left(\mathbf{a}_t \mid \mathbf{s}_t\right) + \log p\left(\mathbf{s}_{t+1} \mid \mathbf{s}_t, \mathbf{a}_t\right) \tag{3.1.7}$$

Substituting into equation (4.5), we obtain the *direct policy differentiation.*

Theorem 3.1 (Direct policy differentiation).

$$\begin{aligned}\nabla_\theta J(\theta) &= E_{\tau\sim p_\theta(\tau)}\left[\left(\sum_{t=1}^{T}\nabla_\theta \log \pi_\theta\left(\mathbf{a}_t \mid \mathbf{s}_t\right)\right)\left(\sum_{t=1}^{T} r\left(\mathbf{s}_t, \mathbf{a}_t\right)\right)\right] \\ &\approx \frac{1}{N}\sum_{i=1}^{N}\left(\sum_{t=1}^{T}\nabla_\theta \log \pi_\theta\left(\mathbf{a}_{i,t} \mid \mathbf{s}_{i,t}\right)\right)\left(\sum_{t=1}^{T} r\left(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}\right)\right)\end{aligned}$$

1. 最大似然 (Maximum Likelihood) 的解释

首先看最大似然的公式：

$$\nabla_\theta J_{ML}(\theta) \approx \frac{1}{N}\sum_{i=1}^{N}\left(\sum_{t=1}^{T}\nabla_\theta log\ \pi_\theta(a_{i,t}|s_{i,t})\right)$$

- **使用场景**: 这通常用于**监督学习 (Supervised Learning)** 或**模仿学习 (Imitation Learning)**。在这种场景下，我们拥有一批由专家生成的“正确”轨迹数据 $\{(s_1,a_1),(s_2,a_2),...\}_{expert}$。
- **目标**: 我们的目标是调整策略 π_θ 的参数 θ，使得策略网络在看到专家所处的状态 $s_{i,t}$ 时，能够以尽可能大的概率输出专家所采取的动作 $a_{i,t}$。
- **公式解读**:
  - $\log\ \pi_\theta(a_{i,t}|s_{i,t})$ 是在状态 $s_{i,t}$ 时采取动作 $a_{i,t}$ 的对数概率。
  - $\nabla_\theta \log \pi_\theta$ 是这个对数概率的梯度。这个梯度指向一个方向，如果参数 θ 朝着这个方向更新，那么 $\pi_\theta(a_{i,t}|s_{i,t})$ 的概率值就会上升。
  - 整个公式的含义是：对于收集到的所有专家数据（N条轨迹），计算一个平均的梯度方向，这个方向会让我们的策略更有可能复现出专家的行为。
- **关键点**: 它**无条件地**认为专家数据中的每一个动作都是“好”的，并试图提高复现这些动作的概率。

2. 策略梯度 (Policy Gradient) 的解释

接下来看策略梯度的公式：

$$\nabla_\theta J(\theta) \approx \frac{1}{N}\sum_{i=1}^{N}\left(\sum_{t=1}^{T}\nabla_\theta log\ \pi_\theta(a_{i,t}|s_{i,t})\right)\left(\sum_{t=1}^{T} r(s_{i,t},a_{i,t})\right)$$

- **使用场景**: 这是**强化学习 (Reinforcement Learning)** 的核心。我们没有专家数据，智能体只能通过自己在环境中探索来生成轨迹。
- **目标**: 调整参数 θ 以最大化期望总回报。
- **公式解读**:
  - 这个公式与最大似然公式相比，最关键的区别在于多了一个**回报项**: $(\sum_{t=1}^{T} r(s_{i,t},a_{i,t}))$。这个项代表了第 i 条轨迹的**总回报**。
  - 这个总回报项在这里扮演了**权重**的角色，它的正负和大小至关重要：
    - **如果总回报是大的正数**: 这说明这条轨迹中的一系列动作是“**好的**”。这个正权重会乘以梯度 $\nabla_\theta \log \pi_\theta$，使得参数 θ 朝着**增强**这些动作概率的方向更新。
    - **如果总回报是负数**: 这说明这条轨迹中的一系列动作是“**坏的**”。这个负权重会使得参数 θ 朝着**减弱**这些动作概率的方向更新。
    - **如果总回报接近零**: 那么权重就很小，这次更新对策略的影响也微乎其微。

策略梯度是“在策略 (On-Policy)”算法:期望所用的数据（轨迹 τ）必须是由我们当前正在优化的策略 πθ 生成的。而专家轨迹是“离策略 (Off-Policy)”数据

直接应用策略梯度的后果

如果您直接将标准的策略梯度公式应用在这批专家数据上，您实际上是在计算一个**错误的梯度**。

- 您计算出的梯度 $\nabla_\theta \log \pi_\theta$ 是在告诉模型“如何调整参数，才能让你（π_θ）更有可能做出专家数据集里的这些动作”。
- 但是，您用来给这个梯度加权的奖励信号 $r(\tau)$，是**专家策略** π_{expert} 执行这些动作后得到的结果。

Algorithm 15 REINFORCE algorithm

Require: arbitrarily initialized π_θ

1: **repeat**
2: sample $\{\tau^i\}$ from $\pi_\theta(\mathbf{a}_t \mid \mathbf{s}_t)$ (run the policy)
3: $\nabla_\theta J(\theta) \approx \sum_i \left(\sum_t \nabla_\theta \log \pi_\theta\left(\mathbf{a}_t^i \mid \mathbf{s}_t^i\right)\right)\left(\sum_t r\left(\mathbf{s}_t^i, \mathbf{a}_t^i\right)\right)$
4: $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$
5: **until** convergence

Note. Markov property is not actually used in policy gradient. Therefore, you can use it in partially observed MDPs without modification.

3.1.2 Comparison to Maximum Likelihood

In policy gradient:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \left(\left(\sum_{t=1}^{T} \nabla_\theta \log \pi_\theta\left(\mathbf{a}_{i,t} \mid \mathbf{s}_{i,t}\right)\right)\left(\sum_{t=1}^{T} r\left(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}\right)\right)\right) \tag{3.1.8}$$

In maximum likelihood:

$$\nabla_\theta J_{\mathrm{ML}}(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \left(\sum_{t=1}^{T} \nabla_\theta \log \pi_\theta\left(\mathbf{a}_{i,t} \mid \mathbf{s}_{i,t}\right)\right) \tag{3.1.9}$$

Insight. Good stuff is made more likely; bad stuff is made less likely. What we just did in policy gradient simply formalize the notion of "through trial and error"!

3.1.3 The High Variance of Policy Gradient

One significant drawback of policy gradient is that the gradient is often noisy (i.e. the variance is high). Here are some methods of reducing variance.

Causality

利用因果性质，将原本的全轨迹奖励改为reward to go；这个权重原本是作用于一次采样过程i，现在改为这个采样过程i的每一步t使用不同的reward to go进行加权

Firstly, causality tells us that policy at time t' should not affect the reward at time t when $t \leq t'$. The modified gradient function is:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta\left(\mathbf{a}_{i,t} \mid \mathbf{s}_{i,t}\right) \left(\sum_{t'=t}^{T} r\left(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'}\right)\right) \tag{3.1.10}$$

The expression in brackets means "reward to go". And sometimes we also write it as $\hat{Q}_{i,t}^{\pi}$: estimate of expected reward if we take action $\mathbf{a}_{i,t}$ in state $\mathbf{s}_{i,t}$ ($\hat{Q}_{i,t}^{\pi} = \sum_{t'=t}^{T} r\left(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'}\right)$)

Baseline

The second approach is to introduce **baseline** to the reward part:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \nabla_\theta \log p_\theta(\tau)[r(\tau) - b], \quad b = \frac{1}{N} \sum_{i=1}^{N} r(\tau) \tag{3.1.11}$$

We are allowed to do that because what we really care about is *expectation* ($E\left[\nabla_\theta \log p_\theta(\tau) b\right]$), and subtracting a baseline is *unbiased in expectation*:

$$E\left[\nabla_\theta \log p_\theta(\tau) b\right] = \int p_\theta(\tau) \nabla_\theta \log p_\theta(\tau) b d\tau = \int \nabla_\theta p_\theta(\tau) b d\tau = b \nabla_\theta \int p_\theta(\tau) d\tau = b \nabla_\theta 1 = 0 \tag{3.1.12}$$

Here we use the average reward as the baseline. It is not the best baseline (as to reducing variance), but it works pretty good. If you are not satisfied with it, we can try to find the best baseline:

$$\mathrm{Var}[x] = E\left[x^2\right] - E[x]^2 \tag{3.1.13}$$

$$\nabla_\theta J(\theta) = E_{\tau \sim p_\theta(\tau)}\left[\nabla_\theta \log p_\theta(\tau)(r(\tau) - b)\right] \tag{3.1.14}$$

$$\text{Var} = E_{\tau\sim p_\theta(\tau)}\left[(\nabla_\theta \log p_\theta(\tau)(r(\tau)-b))^2\right] - E_{\tau\sim p_\theta(\tau)}\left[\nabla_\theta \log p_\theta(\tau)(r(\tau)-b)\right]^2 \tag{3.1.15}$$

$$\frac{d\,\text{Var}}{db} = \frac{d}{db}E\left[g(\tau)^2(r(\tau)-b)^2\right] = -2E\left[g(\tau)^2 r(\tau)\right] + 2bE\left[g(\tau)^2\right] = 0 \tag{3.1.16}$$

Solve this equation and you'll find the optimal baseline:

$$b = \frac{E\left[g(\tau)^2 r(\tau)\right]}{E\left[g(\tau)^2\right]} \tag{3.1.17}$$

Note that this is just expected reward, but weighted by gradient magnitudes!

3.1.4 More about reducing the variance

Before moving on, let's take a moment to review what it actually means to reduce the variance.

In a nutshell, in REINFORCE algorithm we are trying to optimize some function (i.e. the objective function $J(\theta)$) on a random variable (the return of some stochastic policy). What you need to do is sample a bunch of trajectories from the current policy, estimate the current expected value (remember, you are **sampling** trajectories so you don't know the expected value you can only **estimate** it), and update the parameters of the current policy in the direction that optimizes your estimate.

But you only have a finite amount of time to sample (at some point you have to update the parameters to make progress), so your estimate will be off of the true value and you will making updates based on that estimate. So let's say you are only going to sample 10 times, if the distribution of the values of that random variable has high variance, then sampling only 10 times will be a pretty poor estimate. If it has low variance, then 10 samples might be enough to approximate the value you want. Therefore we introduce the baseline to reduce the variance of the samples.

Another way of understanding *reducing the variance* is *reducing the aggressiveness of the updates*. Here is a concrete example: assuming we have a reward function that looks like

$$f(x) = \begin{cases} -|x|, & x < 0 \\ 0, & x \geq 0 \end{cases} \tag{3.1.18}$$

在"好"的一侧采样（`x > 0`）：

- 假设策略采样了一个动作，比如 `x = 0.1`。
- 根据奖励函数，`f(0.1) = 0`。
- 那么参数更新量就变成了 `∇_θ log π * 0 = 0`。
- **结论**：当智能体做出正确的行为时（逃离火场），它得到的奖励是0，导致梯度更新量也为0。换句话说，**它从成功的经验中学不到任何东西**。

通常，基线 `b` 被设为奖励的期望值（平均值），即 `b = E[R]`。

在这个场景中，因为智能体总会采样到一些负值，所以奖励的期望值 `b` 必然是一个**负数**（比如 `b = -0.5`）。

我们再来看两种情况：

1. **在"好"的一侧采样（`x > 0`）：**
   - 奖励 `R = 0`。
   - 更新项 `(R - b) = (0 - (-0.5)) = 0.5`。这是一个**正数**！
   - **结论**：现在，当智能体做出正确的行为时，它会得到一个正的 `(R-b)` 值，即所谓的"优势"（Advantage）。这会产生一个正向的梯度更新，鼓励策略未来更倾向于选择正方向的动作。**智能体开始能从成功的经验中学习了！**

3.2 Off-Policy Policy Gradients

If we revisit the derivation of policy differentiation, it's obvious that policy gradient is on-policy. The underlined part would be a trouble, because the neural networks change only a little bit with each gradient step, but you need to do the sampling all over again. Therefore, on-policy learning can be extremely inefficient!

$$\nabla_\theta J(\theta) = \underline{E_{\tau\sim p_\theta(\tau)}}\left[\nabla_\theta \log p_\theta(\tau) r(\tau)\right]$$

policy gradient是on-policy的，但是每次采样后神经网络都要更新导致inefficient

The question here is: what if we skip the sampling step? What if we don't have samples from $p_\theta(\tau)$ (we have samples from some $\bar{p}(\tau)$ instead)?

Before delving into this issue, let's talk about some maths first: *importance sampling*

Theorem 3.2 (Importance Sampling).

$$\begin{aligned} E_{x\sim p(x)}[f(x)] &= \int p(x)f(x)dx \\ &= \int \frac{q(x)}{q(x)}p(x)f(x)dx \\ &= \int q(x)\frac{p(x)}{q(x)}f(x)dx \\ &= E_{x\sim q(x)}\left[\frac{p(x)}{q(x)}f(x)\right] \end{aligned}$$

Its mathematical essence lies in using a *known distribution* (typically an easy-to-sample distribution) to estimate the expectation of another distribution (usually a difficult-to-sample distribution). Back to off-policy learning,

here we can turn the original objective function $J(\theta) = E_{\tau\sim p_\theta(\tau)}[r(\tau)]$, into a new objective function using importance sampling:

$$J(\theta) = E_{\tau\sim\bar{p}(\tau)}\left[\frac{p_\theta(\tau)}{\bar{p}(\tau)}r(\tau)\right] \tag{3.2.1}$$

Here we have:

$$p_\theta(\tau) = p(\mathbf{s}_1)\prod_{t=1}^{T}\pi_\theta(\mathbf{a}_t \mid \mathbf{s}_t)\,p(\mathbf{s}_{t+1} \mid \mathbf{s}_t, \mathbf{a}_t) \tag{3.2.2}$$

$$\frac{p_\theta(\tau)}{\bar{p}(\tau)} = \frac{p(\mathbf{s}_1)\prod_{t=1}^{T}\pi_\theta(\mathbf{a}_t \mid \mathbf{s}_t)\,p(\mathbf{s}_t, \mathbf{a}_t)}{p(\mathbf{s}_1)\prod_{t=1}^{T}\bar{\pi}(\mathbf{a}_t \mid \mathbf{s}_t)\,p(\mathbf{s}_t, \mathbf{a}_t)} = \frac{\prod_{t=1}^{T}\pi_\theta(\mathbf{a}_t \mid \mathbf{s}_t)}{\prod_{t=1}^{T}\bar{\pi}(\mathbf{a}_t \mid \mathbf{s}_t)} \tag{3.2.3}$$

for some new parameters θ', similarly we have:

想优化的新策略（需要更新的最终策略）
采样策略（用于采样的策略[off-policy]）

$$\frac{p_{\theta'}(\tau)}{p_\theta(\tau)} = \frac{\prod_{t=1}^{T}\pi_{\theta'}(\mathbf{a}_t \mid \mathbf{s}_t)}{\prod_{t=1}^{T}\pi_\theta(\mathbf{a}_t \mid \mathbf{s}_t)} \tag{3.2.4}$$

therefore,

$$\nabla_{\theta'}J(\theta') = E_{\tau\sim p_\theta(\tau)}\left[\frac{\nabla_{\theta'}p_{\theta'}(\tau)}{p_\theta(\tau)}r(\tau)\right] = E_{\tau\sim p_\theta(\tau)}\left[\frac{p_{\theta'}(\tau)}{p_\theta(\tau)}\nabla_{\theta'}\log p_{\theta'}(\tau)r(\tau)\right] \tag{3.2.5}$$

Now estimate locally, at $\theta = \theta'$, we have:

$$\nabla_\theta J(\theta) = E_{\tau\sim p_\theta(\tau)}\left[\nabla_\theta \log p_\theta(\tau)r(\tau)\right] \tag{3.2.6}$$

When $\theta \neq \theta'$, then:

$$\begin{aligned}\nabla_{\theta'}J(\theta') &= E_{\tau\sim p_\theta(\tau)}\left[\frac{p_{\theta'}(\tau)}{p_\theta(\tau)}\nabla_{\theta'}\log\pi_{\theta'}(\tau)r(\tau)\right] \\ &= E_{\tau\sim p_\theta(\tau)}\left[\left(\prod_{t=1}^{T}\frac{\pi_{\theta'}(\mathbf{a}_t \mid \mathbf{s}_t)}{\pi_\theta(\mathbf{a}_t \mid \mathbf{s}_t)}\right)\left(\sum_{t=1}^{T}\nabla_{\theta'}\log\pi_{\theta'}(\mathbf{a}_t \mid \mathbf{s}_t)\right)\left(\sum_{t=1}^{T}r(\mathbf{s}_t, \mathbf{a}_t)\right)\right]\end{aligned} \tag{3.2.7}$$

Also, take causality into consideration, future actions don't affect current weight:

对回报应用因果性: 在t时刻的梯度项不应该乘以整条轨迹的总回报，而只应该乘以从 t 时刻开始的reward to go

$$\nabla_{\theta'}J(\theta') = E_{\tau\sim p_\theta(\tau)}\left[\sum_{t=1}^{T}\nabla_{\theta'}\log\pi_{\theta'}(\mathbf{a}_t \mid \mathbf{s}_t)\left(\prod_{t'=1}^{t}\frac{\pi_{\theta'}(\mathbf{a}_{t'} \mid \mathbf{s}_{t'})}{\pi_\theta(\mathbf{a}_{t'} \mid \mathbf{s}_{t'})}\right)\left(\sum_{t'=t}^{T}r(\mathbf{s}_{t'}, \mathbf{a}_{t'})\right)\right] \tag{3.2.9}$$

对权重应用因果性: t时刻的梯度项重要性权重应该只取决于到达t时刻所经历的路径。权重应该是从开始到t时刻的概率比连乘

In the next section, we'll see that 3.2.9 is equivalent to policy iteration.

3.3 (Optional) Policy Gradient as Policy Iteration

Now we give further analysis of policy gradient as policy iteration. The trick here is to express the expected value under policy π_θ in terms of the expected value with repect to the trajectory τ under policy $\pi_{\theta'}$:

优势函数A(s,a)的由来：如何使用Policy Iteration-->
一个新策略θ'的性能J提升量，等于它所执行的一系列动作，用旧策略θ的价值观来衡量所产生的"优势"的总和的期望值

这部分推导的核心目标是建立新策略 $\pi_{\theta'}$ 相对于旧策略 π_θ 的**性能差异**与**优势函数 (Advantage Function)** 之间的关系，这是后续TRPO和PPO等高级策略梯度算法的理论基础。

公式 (3.3.1) 的推导

这一部分的目的是证明 $J(\theta') - J(\theta) = E_{\tau\sim p_{\theta'}(\tau)}[\sum_{t=0}^{\infty}\gamma^t A^{\pi_\theta}(s_t, a_t)]$。

第一行 -> 第二行: $J(\theta') - J(\theta) = J(\theta') - E_{s_0\sim p(s_0)}[V^{\pi_\theta}(s_0)] = J(\theta') - E_{\tau\sim p_{\theta'}(\tau)}[V^{\pi_\theta}(s_0)]$

- **原理**: 期望的恒等变换。
- **解释**:
  - 首先，根据定义，旧策略的目标函数值 $J(\theta)$ 等于在初始状态分布下值函数 $V^{\pi_\theta}(s_0)$ 的期望，即 $J(\theta) = E_{s_0\sim p(s_0)}[V^{\pi_\theta}(s_0)]$ 。
  - 接下来的关键一步（图中绿色高亮部分到蓝色高亮部分）是 $E_{s_0\sim p(s_0)}[V^{\pi_\theta}(s_0)] = E_{\tau\sim p_{\theta'}(\tau)}[V^{\pi_\theta}(s_0)]$ 。这一步之所以成立，是因为 $V^{\pi_\theta}(s_0)$ 的值**仅取决于初始状态** s_0，而与后续的轨迹 τ 是如何生成的无关。换句话说，无论我们接下来遵循哪个策略（π_θ 或 $\pi_{\theta'}$），对于一个给定的初始状态 s_0，其在旧策略 π_θ 下的价值是固定的。因此，在由新策略 $\pi_{\theta'}$ 生成的轨迹分布上对 $V^{\pi_\theta}(s_0)$ 求期望，等价于在初始状态分布上求期望。

第二行 -> 第三行: $= J(\theta') - E_{\tau\sim p_{\theta'}(\tau)}[\sum_{t=0}^{\infty}\gamma^t V^{\pi_\theta}(s_t) - \sum_{t=1}^{\infty}\gamma^t V^{\pi_\theta}(s_t)]$

- **原理**: 伸缩求和 (Telescoping Sum)。
- **解释**: 这一步是一个巧妙的数学技巧，将 $V^{\pi_\theta}(s_0)$ 展开成一个无穷级数的形式。
  - $V^{\pi_\theta}(s_0)$ 可以写作 $V^{\pi_\theta}(s_0) + 0$。
  - 这个 "0" 被构造成一个无穷级数的差：$(\gamma V^{\pi_\theta}(s_1) + \gamma^2 V^{\pi_\theta}(s_2) + ...) - (\gamma V^{\pi_\theta}(s_1) + \gamma^2 V^{\pi_\theta}(s_2) + ...)$。
  - 将 $V^{\pi_\theta}(s_0)$ 放入第一个求和中（当t=0时），就得到了 $\sum_{t=0}^{\infty}\gamma^t V^{\pi_\theta}(s_t) - \sum_{t=0}^{\infty}\gamma^{t+1} V^{\pi_\theta}(s_{t+1})$。
  - 调整第二个求和的下标，令 $t' = t+1$，则第二个求和变为 $\sum_{t'=1}^{\infty}\gamma^{t'} V^{\pi_\theta}(s_{t'})$，从而得到式中的结果 。

第三行 -> 第四行: $= J(\theta') + E_{\tau\sim p_{\theta'}(\tau)}[\sum_{t=0}^{\infty}\gamma^t(\gamma V^{\pi_\theta}(s_{t+1}) - V^{\pi_\theta}(s_t))]$

- **原理**: 合并求和项。
- **解释**: 将 $\sum_{t=1}^{\infty}\gamma^t V^{\pi_\theta}(s_t)$ 中的每一项 $\gamma^t V^{\pi_\theta}(s_t)$ 与 $\sum_{t=0}^{\infty}\gamma^t V^{\pi_\theta}(s_t)$ 中对应的 $\gamma^{t-1} V^{\pi_\theta}(s_{t-1})$ 合并，通过调整系数和下标，最终可以整理成括号内的形式 。简单来说，就是将两个求和重新组合。

第四行 -> 第五行: $= E_{\tau \sim p_{\theta'}(\tau)}[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)] + E_{\tau \sim p_{\theta'}(\tau)}[\sum_{t=0}^{\infty} \gamma^t (\gamma V^{\pi_\theta}(s_{t+1}) - V^{\pi_\theta}(s_t))]$

- **原理**: 代入目标函数的定义。
- **解释**: 根据定义，新策略的目标函数 $J(\theta')$ 是遵循该策略所能获得的累计折扣奖励的期望，即 $J(\theta') = E_{\tau \sim p_{\theta'}(\tau)}[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)]$ 。

第五行 -> 第六行: $= E_{\tau \sim p_{\theta'}(\tau)}[\sum_{t=0}^{\infty} \gamma^t (r(s_t, a_t) + \gamma V^{\pi_\theta}(s_{t+1}) - V^{\pi_\theta}(s_t))]$

- **原理**: 合并期望和求和。
- **解释**: 由于两个期望都是在同一个分布 $p_{\theta'}(\tau)$ 下计算的，并且求和的范围也相同，因此可以根据线性性质将它们合并 。

第六行 -> 第七行: $= E_{\tau \sim p_{\theta'}(\tau)}[\sum_{t=0}^{\infty} \gamma^t A^{\pi_\theta}(s_t, a_t)]$

- **原理**: 优势函数 (Advantage Function) 的定义。
- **解释**: 优势函数 $A^{\pi_\theta}(s_t, a_t)$ 衡量的是在状态 s_t 下，执行动作 a_t 相对于遵循策略 π_θ 的平均表现有多好。其定义为 $A^{\pi_\theta}(s_t, a_t) = Q^{\pi_\theta}(s_t, a_t) - V^{\pi_\theta}(s_t)$。根据贝尔曼方程，$Q^{\pi_\theta}(s_t, a_t) = r(s_t, a_t) + \gamma E[V^{\pi_\theta}(s_{t+1})]$。在单条轨迹的上下文中，这个期望可以用实际观测到的下一个状态 s_{t+1} 来近似，因此 $A^{\pi_\theta}(s_t, a_t) \approx r(s_t, a_t) + \gamma V^{\pi_\theta}(s_{t+1}) - V^{\pi_\theta}(s_t)$ 。将这个定义代入，便得到了最终结果。

公式 (3.3.2) 的推导

这一部分的目的是将对轨迹 τ 的期望，分解为对状态和动作的期望，并引入重要性采样。

第一行 -> 第二行: $E_{\tau\sim p_{\theta'}(\tau)}[\sum_t \gamma^t A^{\pi_\theta}(s_t,a_t)] = \sum_t E_{s_t\sim p_{\theta'}(s_t)}[E_{a_t\sim\pi_{\theta'}(a_t|s_t)}[\gamma^t A^{\pi_\theta}(s_t,a_t)]] = \sum_t E_{s_t\sim p_{\theta'}(s_t)}[E_{a_t\sim\pi_\theta(a_t|s_t)}[\frac{\pi_{\theta'}(a_t|s_t)}{\pi_\theta(a_t|s_t)}\gamma^t A^{\pi_\theta}(s_t,a_t)]]$

- **原理**: 期望的分解与重要性采样 (Importance Sampling)。
- **解释**:
  1. **期望分解**: 首先，对整条轨迹 τ 求期望，可以分解为对轨迹中各个时刻 t 的状态 s_t 求期望，再对给定状态 s_t 下的动作 a_t 求期望。由于轨迹是从策略 $\pi_{\theta'}$ 生成的，所以状态的分布是 $p_{\theta'}(s_t)$，动作的分布是 $\pi_{\theta'}(a_t|s_t)$ 。
  2. **重要性采样**: 接下来的关键一步（图中红色框 `3.2` 指向的部分）是应用了重要性采样。我们的目标是改变动作期望的采样分布，从新策略 $\pi_{\theta'}$ 变为旧策略 π_θ 。根据重要性采样定理 $E_{x\sim p(x)}[f(x)] = E_{x\sim q(x)}[\frac{p(x)}{q(x)}f(x)]$，我们可以将对 $a_t \sim \pi_{\theta'}$ 的期望，转换为对 $a_t \sim \pi_\theta$ 的期望，但代价是必须引入一个重要性权重 $\frac{\pi_{\theta'}(a_t|s_t)}{\pi_\theta(a_t|s_t)}$ 。

这个变换至关重要，因为它使得我们最终可以用旧策略 π_θ 采样的数据（动作 a_t）来评估新策略 $\pi_{\theta'}$ 的表现。

$$
\begin{aligned}
J(\theta') - J(\theta) &= J(\theta') - E_{\mathbf{s}_0 \sim p(\mathbf{s}_0)}\left[V^{\pi_\theta}(\mathbf{s}_0)\right] \\
&= J(\theta') - E_{\tau \sim p_{\theta'}(\tau)}\left[V^{\pi_\theta}(\mathbf{s}_0)\right] \\
&= J(\theta') - E_{\tau \sim p_{\theta'}(\tau)}\left[\sum_{t=0}^{\infty} \gamma^t V^{\pi_\theta}(\mathbf{s}_t) - \sum_{t=1}^{\infty} \gamma^t V^{\pi_\theta}(\mathbf{s}_t)\right] \\
&= J(\theta') + E_{\tau \sim p_{\theta'}(\tau)}\left[\sum_{t=0}^{\infty} \gamma^t \left(\gamma V^{\pi_\theta}(\mathbf{s}_{t+1}) - V^{\pi_\theta}(\mathbf{s}_t)\right)\right] \\
&= E_{\tau \sim p_{\theta'}(\tau)}\left[\sum_{t=0}^{\infty} \gamma^t r(\mathbf{s}_t, \mathbf{a}_t)\right] + E_{\tau \sim p_{\theta'}(\tau)}\left[\sum_{t=0}^{\infty} \gamma^t \left(\gamma V^{\pi_\theta}(\mathbf{s}_{t+1}) - V^{\pi_\theta}(\mathbf{s}_t)\right)\right] \\
&= E_{\tau \sim p_{\theta'}(\tau)}\left[\sum_{t=0}^{\infty} \gamma^t \left(r(\mathbf{s}_t, \mathbf{a}_t) + \gamma V^{\pi_\theta}(\mathbf{s}_{t+1}) - V^{\pi_\theta}(\mathbf{s}_t)\right)\right] \\
&= E_{\tau \sim p_{\theta'}(\tau)}\left[\sum_{t=0}^{\infty} \gamma^t A^{\pi_\theta}(\mathbf{s}_t, \mathbf{a}_t)\right]
\end{aligned}
\tag{3.3.1}
$$

Writing out the expectation with respect to the trajectory τ explicitly:

$$
\begin{aligned}
E_{\tau \sim p_{\theta'}(\tau)}\left[\sum_t \gamma^t A^{\pi_\theta}(\mathbf{s}_t, \mathbf{a}_t)\right] &= \sum_t E_{\mathbf{s}_t \sim p_{\theta'}(\mathbf{s}_t)}\left[E_{\mathbf{a}_t \sim \pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t)}\left[\gamma^t A^{\pi_\theta}(\mathbf{s}_t, \mathbf{a}_t)\right]\right] \\
&= \sum_t E_{\mathbf{s}_t \sim p_{\theta'}(\mathbf{s}_t)}\left[E_{\mathbf{a}_t \sim \pi_\theta(\mathbf{a}_t|\mathbf{s}_t)}\left[\frac{\pi_{\theta'}(\mathbf{a}_t \mid \mathbf{s}_t)}{\pi_\theta(\mathbf{a}_t \mid \mathbf{s}_t)} \gamma^t A^{\pi_\theta}(\mathbf{s}_t, \mathbf{a}_t)\right]\right]
\end{aligned}
\tag{3.3.2}
$$

where the second equation is obtained by using the importance sampling formula 3.2. Now the remaining question is: can we use p'_θ instead of p_θ in the expectation? The answer is yes, because we can actually bound the distribution gap between p_θ and p'_θ if the two policies π_θ and $\pi_{\theta'}$ are close enough.

3.4 (Optional) Bounding the Distribution Gap

3.5 (Optional) Advanced Policy Gradient Methods

In this section, we'll introduce the ideas of two advanced policy gradient methods. Feel free to skip this section if you are not interested in mathy details.

3.5.1 Trust Region Policy Optimization (TRPO)

Add TRPO algorithm

3.5.2 Proximal Policy Optimization (PPO)

Add PPO algorithm

3.6 Implementation Tips

There are some tips in implementing policy gradient:

1. Remember that the gradient has high variance. This isn't the same as supervised learning, actually the gradients would be really noisy.
2. Consider using much larger batches.
3. Tweaking learning rates is very hard.(We'll learn about policy gradient-specific learning rate adjustment methods later)

$$J(\theta') - J(\theta) = \sum_t E_{s_t \sim p_{\theta'}(s_t)}[E_{a_t \sim \pi_\theta(a_t|s_t)}[\frac{\pi_{\theta'}(a_t|s_t)}{\pi_\theta(a_t|s_t)}\gamma^t A^{\pi_\theta}(s_t, a_t)]]$$

这个公式虽然解决了动作采样的问题（可以在旧策略 π_θ 下采样），但状态分布 $p_{\theta'}(s_t)$ 依然依赖于新策略。TRPO 和 PPO 的核心思想就是：**如果我们能保证新旧策略足够接近，那么我们就可以近似认为它们的状态访问分布也是相似的 ($p_{\theta'}(s_t) \approx p_\theta(s_t)$)，从而可以用旧策略采集的样本来安全地优化新策略。**

TRPO 和 PPO 的根本目标就是解决这个问题：**如何在最大化策略提升的同时，避免由于更新步长过大而导致的策略崩溃。**

2. TRPO (Trust Region Policy Optimization) - 信赖域策略优化

TRPO 是第一个从理论上正面解决上述问题的算法。它的核心思想是：**在每次更新时，我们定义一个以当前策略为中心的“信赖域”（Trust Region）。我们只在这个信赖域内寻找能最大化策略性能的新策略。**

2.1 TRPO 的优化目标

TRPO 将策略更新构建为一个带约束的优化问题。它优化的目标（被称为“代理目标函数”，Surrogate Objective）是在旧策略 $\pi_{\theta_{old}}$ 的数据上评估新策略 π_θ 的优势：

$$\underset{\theta}{\text{maximize}} \quad L_{\theta_{old}}(\theta) = E_{s \sim p_{\theta_{old}}, a \sim \pi_{\theta_{old}}}[\frac{\pi_\theta(a|s)}{\pi_{\theta_{old}}(a|s)} A^{\pi_{\theta_{old}}}(s, a)]$$

同时，它增加了一个关键的约束，即新旧策略的“距离”不能超过一个阈值 δ。这个距离使用**KL散度 (Kullback-Leibler Divergence)** 来度量，因为它能很好地衡量两个概率分布的差异。

$$\text{subject to} \quad E_{s \sim p_{\theta_{old}}}[D_{KL}(\pi_{\theta_{old}}(\cdot|s)||\pi_\theta(\cdot|s))] \leq \delta$$

总结一下 TRPO 的优化问题：

$$\theta_{k+1} = \arg\max_\theta L_{\theta_k}(\theta) \quad \text{s.t.} \quad \bar{D}_{KL}(\theta||\theta_k) \leq \delta$$

2.2 TRPO 的挑战与实现

- **理论优雅，实现复杂**：直接求解上述带KL约束的问题非常困难。
- **近似求解**：TRPO 使用二阶优化方法。它对目标函数进行一阶泰勒展开，对KL约束进行二阶泰lere展开，最终转化为一个可以通过**共轭梯度法 (Conjugate Gradient)** 高效求解的问题。其中涉及到了**费雪信息矩阵 (Fisher Information Matrix)** 的计算。
- **优点**：理论基础坚实，更新非常稳定，单调性能提升在理论上有所保证。
- **缺点**：实现极其复杂，计算开销大（尤其是在处理大模型如 CNN/RNN 时），并且与现代深度学习框架（如PyTorch, TensorFlow）中主流的一阶优化器（如Adam）不兼容。

PPO 没有像 TRPO 那样使用硬性的 KL 约束，而是通过修改目标函数（Objective Function）的方式来间接实现“信赖域”的效果。最主流的 PPO 变体是 **PPO-Clip**。

3.1 PPO-Clip 的核心思想：裁剪目标函数

PPO-Clip 的思想非常巧妙。它定义了一个概率比率 (Probability Ratio):

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$$

这个比率 $r_t(\theta)$ 表示新旧策略在同一样本 (s_t, a_t) 上输出动作概率的比值。

- $r_t(\theta) > 1$：新策略更倾向于采取动作 a_t。
- $r_t(\theta) < 1$：新策略不太倾向于采取动作 a_t。

PPO-Clip 的目标函数如下：

$$L^{CLIP}(\theta) = E_t[\min(r_t(\theta)A_t, \quad \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t)]$$

让我们来详细解读这个公式：

- A_t 是在 t 时刻的优势函数 $A^{\pi_{\theta_{old}}}(s_t, a_t)$ 的估计值。
- ϵ 是一个超参数（通常为 0.1 或 0.2），它定义了裁剪的范围。
- $\text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)$ 的意思是将 $r_t(\theta)$ 的值限制在 $[1-\epsilon, 1+\epsilon]$ 区间内。

3.2 PPO-Clip 如何工作?

PPO-Clip 的精髓在于 `min` 函数的选择，我们需要分两种情况讨论优势函数 A_t 的正负：

1. **当 $A_t > 0$ 时 (这是一个“好”的动作，我们想增大其概率)**:
   - 目标函数变为 $L(\theta) = E_t[\min(r_t(\theta)A_t, (1+\epsilon)A_t)]$。
   - 如果 $r_t(\theta)$ 过大（超过 $1+\epsilon$），那么 min 会选择 $(1+\epsilon)A_t$ 这一项。这意味着，尽管我们可以通过大幅提高 r_t 来获得更大的目标函数值，但PPO会将其“裁剪”，施加一个上限。这**防止了策略更新过分激进**，即使这是一个好的动作。
2. **当 $A_t < 0$ 时 (这是一个“坏”的动作，我们想减小其概率)**:
   - 目标函数变为 $L(\theta) = E_t[\max(r_t(\theta)A_t, (1-\epsilon)A_t)]$。（注意：因为 A_t 是负数，所以 `min` 的效果等同于 `max`）
   - 如果 $r_t(\theta)$ 过小（低于 $1-\epsilon$），那么 `max` 会选择 $(1-\epsilon)A_t$ 这一项。这相当于给目标函数施加了一个下限。这**防止了为了惩罚一个坏动作而过度降低其概率**，从而导致策略过于确定而丧失探索性。

Chapter 4 Actor-Critic Methods

4.1 Introducing the Actor-Critic Methods

4.1.1 Recap

We mention Q function and V function here again as a recap.

$$Q^{\pi}\left(\mathbf{s}_t, \mathbf{a}_t\right) = \sum_{t'=t}^{T} E_{\pi_\theta}\left[r\left(\mathbf{s}_{t'}, \mathbf{a}_{t'}\right) \mid \mathbf{s}_t, \mathbf{a}_t\right] : \text{ total reward from taking } \mathbf{a}_t \text{ in } \mathbf{s}_t$$

$$V^{\pi}\left(\mathbf{s}_t\right) = E_{\mathbf{a}_t \sim \pi_\theta\left(\mathbf{a}_t \mid \mathbf{s}_t\right)}\left[Q^{\pi}\left(\mathbf{s}_t, \mathbf{a}_t\right)\right] : \text{ total reward from } \mathbf{s}_t$$

$$A^{\pi}\left(\mathbf{s}_t, \mathbf{a}_t\right) = Q^{\pi}\left(\mathbf{s}_t, \mathbf{a}_t\right) - V^{\pi}\left(\mathbf{s}_t\right) : \text{ how much better } \mathbf{a}_t \text{ is than average}$$

Policy gradient:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta\left(\mathbf{a}_{i,t} \mid \mathbf{s}_{i,t}\right) Q\left(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}\right) \tag{4.1.1}$$

Adding a baseline:

$$\begin{aligned} \nabla_\theta J(\theta) &\approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta\left(\mathbf{a}_{i,t} \mid \mathbf{s}_{i,t}\right)\left(Q\left(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}\right) - V\left(\mathbf{s}_{i,t}\right)\right) \\ &\approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta\left(\mathbf{a}_{i,t} \mid \mathbf{s}_{i,t}\right) A^{\pi}\left(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}\right) \end{aligned} \tag{4.1.2}$$

4.1.2 Policy Evaluation

Let's go back to three basics steps of RL: **generate samples** (i.e. run the policy), **fit a model to estimate return**, and **improve the policy**. In policy gradient, we figure out how to calculate $\nabla_\theta J(\theta)$, which tells us how to improve the policy. But we still need to fit the model, which quantatively tell us how good the policy is. We have three possible quantities, Q^{π}, V^{π} or A^{π} , so the question is: *what* should we fit to *what*? Since the current reward of taking $\mathbf{a}_t$ at $\mathbf{s}_t$ is fixed, so we can rewrite $Q^{\pi}\left(\mathbf{s}_t, \mathbf{a}_t\right)$ and $A^{\pi}\left(\mathbf{s}_t, \mathbf{a}_t\right)$:

$$\begin{aligned} Q^{\pi}\left(\mathbf{s}_t, \mathbf{a}_t\right) &= r\left(\mathbf{s}_t, \mathbf{a}_t\right) + \sum_{t'=t+1}^{T} E_{\pi_\theta}\left[r\left(\mathbf{s}_{t'}, \mathbf{a}_{t'}\right) \mid \mathbf{s}_t, \mathbf{a}_t\right] \\ &\approx r\left(\mathbf{s}_t, \mathbf{a}_t\right) + V^{\pi}\left(\mathbf{s}_{t+1}\right) \end{aligned} \tag{4.1.3}$$

$$A^{\pi}\left(\mathbf{s}_t, \mathbf{a}_t\right) \approx r\left(\mathbf{s}_t, \mathbf{a}_t\right) + V^{\pi}\left(\mathbf{s}_{t+1}\right) - V^{\pi}\left(\mathbf{s}_t\right) \tag{4.1.4}$$

Therefore V^{π} would be a nice choice, since Q^{π} and A^{π} depend on both actions and states, while V^{π} depends on solely states. Of course this is not the only option in actor-critic algorithm. We'll talk about that later.

Actually calculating the value function is essentially calculating the expectation of reward under a given policy. That's why the process of fitting value fuction is also called policy evaluation.

Theorem 4.1 (Monte Carlo policy evaluation). Monte Carlo policy evaluation estimates the value of state-action pairs based on random sampling of experiences. It involves averaging the returns observed from multiple episodes to approximate the true value function for a given policy.

$$V^{\pi}\left(\mathbf{s}_t\right) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t'=t}^{T} r\left(\mathbf{s}_{t'}, \mathbf{a}_{t'}\right)$$

与 **Policy Gradient 的关系**：Actor-Critic 正是建立在使用优势函数的 Policy Gradient 算法之上的。在传统的 PG（如 REINFORCE）算法中，A^π 是通过运行完整的轨迹（episode）然后计算得到的。而在 Actor-Critic 中，我们不再完整地运行轨迹来计算它，而是训练一个“评论家”（Critic）网络来直接估计这个优势函数或与之相关的值函数。

Note that in traditional Monte Carlo policy evaluation you need to generate samples from the same initial state, which means constantly resetting the simulator. Another way of doing that is training a neural network to estimate value function:

$$\text{training data: } \left\{\left(\mathbf{s}_{i,t}, \sum_{t'=t}^{T} r\left(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'}\right)\right)\right\}$$

$$\text{supervised regression: } \mathcal{L}(\phi) = \frac{1}{2}\sum_i \left\|\hat{V}_\phi^\pi\left(\mathbf{s}_i\right) - y_i\right\|^2$$

In fact, the value function is very intuitive. For example, when training an agent to play a chess-like game, if we set the probability of winning the game to 1 and the probability of losing to 0, the value function typically indicates the probability of eventually winning the game in the current state.

4.1.3 From Evaluation to Actor Critic

Now that we have learned policy evaluation and policy gradients, now we are able to put the two pieces together. And that makes an actor-critic algorithm:

Algorithm 16 Batch actor-critic algorithm

1: **repeat**
2: sample $\{\tau^i\}$ from $\pi_\theta\left(\mathbf{a}_t \mid \mathbf{s}_t\right)$ (run the policy) 轨迹
3: fit $\hat{V}_\phi^\pi(\mathbf{s})$ to sampled reward sums
4: evaluate $\hat{A}^\pi\left(\mathbf{s}_i, \mathbf{a}_i\right) = r\left(\mathbf{s}_i, \mathbf{a}_i\right) + \hat{V}_\phi^\pi\left(\mathbf{s}_i'\right) - \hat{V}_\phi^\pi\left(\mathbf{s}_i\right)$
5: $\nabla_\theta J(\theta) \approx \sum_i \nabla_\theta \log \pi_\theta\left(\mathbf{a}_i \mid \mathbf{s}_i\right) \hat{A}^\pi\left(\mathbf{s}_i, \mathbf{a}_i\right)$
6: $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$
7: **until** convergence

1. **Actor** $\pi_\theta(a|s)$：一个策略网络，用于输出动作。
2. **Critic** $\hat{V}_\phi^\pi(s)$：一个价值网络，参数为 ϕ，用于估计真实的状态价值 $V^\pi(s)$。

我们使用 Critic 网络的输出 $\hat{V}_\phi^\pi(s)$ 来计算优势函数的估计值：

$$\hat{A}^\pi(s,a) = r(s,a) + \hat{V}_\phi^\pi(s') - \hat{V}_\phi^\pi(s)$$

Note that while fitting the value function, we are fitting the sum of reward in a given episode length T. What if T is ∞ ? In many cases $\hat{V}_\phi^\pi$ can get infinitely large. A simple trick here is to all discount factor γ, which would tell the policy that its better to get rewards sooner than later.

Without discount factors, the target function and loss function of the neural network is:

$$\begin{aligned} y_{i,t} &\approx r\left(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}\right) + \hat{V}_\phi^\pi\left(\mathbf{s}_{i,t+1}\right) \\ \mathcal{L}(\phi) &= \tfrac{1}{2}\textstyle\sum_i \left\|\hat{V}_\phi^\pi\left(\mathbf{s}_i\right) - y_i\right\|^2 \end{aligned} \tag{4.1.5}$$

Now adding the discount factors, we'll have:

$$y_{i,t} \approx r\left(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}\right) + \gamma \hat{V}_\phi^\pi\left(\mathbf{s}_{i,t+1}\right), \quad \gamma \in [0,1] \text{ (0.99 works well)} \tag{4.1.6}$$

Insight. One way of understanding discount factors is that we change the MDP by adding an extra state of death. Once we enter the death state we never leave, and the reward is zero. In each state the agent has the probability of $1-\gamma$ of falling into the death state. 其实就是未来的状态存在一定概率的未知危险

4.1.4 Aside: the Discount Factor

Then how do we introduce discount factors to (Monte Carlo) policy gradients? Actually there seems to be two ways of doing this.

$$\begin{aligned} \text{option 1: } & \nabla_\theta J(\theta) \approx \frac{1}{N}\sum_{i=1}^{N}\sum_{t=1}^{T} \nabla_\theta \log \pi_\theta\left(\mathbf{a}_{i,t} \mid \mathbf{s}_{i,t}\right)\left(\sum_{t'=t}^{T} \gamma^{t'-t} r\left(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'}\right)\right) \\ \text{option 2: } & \nabla_\theta J(\theta) \approx \frac{1}{N}\sum_{i=1}^{N}\left(\sum_{t=1}^{T} \nabla_\theta \log \pi_\theta\left(\mathbf{a}_{i,t} \mid \mathbf{s}_{i,t}\right)\right)\left(\sum_{t=1}^{T} \gamma^{t-1} r\left(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'}\right)\right) \end{aligned} \tag{4.1.7}$$

The only difference is whether we introduce the discount factors before or after we use the causality rules.

To showcase the differences more effectively, we can rewrite the expression of option 2 in the form of option 1.

$$\begin{aligned}\nabla_\theta J(\theta) &\approx \frac{1}{N}\sum_{i=1}^{N}\left(\sum_{t=1}^{T}\nabla_\theta \log \pi_\theta\left(\mathbf{a}_{i,t} \mid \mathbf{s}_{i,t}\right)\right)\left(\sum_{t=1}^{T}\gamma^{t-1} r\left(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'}\right)\right) \\ &= \frac{1}{N}\sum_{i=1}^{N}\sum_{t=1}^{T}\nabla_\theta \log \pi_\theta\left(\mathbf{a}_{i,t} \mid \mathbf{s}_{i,t}\right)\left(\sum_{t'=t}^{T}\gamma^{t'}\left(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'}\right)\right) \\ &= \frac{1}{N}\sum_{i=1}^{N}\sum_{t=1}^{T}\gamma^{t-1}\nabla_\theta \log \pi_\theta\left(\mathbf{a}_{i,t} \mid \mathbf{s}_{i,t}\right)\left(\sum_{t'=t}^{T}\gamma^{t'-t} r\left(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'}\right)\right)\end{aligned} \tag{4.1.8}$$

将后一个括号放进前一个括号的求和内部，需要更改γ项对应的次数

Option 2 极大地看重早期决策的梯度，而严重地忽略（或“折扣”）了后期决策的梯度。它认为“在轨迹初期做对决策，远比在后期做对决策重要”。

Note that γ^{t-1} comes before $\nabla_\theta \log \pi_\theta$. Then it becomes much clearer. Option 2 implies that we not only care less about the rewards in the future, we also care less about the decisions in the future. As a result you are discounting the gradients. In other word, making right decision in the first time step is more important than making right decision in future time steps. If you are solving a real discount problem, this is exactly what we are trying to do. Because remember the modified MDP settings, later steps don't matter when you are dead. **But**, in reality, this is often not quite what we want. The version that we actually use is **option 1**.

Option 2 的本质，其实就是在**标准 REINFORCE 算法 (Option 1) 的基础上，给每个时间步的梯度项** $\nabla_\theta \log \pi_\theta$ **额外乘上了一个折扣因子** γ^{t-1}。

Take some time to review why we introduced the discount factor. For tasks with particularly long time episodes, we introduced the discount factor to artificially reduce the rewards for future actions in order to compute the value function. However, in practical tasks, we do not want to do this. For example, if we want a robot to run steadily forward, we actually want it to keep running. We want to learn a policy that can do the right thing for a long time, rather than just at nearby timesteps. Therefore, option 1 becomes more reasonable.

Algorithm 17 Batch actor-critic algorithm (with discount)

1: **repeat**
2: sample $\left\{\tau^i\right\}$ from $\pi_\theta\left(\mathbf{a}_t \mid \mathbf{s}_t\right)$ (run the policy)
3: fit $\hat{V}_\phi^\pi(\mathbf{s})$ to sampled reward sums
4: evaluate $\hat{A}^\pi\left(\mathbf{s}_i, \mathbf{a}_i\right) = r\left(\mathbf{s}_i, \mathbf{a}_i\right) + \gamma \hat{V}_\phi^\pi\left(\mathbf{s}_i'\right) - \hat{V}_\phi^\pi\left(\mathbf{s}_i\right)$
5: $\nabla_\theta J(\theta) \approx \sum_i \nabla_\theta \log \pi_\theta\left(\mathbf{a}_i \mid \mathbf{s}_i\right) \hat{A}^\pi\left(\mathbf{s}_i, \mathbf{a}_i\right)$
6: $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$
7: **until** convergence

The only difference that in step 3 we introduce the discount factor. In previous discussions we have been using policy gradients in episodic batch mode settings. If we modify a little bit, we'll get the online actor-critic algorithm:

Algorithm 18 Online actor-critic algorithm (with discount)

1: **repeat**
2: take action $\mathbf{a} \sim \pi_\theta(\mathbf{a} \mid \mathbf{s})$, get $(\mathbf{s}, \mathbf{a}, \mathbf{s}', r)$
3: update $\hat{V}_\phi^\pi(\mathbf{s})$ using target $r + \gamma \hat{V}_\phi^\pi\left(\mathbf{s}'\right)$
4: evaluate $\hat{A}^\pi\left(\mathbf{s}, \mathbf{a}\right) = r\left(\mathbf{s}, \mathbf{a}\right) + \gamma \hat{V}_\phi^\pi\left(\mathbf{s}'\right) - \hat{V}_\phi^\pi\left(\mathbf{s}\right)$
5: $\nabla_\theta J(\theta) \approx \nabla_\theta \log \pi_\theta(\mathbf{a} \mid \mathbf{s}) \hat{A}^\pi(\mathbf{s}, \mathbf{a})$
6: $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$
7: **until** convergence

Insight. In actor-critic algorithm, **actor** is the policy, and **critic** is the value function. It can be viewed as a improved version of policy gradient, with reduced variance.

公式 (4.2.1): 两种梯度计算方法的对比

1． **Actor-Critic 的梯度**:

$$\nabla_\theta J(\theta) \approx \ldots \nabla_\theta log\ \pi_\theta(\ldots) \underbrace{(r(s_{i,t}, a_{i,t}) + \gamma \hat{V}_\phi^\pi(s_{i,t+1}) - \hat{V}_\phi^\pi(s_{i,t}))}_{\text{TD 优势估计}}$$

- **推导细节**: 括号中的部分是优势函数 $A(s,a)$ 的一种估计，称为时序差分误差（TD Error）。它基于贝尔曼方程 $Q^\pi(s,a) = E[r + \gamma V^\pi(s')]$，并用 $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$ 推导得出。这里我们用评论家网络 $\hat{V}_\phi^\pi$ 来近似真实的 V^π。
- **关键点**: 这个估计只用到了**一步**的真实奖励 r 和下一步的状态价值估计 $\hat{V}_\phi^\pi(s_{i,t+1})$。因为随机性来源少（只有一个 r 和一次状态转移），所以它的**方差（Variance）较低**。但由于 $\hat{V}_\phi^\pi$ 只是一个近似，可能不准确，因此这个估计是**有偏的（Biased）**。

2． **Policy Gradient 的梯度**:

$$\nabla_\theta J(\theta) \approx \ldots \nabla_\theta log\ \pi_\theta(\ldots) \underbrace{((\sum_{t'=t}^{T} \gamma^{t'-t} r(s_{i,t'}, a_{i,t'})) - b)}_{\text{MC 优势估计}}$$

- **推导细节**: 括号中的部分是优势函数的另一种估计。$\sum \ldots r$ 是从当前时刻 t 直到轨迹结束的所有未来折扣奖励的总和，称为蒙特卡洛（Monte Carlo）回报，它是对 $Q^\pi(s,a)$ 的无偏估计。b 是一个基线（Baseline），通常选择为 $\hat{V}_\phi^\pi(s_t)$。
- **关键点**: 这个估计使用了**整条轨迹**未来的所有真实奖励，它在统计上是**无偏的（Unbiased）**。但由于累加了多个随机的奖励值，其结果波动很大，因此**方差（Variance）非常高**。

公式 (4.2.2): 无偏的 PG 优势估计

$$\hat{A}^\pi(s_t, a_t) = (\sum_{t'=t}^{T} \gamma^{t'-t} r(s_{i,t'}, a_{i,t'})) - \hat{V}_\phi^\pi(s_{i,t})$$

- **推导细节**: 这是将公式 (4.2.1) 中 Policy Gradient 的基线 b 明确替换为评论家网络 $\hat{V}_\phi^\pi(s_{i,t})$ 的结果。
- **关键点**: 这是最经典的优势函数估计形式，在 A2C (Advantage Actor-Critic) 等算法中广泛使用。它结合了蒙特卡洛回报（无偏的Q值估计）和价值函数基线（用于降低方差）。

公式 (4.2.5) & (4.2.6): 控制变量 (Control Variates)

- **推导细节**: 公式 (4.2.5) 提出了一种新的基线思路：不用状态价值函数 $V(s)$ 作基线，而是用动作价值函数 $Q(s,a)$ 作基线。
- **关键点 (非常重要)**:
  - 当基线**只与状态 s 有关**时（如 $V(s)$），可以直接从梯度中减去，这在数学上是严格无偏的。
  - 但是，当基线**与动作 a 也有关**时（如 $Q(s,a)$），直接减去就会引入偏差。因为此时策略梯度不仅会优化原始奖励，还会“错误地”去优化这个基线本身。
  - 公式 (4.2.6) 给出了修正这个偏差的方法：必须加上一个**修正项**（公式中的第二项）。这个修正项的作用是抵消掉优化基线所带来的错误梯度。
  - **结论**: 除非进行特殊处理（如公式4.2.6），否则基线函数不能依赖于动作 a。

公式 (4.2.7): 两种优势函数估计的总结

- A^{π}_{C} (Actor-Critic TD 估计): **低方差，高偏差** (如果价值函数不准)。
- $\hat{A}^{\pi}_{MC}$ (Policy Gradient MC 估计): **高方差，无偏差** (统计意义上)。

GAE 的目标是在上述的“高偏差/低方差”和“无偏差/高方差”之间找到一个最佳的平衡点。

公式 (4.2.8): n-步回报优势估计器

- **推导细节**: 传统的 Actor-Critic 使用1步回报，传统的 Policy Gradient 使用无限步（整条轨迹）回报。n-步回报则是一个折中方案，它使用未来 n 步的真实奖励，并在第 n 步时使用价值函数 $\hat{V}^{\pi}_{\phi}(s_{t+n})$ 来估计之后的所有回报。
- **关键点**:
  - 通过调整 n 的大小，我们可以控制偏差和方差的权衡。
  - 当 $n=1$ 时，它退化为 Actor-Critic 的 TD 优势估计（高偏差，低方差）。
  - 当 $n \to \infty$ 时，它就变成了 Policy Gradient 的 MC 优势估计（无偏差，高方差）。

公式 (4.2.9): GAE 的定义

$$\hat{A}^{\pi}_{GAE}(s_t, \mathbf{a}_t) = \sum_{n=1}^{\infty} w_n \hat{A}^{\pi}_{n}(s_t, \mathbf{a}_t)$$

- **推导细节**: GAE 的思想非常巧妙：我们不选择单一固定的 n，而是将所有可能的 n-步优势估计 $\hat{A}^{\pi}_{n}$ 进行加权平均。

公式 (4.2.10): GAE 的实用计算公式

$$\hat{A}^{\pi}_{GAE}(s_t, a_t) = \sum_{t'=t}^{\infty} (\gamma\lambda)^{t'-t}\delta_{t'} \quad \text{其中 } \delta_{t'} = r(s_{t'}, a_{t'}) + \gamma\hat{V}^{\pi}_{\phi}(s_{t'+1}) - \hat{V}^{\pi}_{\phi}(s_{t'})$$

- **推导细节 (核心)**: 当权重 w_n 被设置为与 λ^{n-1} 成正比的指数衰减形式时（其中 $\lambda \in [0, 1]$ 是一个超参数），经过数学上的重新整理（对各项 $\delta_{t'}$ 合并同类项），上面复杂的加权求和可以被简化为这个极其优雅的形式。
- **关键点**:
  - 这个公式的含义是：GAE 优势估计等于未来所有 TD 误差 δ 的折扣累加和。
  - 折扣因子不再是单纯的 γ，而是变成了 $(\gamma\lambda)$。
  - **超参数 λ 成为了控制偏差-方差的“旋钮”**:
    - 当 $\lambda = 0$ 时，$(\gamma\lambda)$ 为0，GAE 只剩下第一项 δ_t，完全等价于 Actor-Critic 的 TD 优势估计（高偏差，低方差）。
    - 当 $\lambda = 1$ 时，GAE 等价于 Policy Gradient 的 MC 优势估计（无偏差，高方差）。
  - 在实践中，通常选择一个介于0和1之间的 λ 值（例如0.95），可以在偏差和方差之间取得很好的平衡，从而获得最佳的训练效果。这是 PPO、TRPO 等现代强化学习算法的标准配置。

$\hat{A}^{\pi}_{n}$ 这是使用未来 n 步真实奖励和第 n 步的价值估计来计算的优势。

$$\hat{A}^{\pi}_{n}(s_t, a_t) = \left(\sum_{k=0}^{n-1} \gamma^k r_{t+k}\right) + \gamma^n \hat{V}(s_{t+n}) - \hat{V}(s_t)$$

GAE 的定义 (公式 4.2.9) GAE 是所有 n-步优势估计的加权平均 。笔记中提到，权重 w_n 采用指数衰减形式，即 $w_n \propto \lambda^{n-1}$ 。为了使权重之和为1，我们使用归一化的权重 $w_n = (1-\lambda)\lambda^{n-1}$。

$$\hat{A}^{\pi}_{GAE} = \sum_{n=1}^{\infty} (1-\lambda)\lambda^{n-1}\hat{A}^{\pi}_{n}$$

$\hat{A}^{\pi}_{1} = r_t + \gamma\hat{V}(s_{t+1}) - \hat{V}(s_t) = \delta_t$ **推广规律**：通过数学归纳法，我们可以证明：

$$\hat{A}^{\pi}_{n} = \sum_{k=0}^{n-1} \gamma^k \delta_{t+k}$$

n-步优势估计等于未来 n 个 TD 误差的折扣累加和。

$$\hat{A}^{\pi}_{GAE} = \sum_{n=1}^{\infty} (1-\lambda)\lambda^{n-1}\left(\sum_{k=0}^{n-1} \gamma^k \delta_{t+k}\right) = (\gamma\lambda)^0\delta_t + (\gamma\lambda)^1\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \cdots = \sum_{k=0}^{\infty} (\gamma\lambda)^k \delta_{t+k}$$

4.2 Further Analysis

4.2.1 Critics as Baselines

Let's make a comparison here between the actor-critic algorithm and the policy gradient algorithm:

$$\begin{aligned}&\text{Actor-critic:} \quad \nabla_\theta J(\theta) \approx \frac{1}{N}\sum_{i=1}^{N}\sum_{t=1}^{T}\nabla_\theta \log \pi_\theta\left(\mathbf{a}_{i,t} \mid \mathbf{s}_{i,t}\right)\left(r\left(\mathbf{s}_{i,t},\mathbf{a}_{i,t}\right)+\gamma\hat{V}_\phi^\pi\left(\mathbf{s}_{i,t+1}\right)-\hat{V}_\phi^\pi\left(\mathbf{s}_{i,t}\right)\right)\\&\text{Policy gradient:} \quad \nabla_\theta J(\theta) \approx \frac{1}{N}\sum_{i=1}^{N}\sum_{t=1}^{T}\nabla_\theta \log \pi_\theta\left(\mathbf{a}_{i,t} \mid \mathbf{s}_{i,t}\right)\left(\left(\sum_{t'=t}^{T}\gamma^{t'-t}r\left(\mathbf{s}_{i,t'},\mathbf{a}_{i,t'}\right)\right)-b\right)\end{aligned} \tag{4.2.1}$$

In actor-critic, we have lower variance since we have the critic, but it is not unbiased if the critic is not perfect. While in policy gradient, it is unbiased but the variance would be high due to the single-sample estimate.

So intuitively a nature step we can take is to use $\hat{V}_\phi^\pi$ while still keep the estimater unbiased:

$$\nabla_\theta J(\theta) \approx \frac{1}{N}\sum_{i=1}^{N}\sum_{t=1}^{T}\nabla_\theta \log \pi_\theta\left(\mathbf{a}_{i,t} \mid \mathbf{s}_{i,t}\right)\left(\left(\sum_{t'=t}^{T}\gamma^{t'-t}r\left(\mathbf{s}_{i,t'},\mathbf{a}_{i,t'}\right)\right)-\hat{V}_\phi^\pi\left(\mathbf{s}_{i,t}\right)\right) \tag{4.2.2}$$

Up to now we are all using the state-dependent functions as baselines. So can we use functions that involves both action and state as baselines? Here we come to control variates, which means baselines that are action-dependent. Here we are going to talk about that.

Baselines that use both actions and states are also called *control variates*. The true advantage function is:

$$\begin{aligned}A^\pi\left(\mathbf{s}_t,\mathbf{a}_t\right) &= Q^\pi\left(\mathbf{s}_t,\mathbf{a}_t\right)-V^\pi\left(\mathbf{s}_t\right)\\&=\sum_{t'=t}^{T}E_{\pi_\theta}\left[r\left(\mathbf{s}_{t'},\mathbf{a}_{t'}\right) \mid \mathbf{s}_t,\mathbf{a}_t\right]-E_{\mathbf{a}_t\sim\pi_\theta\left(\mathbf{a}_t\mid\mathbf{s}_t\right)}\left[Q^\pi\left(\mathbf{s}_t,\mathbf{a}_t\right)\right]\end{aligned} \tag{4.2.3}$$

While the approximate advantage function we use in policy gradient is:

$$\hat{A}^\pi\left(\mathbf{s}_t,\mathbf{a}_t\right)=\sum_{t'=t}^{\infty}\gamma^{t'-t}r\left(\mathbf{s}_{t'},\mathbf{a}_{t'}\right)-V_\phi^\pi\left(\mathbf{s}_t\right) \tag{4.2.4}$$

This is unbiased and has a lower variance (but still a higer variance than the actor-critic because of the single sample estimate). But we can make the variance even lower if we substract the Q value.

$$\hat{A}^\pi\left(\mathbf{s}_t,\mathbf{a}_t\right)=\sum_{t'=t}^{\infty}\gamma^{t'-t}r\left(\mathbf{s}_{t'},\mathbf{a}_{t'}\right)-Q_\phi^\pi\left(\mathbf{s}_t,\mathbf{a}_t\right) \tag{4.2.5}$$

This version has a nice property that it goes to zero in expectation if critic is correct. Unfortunately, it does not work if you simply plug it into the policy gradient, since there is an error term you have to compensate for. Now taking that error term into accounts, we have:

$$\nabla_\theta J(\theta) \approx \frac{1}{N}\sum_{i=1}^{N}\sum_{t=1}^{T}\nabla_\theta \log \pi_\theta\left(\mathbf{a}_{i,t} \mid \mathbf{s}_{i,t}\right)\left(\hat{Q}_{i,t}-Q_\phi^\pi\left(\mathbf{s}_{i,t},\mathbf{a}_{i,t}\right)\right)+\frac{1}{N}\sum_{i=1}^{N}\sum_{t=1}^{T}\nabla_\theta E_{\mathbf{a}\sim\pi_\theta\left(\mathbf{a}_t\mid\mathbf{s}_{i,t}\right)}\left[Q_\phi^\pi\left(\mathbf{s}_{i,t},\mathbf{a}_t\right)\right] \tag{4.2.6}$$

The second term represents the gradient of expectation value under the policy of the baseline. Note that this equation is valid even when the baseline depends merely on the state functions, but in that case the second term equals to zero. This kind of trick can provide for a very low variance policy gradient.

We can also take a look at the two different versions of advantage function:

$$\begin{aligned}&\hat{A}_{\mathrm{C}}^\pi\left(\mathbf{s}_t,\mathbf{a}_t\right)=r\left(\mathbf{s}_t,\mathbf{a}_t\right)+\gamma\hat{V}_\phi^\pi\left(\mathbf{s}_{t+1}\right)-\hat{V}_\phi^\pi\left(\mathbf{s}_t\right) && \text{+ lower variance}\\&&& \text{- higher bias if value is wrong (it always is)}\\&\hat{A}_{\mathrm{MC}}^\pi\left(\mathbf{s}_t,\mathbf{a}_t\right)=\textstyle\sum_{t'=t}^{\infty}\gamma^{t'-t}r\left(\mathbf{s}_{t'},\mathbf{a}_{t'}\right)-\hat{V}_\phi^\pi\left(\mathbf{s}_t\right) && \text{+ no bias}\\&&& \text{- higher variance (because single-sample estimate)}\end{aligned} \tag{4.2.7}$$

So can we find something in the middle, combine these two, to control bias/variance tradeoff? The trick here is that instead of using an infinite time horizon, you cut it off before the variance goes too high, given that the variance is generally small in the near future and goes higher in the far future. Here is what an n-step return estimater does:

$$\hat{A}_n^\pi(\mathbf{s}_t, \mathbf{a}_t) = \sum_{t'=t}^{t+n} \gamma^{t'-t} r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) - \hat{V}_\phi^\pi(\mathbf{s}_t) + \gamma^n \hat{V}_\phi^\pi(\mathbf{s}_{t+n}) \tag{4.2.8}$$

Generally the larger n you choose, the bias goes smaller and the variance goes larger. The first and the third term contributes to variance, and the second term contributes to bias. The n here we use in n-step return estimater is fixed. Actually we do not have to choose a single n. Instead, we can take all possible n at one time, which is the generalized advantage estimation.

$$\hat{A}_{\text{GAE}}^\pi(\mathbf{s}_t, \mathbf{a}_t) = \sum_{n=1}^{\infty} w_n \hat{A}_n^\pi(\mathbf{s}_t, \mathbf{a}_t) \tag{4.2.9}$$

The generalized advantage estimation is actually a weighted combination of all n-step returns. In terms of determining the weights, we mostly prefer cutting earlier because it brings less variance. Therefore, a descent choice is to use exponential falloff ($w_n \propto \lambda^{n-1}$).

$$\hat{A}_{\text{GAE}}^\pi(\mathbf{s}_t, \mathbf{a}_t) = \sum_{t'=t}^{\infty} (\gamma\lambda)^{t'-t} \delta_{t'} \quad \delta_{t'} = r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) + \gamma \hat{V}_\phi^\pi(\mathbf{s}_{t'+1}) - \hat{V}_\phi^\pi(\mathbf{s}_{t'}) \tag{4.2.10}$$

4.3 (Optional) Advanced Actor-Critic Methods

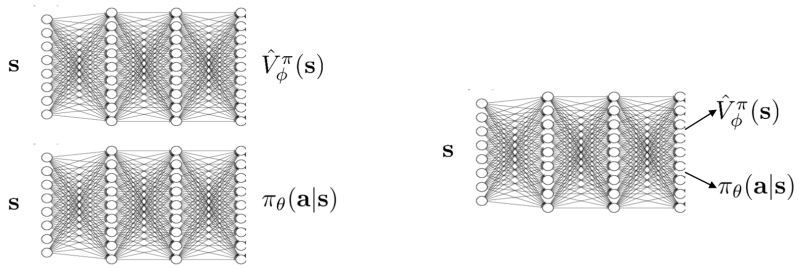
4.3.1 Soft Actor-Critic (SAC)

Add SAC

4.4 Implementations Tips

4.4.1 Architecture Design

There are two options of the neural network architecture design: two network design and shared network design.



(a) two network design (b) shared network design

The two-network design is simple and stable, but lacks efficient feature sharing between the actor and critic. In contrast, the shared network design allows for potential feature sharing efficiency, but it has more hyperparameters and can be more complex and less stable during training.

4.4.2 Parallel Actor-Critic

In a typical online actor-critic algorithm, step2 and step4 work best with a batch (e.g., parallel workers). There are also two types of parallel actor-critic design: **sychronized parallel actor-critic** and **asynchronous parallel actor-critic**.

Sychronized parallel actor-critic: multiple agents learn simultaneously, but they must synchronously update parameters at each step to ensure consistent behavior across all agents. However different agents would use different random seeds so their actions would be a little bit different.

Asynchronous parallel actor-critic: different agents independently update parameters during learning without the need for synchronous updates, leading to improved learning efficiency. However, each agent independently update parameters at their own pace, therefore the parameters update may be inconsistent.

4.4.3 Off-Policy Actor-Critic Algorithm

The problem of sample effeciency gets us thinking about another problem: can we remove the on-policy assumption entirely? A primitive way is to use a replay buffer where we store the transitions we saw in prior time steps. Based on this we have an immature off policy actor-critic algorithm:

Algorithm 19 (Fake) Off-policy actor-critic algorithm

1: **repeat**
2: take action $\mathbf{a} \sim \pi_\theta(\mathbf{a} \mid \mathbf{s})$, get $(\mathbf{s}, \mathbf{a}, \mathbf{s}', r)$, store in $\mathcal{R}$
3: sample a batch $\{\mathbf{s}_i, \mathbf{a}_i, r_i, \mathbf{s}'_i\}$ from buffer $\mathcal{R}$
4: update $\hat{V}_\phi^\pi(\mathbf{s})$ using target $y_i = r_i + \gamma \hat{V}_\phi^\pi(\mathbf{s}'_i)$ for each $\mathbf{s_i}$, $\mathcal{L}(\phi) = \frac{1}{N}\sum_i \left\| \hat{V}_\phi^\pi(\mathbf{s}_i) - y_i \right\|^2$
5: evaluate $\hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i) = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \hat{V}_\phi^\pi(\mathbf{s}'_i) - \hat{V}_\phi^\pi(\mathbf{s}_i)$
6: $\nabla_\theta J(\theta) \approx \sum_i \nabla_\theta \log \pi_\theta(\mathbf{a}_i \mid \mathbf{s}_i) \hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i)$
7: $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$
8: **until** convergence

价值函数 $V^\pi(s)$ 的定义是：从状态 s 开始，**遵循策略** π，能够获得的期望回报。
因此，在计算目标值时，下一状态 s'_i 必须是当前策略 π 执行某个动作后可能到达的状态。
然而，回放池中的 s'_i 是由一个**旧动作**（即旧策略产生的 a_i）导致的 。

策略梯度定理是一个**在线策略（On-Policy）**的定理。它的直观含义是："如果你在状态 s_i 执行了动作 a_i 并获得了好的结果，那么就增加未来在 s_i 状态下执行 a_i 的概率"。
这个"执行了的动作 a_i"必须是**当前策略** π_θ 生成的。
然而，回放池中的动作 a_i 来自于一个**旧版本的策略** 。

There are two fallacies lying in this immature algorithm. The first problem comes from the traget value. When you are loading transitions from the replay buffer, you are actually loading actions from the older version of policy, not the latest one. Therefore $\mathbf{s}'$ comes from the older actions, which is not we actually want.

The second issue comes from the same reason. Because the action $\mathbf{a_i}$ does not come from the latest policy, you cannot calculate policy gradient.

To fix the first problem, the method we take here is to substitute V-function with Q-function. Note the difference between these two functions:

$$\begin{aligned} V^\pi(\mathbf{s}_t) &= \sum_{t'=t}^{T} E_{\pi_\theta}\left[r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) \mid \mathbf{s}_t\right] \\ Q^\pi(\mathbf{s}_t, \mathbf{a}_t) &= \sum_{t'=t}^{T} E_{\pi_\theta}\left[r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) \mid \mathbf{s}_t, \mathbf{a}_t\right] \end{aligned} \tag{4.4.1}$$

Q-function cares about the total reward from taking $\mathbf{a}_t$ in $\mathbf{s}_t$, and then following the policy π_θ. However we do not restrict that action $\mathbf{a}_t$ must come from π_θ. It is a valid function for any action.

So here in the thrid step in Algorithm 6, instead of using $\hat{V}_\phi^\pi(\mathbf{s})$, we update $\hat{Q}_\phi^\pi(\mathbf{s})$ using target y_i. This time we have:

$$\begin{aligned} y_i &= r_i + \gamma \hat{V}_\phi^\pi(\mathbf{s}'_i) \\ &= r_i + \gamma \hat{Q}_\phi^\pi(\mathbf{s}'_i, \mathbf{a}'_i) \end{aligned} \tag{4.4.2}$$

The trick here is to use:

$$V^\pi(\mathbf{s}_t) = \sum_{t'=t}^{T} E_{\pi_\theta}\left[r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) \mid \mathbf{s}_t\right] = E_{\mathbf{a}\sim\pi(\mathbf{a}_t|\mathbf{s}_t)}\left[Q(\mathbf{s}_t, \mathbf{a}_t)\right] \tag{4.4.3}$$

Note that action $\mathbf{a}'_i$ in (4.4.2) does not come from the replay buffer $\mathcal{R}$. Instead, it is the action an agent would have taken following policy π_θ under state $\mathbf{s}'_i$. This procedure does not require state $\mathbf{s}'_i$ to actually happens in a simulator, because all you need is to plug state $\mathbf{s}'_i$ into the neural network and get the output action.

We can use a similar approach to resolve the policy gradient issue. Instead of using $\mathbf{a_i}$ which comes from the replay buffer, we use $\mathbf{a}_i^\pi \sim \pi_\theta(\mathbf{a} \mid \mathbf{s}_i)$ that comes from the latest policy.

So now we have step 5 in Algorithm 6 as:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_i \nabla_\theta \log \pi_\theta(\mathbf{a}_i^\pi \mid \mathbf{s}_i) \hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i^\pi) \tag{4.4.4}$$

修正 1 (针对 Critic)：用 Q-function 替代 V-function

- **解决方案**: 我们不再学习状态价值函数 $V(s)$，而是学习**状态-动作价值函数** $Q(s,a)$ 。
- **逻辑推导**:
  1. $Q^\pi(s,a)$ 的定义是：在状态 s 执行动作 a 之后，再**遵循策略** π 所能获得的期望回报 。
  2. 关键在于，这个定义**并不要求第一个动作** a **必须来自策略** π 。这为我们使用回放池中的旧动作 a_i 提供了理论依据。
  3. 现在，Critic 的更新目标变为 $y_i = r_i + \gamma \hat{Q}^\pi_\phi(s'_i, a'_i)$ 。
  4. **推导中的核心技巧**: 这里的下一个动作 a'_i **不能** 从回放池中获取。为了保证目标值与当前策略 π_θ 保持一致，a'_i **必须是由当前最新的 Actor 策略** π_θ **在下一状态** s'_i **生成的动作**，即 $a'_i \sim \pi_\theta(a|s'_i)$ 。这个动作可以通过将 s'_i 输入当前 Actor 网络来直接获得，无需与环境交互 。

修正 2 (针对 Actor)：使用当前策略重新采样动作

- **解决方案**: 在计算策略梯度时，我们不使用回放池中的旧动作 a_i，而是在当前状态 s_i 下，用**最新的 Actor 策略** π_θ **重新采样一个新动作** a_i^π 。
- **逻辑推导**:
  1. 我们将梯度计算公式修改为：$\nabla_\theta J(\theta) \approx \sum_i \nabla_\theta log\ \pi_\theta(a_i^\pi|s_i)\hat{A}^\pi(s_i, a_i^\pi)$ 。
  2. 因为 a_i^π 是从当前策略 π_θ 中采样的，所以 $\nabla_\theta log\ \pi_\theta(a_i^\pi|s_i)$ 的计算是完全合法的。
  3. 这样，虽然我们使用了离策略的状态 s_i，但梯度计算本身在“动作”层面是符合在线策略要求的，从而修正了谬误。

结合上述修正，我们得到了一个功能完备的离策略 Actor-Critic 算法 (Algorithm 20)。

一个实用性的简化

- **问题**: 在 Actor 的梯度更新中，我们通常使用 Q-function 的值 $\hat{Q}^\pi(s_i, a_i^\pi)$ 直接作为打分，而不是计算复杂的优势函数 $\hat{A}^\pi$ 。

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_i \nabla_\theta log\ \pi_\theta(a_i^\pi|s_i)\hat{Q}^\pi(s_i, a_i^\pi)$$

- **逻辑推导**:
  1. 直接使用 $\hat{Q}$ 会增加梯度估计的方差，因为它缺少了基线（Baseline）$V(s)$ 。
  2. **为什么可以接受高方差?** 因为我们生成新动作 a_i^π **不需要与模拟器交互** 。对于一个给定的状态 s_i，我们可以廉价地、大量地从当前策略 π_θ 中采样动作，并计算多个梯度值的平均。
  3. 通过这种方式，即使单次梯度估计的方差很高，我们也可以通过增加采样数量来轻松降低最终梯度的方差 。

One more thing to say. In practice, we don't actually use the advantage function in policy gradient. We simply use the Q-function here:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_i \nabla_\theta \log \pi_\theta \left(\mathbf{a}_i^\pi \mid \mathbf{s}_i \right) \hat{Q}^\pi \left(\mathbf{s}_i, \mathbf{a}_i^\pi \right) \tag{4.4.5}$$

This would increase the variance because it does not involve a baseline. However, a high variance here is OK since we don't need to interact with the simulators to sample actions. So it's easy to lower the variance by generating more samples of actions $\mathbf{a}_i^\pi$, without generating states $\mathbf{s}_i$

So we have the fixed version of off-policy actor-critic algorithm here:

Algorithm 20 Off-policy actor-critic algorithm

1: **repeat**
2: take action $\mathbf{a} \sim \pi_\theta(\mathbf{a} \mid \mathbf{s})$, get $(\mathbf{s}, \mathbf{a}, \mathbf{s}', r)$, store in $\mathcal{R}$
3: sample a batch $\{\mathbf{s}_i, \mathbf{a}_i, r_i, \mathbf{s}_i'\}$ from buffer $\mathcal{R}$
4: update $\hat{Q}_\phi^\pi(\mathbf{s})$ using target $y_i = r_i + \gamma \hat{Q}_\phi^\pi \left(\mathbf{s}_i', \mathbf{a}_i' \right)$ for each $\mathbf{s_i}$, $\mathcal{L}(\phi) = \frac{1}{N} \sum_i \left\| \hat{Q}_\phi^\pi \left(\mathbf{s}_i \right) - y_i \right\|^2$
5: evaluate $\hat{A}^\pi \left(\mathbf{s}_i, \mathbf{a}_i \right) = r \left(\mathbf{s}_i, \mathbf{a}_i \right) + \gamma \hat{V}_\phi^\pi \left(\mathbf{s}_i' \right) - \hat{V}_\phi^\pi \left(\mathbf{s}_i \right)$
6: $\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_i \nabla_\theta \log \pi_\theta \left(\mathbf{a}_i^\pi \mid \mathbf{s}_i \right) \hat{Q}^\pi \left(\mathbf{s}_i, \mathbf{a}_i^\pi \right)$
7: $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$
8: **until** convergence

Chapter 5 Model-Based Reinforcement Learning

5.1 Introduction to Model-Based reinforcement learning

What we have covered so far can be categorized as "model-free" reinforcement learning. This is because in previous discussions the transition probabilities are unknown and we did not even attempt to learn the transition probabilities. In reinforcement learning, we have the objective of maximizing the expectation of reward along the trajectory given as follows:

$$\pi_\theta(\tau) = p_\theta\left(\mathbf{s}_1, \mathbf{a}_1, \ldots, \mathbf{s}_T, \mathbf{a}_T\right) = p\left(\mathbf{s}_1\right) \prod_{t=1}^{T} \pi_\theta\left(\mathbf{a}_t \mid \mathbf{s}_t\right) p\left(\mathbf{s}_{t+1} \mid \mathbf{s}_t, \mathbf{a}_t\right) \tag{5.1.1}$$

$$\theta^\star = \arg\max_\theta E_{\tau \sim p_\theta(\tau)} \left[\sum_t r\left(\mathbf{s}_t, \mathbf{a}_t\right) \right] \tag{5.1.2}$$

The transition probabilities $p\left(\mathbf{s}_{t+1} \mid \mathbf{s}_t, \mathbf{a}_t\right)$ is not known in all the model-free RL algorithms that we have learned such as Q-learning and policy gradients. But what if we know the transition dynamics? Recall that at the very beginning of the notes we drew an analogy of RL and control theory. In many cases, we do know the system's internal transition. For example, in games (e.g., Atari games, chess, Go), easily modeled systems (e.g., navigating a car), and simulated environments (e.g., simulated robots, video games), the transitions are given to us.

Moreover, it is not uncommon to learn the transition models: in classic robotics, system identification fits unknown parameters of a known model to learn how the system evolves, and one could also imagine a deep learning approach where we could potentially fit a general-purpose model to observed transition data for later use.

It holds true that knowing the transition dynamics does make things easier, and this is what we are going to deal with. In model-based reinforcement learning, we are going to learn the transition dynamics, and then figure out how to choose actions. In this section we will mainly focus on how can we make decisions if we *know* the dynamics, specifically the following two topics:

1. How can we choose actions under perfect knowledge of the system dynamics?
2. Optimal control, trajectory optimization, planning.

So in this chapter we assume the transition model is **known** or **approximated** to further help planning.

5.2 Optimal Control and Planning

5.2.1 Optimal Control

Optimal control is a task that we come across when we are well aware of the transition probabilities and we try to learn how to control the system optimally. In optimal control, there are two different categories of controller design: the first one is open-loop control, where we do not have any state feedbacks, and we roll out a sequence of actions based on the current state that we observe. The second one is called closed-loop control, where we determine the action at each time step based on the current state, and how we determine the action to apply is based on state feedbacks. In terms of transition model, there are also two categories depending on whether the

model is deterministic or stochastic. While in reinforcement learning, we generally deal with stochastic transition models and close loop control.

In a open loop controller, if we have a deterministic transition in our system such that $\mathbf{s}_{t+1} = f(\mathbf{s}_t, \mathbf{a}_t)$, then our action sequence should be determined by choosing those that can return the maximum rewards:

确定性开环控制

$$\mathbf{a}_1, \dots, \mathbf{a}_T = \arg\max_{\mathbf{a}_1,\dots,\mathbf{a}_T} \sum_{t=1}^{T} r(\mathbf{s}_t, \mathbf{a}_t) \quad \text{s.t. } \mathbf{s}_{t+1} = f(\mathbf{s}_t, \mathbf{a}_t) \tag{5.2.1}$$

In stochastic scenarios, the transition function is a probabilistic distribution, where we have $p(\mathbf{s}_{t+1} \mid \mathbf{s}_t, \mathbf{a}_t)$, and the action sequence should be chosen based on expectation of the rewards:

随机性开环控制

$$\begin{aligned} &p_\theta(\mathbf{s}_1, \dots, \mathbf{s}_T \mid \mathbf{a}_1, \dots, \mathbf{a}_T) = p(\mathbf{s}_1) \prod_{t=1}^{T} p(\mathbf{s}_{t+1} \mid \mathbf{s}_t, \mathbf{a}_t) \\ &\mathbf{a}_1, \dots, \mathbf{a}_T = \arg\max_{\mathbf{a}_1,\dots,\mathbf{a}_T} E\left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \mid \mathbf{a}_1, \dots, \mathbf{a}_T\right] \end{aligned} \tag{5.2.2}$$

Note that this policy might be suboptimal, since future states may reveal more information helpful to decision making. While in open loop stochastic cases we roll out all actions to apply only based on the initial state marginal and we do not consider any state-feedback.

In a closed-loop controller, however, we keep interacting with the world, so we need a policy function that can tell us the action to apply if we input the current state: $\pi(\mathbf{a}_t \mid \mathbf{s}_t)$, which we call a state-feedback. We choose our policy function as follows:

随机性闭环控制

$$\begin{aligned} &p(\mathbf{s}_1, \mathbf{a}_1, \dots, \mathbf{s}_T, \mathbf{a}_T) = p(\mathbf{s}_1) \prod_{t=1}^{T} \pi(\mathbf{a}_t \mid \mathbf{s}_t)\, p(\mathbf{s}_{t+1} \mid \mathbf{s}_t, \mathbf{a}_t) \\ &\pi = \arg\max_\pi E_{\tau \sim p(\tau)}\left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t)\right] \end{aligned} \tag{5.2.3}$$

Note that the form of policy π may vary, including nerual nets, or simply takes a time-varying linear form: $\mathbf{K}_t \mathbf{s}_t + \mathbf{k}_t$.

5.2.2 Open-Loop Planning

We'll start from open-loop planning first. It has the minimal assumption about the dynamics: it does not care about whether the transition model is continuous or discrete, deterministic or stochastic, or whether it is differentiable.

Recall the objective of stochastic open-loop planning, we roll out a sequence of actions by doing arg max on the sum of rewards:

$$\mathbf{a}_1, \dots, \mathbf{a}_T = \arg\max_{\mathbf{a}_1,\dots,\mathbf{a}_T} \sum_{t=1}^{T} r(\mathbf{s}_t, \mathbf{a}_t) \text{ s.t. } \mathbf{a}_{t+1} = f(\mathbf{s}_t, \mathbf{a}_t) \tag{5.2.4}$$

We can abstract away the control and planning part:

$$\mathbf{a}_1, \dots, \mathbf{a}_T = \arg\max_{\mathbf{a}_1,\dots,\mathbf{a}_T} J(\mathbf{a}_1, \dots, \mathbf{a}_T) \tag{5.2.5}$$

Or compactly, since we do not care about whether the action is sequential or discrete, we can say:

$$\mathbf{A} = \arg\max_{\mathbf{A}} J(\mathbf{A}) \tag{5.2.6}$$

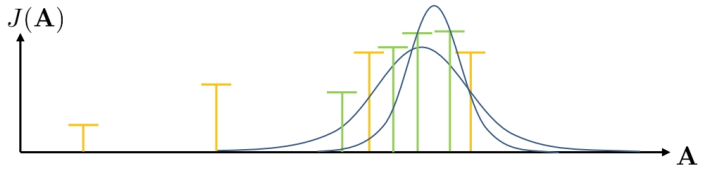
The simplest method is guess and check, or you can call it random shooting method:

1. pick $\mathbf{A}_1, \dots, \mathbf{A}_N$ from some distribution (e.g., uniform) 纯随机搜索
2. choose $\mathbf{A}_i$ based on $\arg\max_i J(\mathbf{A}_i)$

Random shooting method has the advantage that it is super simple to implement, though it could be highly inefficient in that we are not improving what we sample, so we might get stuck in some mediocre action sequence.

Cross-entropy method

We can dramatically improve this random shooting method by using cross-entropy method (CEM) Instead of randomly selecting samples, in CEM we first randomly initialize a set of samples and then evaluate the objective function for each sample. Next, we use these objective function values to update the sampling distribution, so that the samples are more likely to fall in regions with higher objective function values. This image illustrates the process of iteratively sampling from regions with higher objective function values.



Algorithm 21 Cross-entropy method

1: **repeat**
2: sample $\mathbf{A}_1, \ldots, \mathbf{A}_N$ from $p(\mathbf{A})$
3: evaluate $J(\mathbf{A}_1), \ldots, J(\mathbf{A}_N)$
4: pick the elites $\mathbf{A}_{i_1}, \ldots, \mathbf{A}_{i_M}$ with the highest value, where $M < N$
5: refit $p(\mathbf{A})$ to the elites $\mathbf{A}_{i_1}, \ldots, \mathbf{A}_{i_M}$
6: **until** Satisfactory result

评估: 对每个序列，使用模型预测其累积奖励 $J(A)$ 。

筛选: 选出奖励最高的 M 个序列，称之为"精英样本" (elites) 。

更新: 使用这 M 个精英样本，重新拟合采样分布 $p(A)$（例如，计算这些样本的均值和方差，作为新的高斯分布参数）。

When we "fit" a distribution, what we are actually doing is to iteratively search for the best parameters to find a distribution from which we can sample to give us the best cost-to-go value. Cross-entropy method is also very fast if parallelized, and also extremely simple to implement. However it has a harsh limit on dimensionality. And it only works for open-loop scenarios.

Monte Carlo tree search (MCTS)

In discrete scenarios, we use another method called Monte Carlo tree search, which is very popular in planning in stochastic games.

The gist of this method is that in discrete action space, we are essentially expanding out a tree, but we can not fullly explore the tree due to the exponential computational cost. One way to save the computational cost is to partially expand the tree (for example explore to a fixed depth in each branch) and then use a given policy (e.g., random distribution) to simulate a trajectory from the last expanded node.

Then the question would be: how do we decide on where to search first?

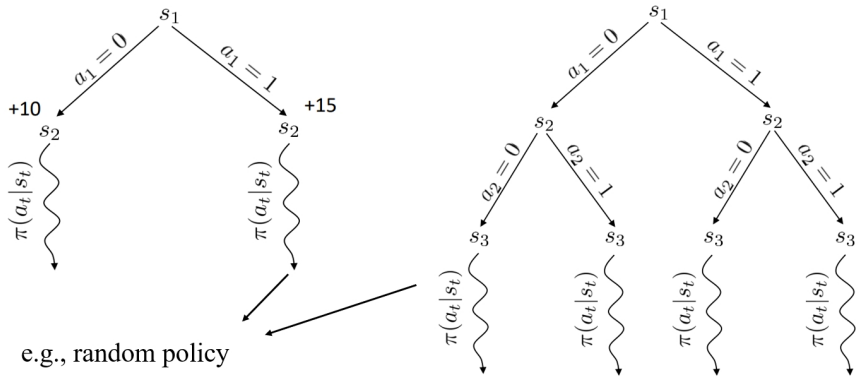


Figure 5.1: Monte Carlo tree search

Suppose we have two actions to take, $a = 0$ and $a = 1$. After taking either action, the agent follows a fixed policy to complete the episode, and the rewards obtained are 10 and 15, respectively.Note that the values of 10 and 15 are not the true values of the rewards, since this is a stochastic process and we have only executed it once.

Our intuition tells us that we should further exploring the action that gave us a higher reward, which is $a = 1$. Besides, if there is a third action that we have not yet explored, then we should also explore this unknown path. That is to say, we should choose nodes with best reward, but also prefer rarely visited nodes. This intuition actually gives the skectch of generic MCTS:

Algorithm 22 Generic MCTS algorithm

1: **repeat**
2: find a leaf s_l using TreePolicy (s_1)
3: evaluate the leaf using DefaultPolicy (s_l)
4: update all values in tree between s_1 and s_l
5: refit $p(\mathbf{A})$ to the elites $\mathbf{A}_{i_1}, \ldots, \mathbf{A}_{i_M}$
6: **until** take best action from s_1

推导和细节: 这是蒙特卡洛树搜索 (MCTS) 中用于选择下一步要探索哪个子节点的关键公式，它巧妙地平衡了“利用”和“探索”。

1. **第一项 (利用, Exploitation)**: $\frac{Q(s_t)}{N(s_t)}$。
   - $Q(s_t)$ 是从节点 s_t 出发获得的总奖励，$N(s_t)$ 是节点 s_t 被访问的总次数。
   - 这一项代表了从该节点出发的**平均奖励**。分数高的节点意味着它在历史探索中表现更好，我们应该多加“利用”。
2. **第二项 (探索, Exploration)**: $2C\sqrt{\frac{2\ln N(s_{t-1})}{N(s_t)}}$。
   - $N(s_{t-1})$ 是父节点的访问次数。
   - 这一项是“探索奖励”。如果一个节点 s_t 的访问次数 $N(s_t)$ 相对于其父节点 $N(s_{t-1})$ 来说很小，那么这一项的值就会很大。
   - 这鼓励算法去探索那些**访问次数较少**的节点，因为它们可能隐藏着未被发现的高奖励路径。
   - C 是一个超参数，用于控制利用和探索之间的权衡。

A frequently used TreePolicy is called UCT TreePolicy: if s_t is not fully expanded, choose the remaining new action a_t; else, choose child branch with best score $S(s_{t+1})$, where the score is calculated by:

$$\text{Score}(s_t) = \frac{Q(s_t)}{N(s_t)} + 2C\sqrt{\frac{2\ln N(s_{t-1})}{N(s_t)}} \tag{5.2.7}$$

Here Q for reward, and N for total steps of a chosen trajectory.

5.2.3 Trajectory Optimization with Derivatives

We are going to use the set of notation more commonly used in control therory for the following discussions. (using $\mathbf{u}_t$ for action and $\mathbf{x}_t$ for state, minimizing cost instead of maximizing reward)

$$\min_{\mathbf{u}_1,\ldots,\mathbf{u}_T} \sum_{t=1}^{T} c(\mathbf{x}_t, \mathbf{u}_t) \text{ s.t. } \mathbf{x}_t = f(\mathbf{x}_{t-1}, \mathbf{u}_{t-1}) \tag{5.2.8}$$

if we plug in the transition constraint, we have:

$$\min_{\mathbf{u}_1,\ldots,\mathbf{u}_T} c(\mathbf{x}_1, \mathbf{u}_1) + c(f(\mathbf{x}_1, \mathbf{u}_1), \mathbf{u}_2) + \cdots + c(f(f(\ldots)\ldots), \mathbf{u}_T) \tag{5.2.9}$$

5.3 Model-Based Reinforcement Learning

5.3.1 Basics

In this section, we are going to cover a rather simpler case of model-based RL. Specifically, we are going to talk about a technique to learn a model of the system first, and then use the optimal control technique we covered to improve the model. Furthermore, we will learn to address uncertainty in the model such as model mismatch and imperfection.

Why do we learn the model? We can learn the model so that we know $f(s_t, a_t) = s_{t+1}$ (in deterministic case) or $p(s_{t+1}|s_t, a_t)$ (in stochastic case), we could use the tools from optimal control to maximize our rewards.

Algorithm 23 Model-based Reinforcement Learning Version 0.5

Require: Some base policy for data collection π_0
1: Run base policy $\pi(a_t|s_t)$ (e.g. random policy) to collect $\mathcal{D} = \{(s, a, s')_i\}$ (supervised learning)
2: Learn dynamics model $f(s, a)$ to minimize $\sum_i ||f(s_i, a_i) - s_i'||^2$
3: Plan through $f(s, a)$ to choose actions

模型 f(s,a)只在初始策略 π0 访问过的状态-动作空间上是准确的

Our first attempt is naive: learn $f(s_t, a_t)$ from data, and then plan through it. We call this approach model-based RL version 0.5, or vanilla model-based RL, as shown in Alg. 23. This is essentially what people do in system identification, which is a technique used in classic robotics, and it is effective when we can hand-engineer a dynamics representation using our knowledge of physics, and fit just a few parameters. However, it does not work in general cases because of distribution mismatch, i.e., $p_{\pi_0}(s_t) \neq p_{\pi_f}(s_t)$. Furthermore, the distribution mismatch exacerbates as we use more expressive model classes (e.g., neural networks), since more complicated models would fit more tightly to the demonstration data (which might be inaccurate).

How to make $p_{\pi_0}(s_t) = p_{\pi_f}(s_t)$? We need to collect data from $p_{\pi_f}(s_t)$. We can do this by keep updating the dataset by running the current model, and then update the model accordingly. Take a look at the updated model-based RL algorithm in Alg. 24.

Algorithm 24 Model-based Reinforcement Learning Version 1.0

Require: Some base policy for data collection π_0
1: Run base policy $\pi(a_t|s_t)$ (e.g. random policy) to collect $\mathcal{D} = \{(s, a, s')_i\}$
2: **while** True **do**
3: Learn dynamics model $f(s, a)$ to minimize $\sum_i ||f(s_i, a_i) - s'_i||^2$
4: Plan through $f(s, a)$ to choose actions
5: Execute those actions and add the resulting data $\{(s, a, s')_j\}$ to $\mathcal{D}$

Version 1.0 addresses the model mismatch issue and drives the current model as close as possible to the true dynamics model. However, we are still blindly following a trajectory in step 5 of Alg. 24, and if we made a mistake, we would follow the wrong step which makes the mistake exacerbate. Therefore, we need to somehow adjust our plan as time goes on. One way to do this is to borrow some ideas from modern control theory: Model Predictive Control (MPC).

In MPC, we are given the system's dynamics model, and we are trying to design an adaptive controller by solving a finite time constrained optimal control problem at each time step, and take only the first action in the generated sequence of actions. Then we replan based on the new state. We essentially aims to take one action in the planned sequence and only observe one new state, and then append the observed transition to our dataset $\mathcal{D}$. The improvement is shown in Alg. 25.

Algorithm 25 Model-based Reinforcement Learning Version 1.5

Require: Some base policy for data collection π_0, hyperparameter N
1: Run base policy $\pi(a_t|s_t)$ (e.g. random policy) to collect $\mathcal{D} = \{(s, a, s')_i\}$
2: **while** True **do**
3: Learn dynamics model $f(s, a)$ to minimize $\sum_i ||f(s_i, a_i) - s'_i||^2$
4: **for** N times **do**
5: Plan through $f(s, a)$ to choose actions
6: Execute the first planned action, observe resulting state s' (MPC)
7: Append (s, a, s') to dataset $\mathcal{D}$

The inner loop in Alg. 25 refers to replanning in MPC, which is solving for an optimization problem at each time step after we take the first action planned. The outer loop means that we are periodically retraining the model in order to make it closer to the true underlying dynamics. Intuitively, the more frequently the agent replans, the less perfect each individual plan needs to be, because since we are frequently replanning, we are able to correct our mistakes made in previous plans more easily. Consequently, one is able to correct the plans as one increases the replanning frequency. Therefore, if we are frequently replanning, we could use shorter horizons.

5.3.2 Performance Gaps in Model-based RL

Sometimes model-based RL performs worse than model-free RL. The problem is from step 5 of Alg. 25. In this step, we plan through the model to choose actions, which means we are solving an optimization problem based on the data we collect. One could imagine that if we overfit the data, the agent might have some wrong belief about the model, thus generating wrong actions, as is illustrated in Fig. 5.2.

Figure 5.2: False belief about the model from overfitting

Will model-based learning run into the overfitting problem? The answer is yes, because the model does not see the true data distribution. In step 5 of Alg. 25, when we choose actions, we only take actions for which we think we will get high reward **in expectation** (with respect to uncertain dynamics). In the model-based setting, the model will quickly adapts to the distribution of these actions and converge, which avoids the model from exploring enough and results in bad actions planned.

过拟合产生的错误尖峰，过早地终止了充分探索，还没有充分了解dynamic model

Therefore, we need to explore to get more representative and holistic data of the model, thus preventing overfitting and false belief. Note that the expected value of the reward is not the same as optimistic or pessimistic, but it is often a good start.

所以需要构建对不确定性敏感的模型

5.3.3 Uncertainty-Aware Models

One way to deal with the problem of bad actions planned is to construct an uncertainty-aware model.

The uncertainty-aware model provides a way to quantitatively estimate the uncertainty in the model, so that we can assess the accuracy of the model and the planned actions. When we take uncertainty into account, the agent will be less likely to take actions that are highly uncertain, and will instead prefer actions that are both highly certain and have high reward. In the context of neural networks, this means that the agent will be less likely to take actions that are the result of overfitting, even if those actions have high predicted reward, because they are highly uncertain.

Remark. Why can uncertainty-aware models help alleviate the above problem?

Add more details

The first idea is to use entropy of output distribution, and as we know, higher entropy means higher uncertainty. We can estimate the entropy of $p(s_{t+1}|s_t, a_t)$. However, this is not enough because even when the model is wrong (e.g., an over-fitted model), we might still have low variance, thus low entropy, as long as the data points all satisfy the over-fitted model.

The reason why entropy of the output distribution alone is not expressive enough is that there are two types of uncertainty:

偶然不确定性，物理世界的内在随机性/数据的噪声

- Aleatoric (statistical) uncertainty, where *the data itself is noisy* .
- Epistemic (model) uncertainty, where *the model is certain about data, but we are not certain about model.*

认知不确定性，源于模型自身的不完美，在数据稀疏的区域，模型对其预测的"自信心"较低

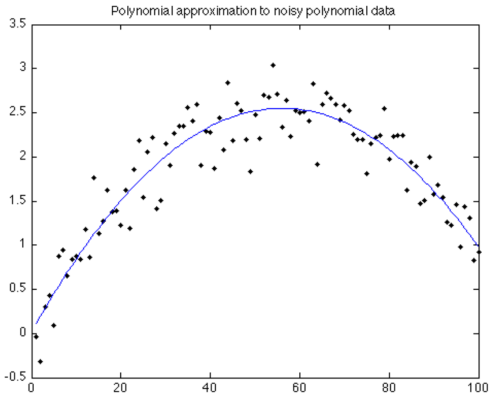


Figure 5.3: Aleatoric uncertainty

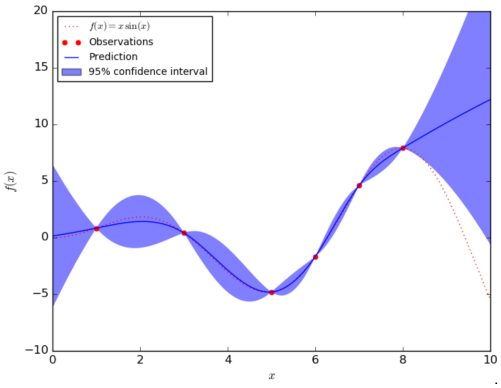


Figure 5.4: Epistemic uncertainty

The second idea is to estimate the epistemic (model) uncertainty, where we essentially estimate how uncertainty we are about the model.

Usually, we use maximum likelihood estimation, where

$$\arg\max_\theta \log p(\theta|\mathcal{D}) = \arg\max_\theta \log p(\mathcal{D}|\theta) \tag{5.3.1}$$

This is because when we are searching for the model parameters θ that are "most likely", there are actually two ways to interpret "most likely" in this context: (1) Given the model parameters θ, the probability of generating

后验概率 (Posterior Probability)：在观测到数据集 D 之后，模型参数为 θ 的概率是多少。

似然函数 (Likelihood)：在给定模型参数 θ 的情况下，观测到我们手中的数据集 D 的概率是多少。

假设我们对模型参数 θ 的先验信念是均匀分布的（即没有任何偏好），那么最大化后验概率（MAP）就等价于最大化似然函数（MLE）。

the dataset $\mathcal{D}$ is maximized. (2) Given the dataset $\mathcal{D}$, the probability that this particular dataset was generated by the model with parameters θ is maximized. And these two interpretations essentially gives you the same parameters when performing arg max.

If we estimate the full distribution of data $p(\theta|\mathcal{D})$ instead of argmax, the entropy of the distribution gives us the model uncertainty from the data. In practice we can predict using

$$\int p(s_{t+1}|s_t, a_t, \theta)p(\theta|\mathcal{D})d\theta. \tag{5.3.2}$$

Bayesian Neural Networks

Bayesian Neural Networks

Bootstrap Ensembles

训练 N 个独立的模型，组成一个“集成”，对于一个新的输入 (s,a)，分别送入 N 个模型得到 N 个不同的预测，之间的方差或分歧，就反映了模型在该区域的认知不确定性，在规划时可以惩罚那些导致高不确定性的动作

To learn the posterior distribution, we can also train bootstrap ensembles, where we use multiple networks to learn the same distribution. Formally, say we have N networks, each with a parameter θ_i to learn $p(s_{t+1}|s_t, a_t)$, we can then estimate the posterior by:

$$p(\theta|\mathcal{D}) \sim \frac{1}{N}\sum_i \delta(\theta_i) \tag{5.3.3}$$

where $\delta(\cdot)$ is the Dirac-delta function. To train it, we need to generate independent datasets to get independent models. One way to do this is to chop the original dataset into multiple subsets. Another way is to train θ_i on $\mathcal{D}_i$ sampled with replacement from $\mathcal{D}$. In reality, resampling with replacement is usually not necessary, because SGD and random initialization usually makes the model sufficiently independent.

With this ensemble of networks, we choose actions a little differently. Before, we choose actions by $J(a_1, \ldots, a_H) = \sum_{t=1}^{H} r(s_t, a_t)$, where $s_{t+1} = f(s_t, a_t)$, and now we average over the ensemble by $J(a_1, \ldots, a_H) = \frac{1}{N}\sum_{i=1}^{N}\sum_{t=1}^{H} r(s_{t,i}, a_{t,i})$, where $s_{t+1,i} = f(s_{t,i}, a_{t,i})$

In general, for candidate action sequence $a_1, \ldots, a_H$, we first sample $\theta \sim p(\theta|\mathcal{D})$, then at each time step t, we sample $s_{t+1} \sim p(s_{t+1}|s_t, a_t, \theta)$, then we calculate the reward $R = \sum_t r(s_t, a_t)$, and we repeat the aforementioned steps and accumulate the average reward.

5.3.4 Latent Space Model

In many cases, we are given very complex observations of states which we do not have full access to, such as pixel-based images. To learn the dynamics using observations, we need to learn from the latent space and infer the states from observations.

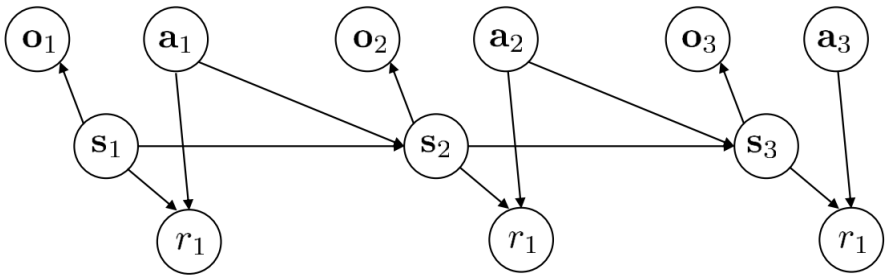


Figure 5.5: Latent space model

From Fig. 5.5, we can see that we need to learn the following models:

- $p(o_t|s_t)$, the observation model
- $p(s_{t+1}|s_t, a_t)$, the dynamics model
- $p(r_t|s_t, a_t)$, the reward model

Recall that in high level, model-based RL algorithms are basically doing a maximum likelihood estimation in training given fully observed states:

$$\max_{\phi} \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \log p_{\phi}(s_{t+1,i}|s_{t,i}, a_{t,i}) \tag{5.3.4}$$

With latent models, then, we are not sure about the actual state, so we take the expected value:

$$\max_{\phi} \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \mathbb{E}\left[\log p_{\phi}(s_{t+1,i}|s_{t,i}, a_{t,i}) + \log p_{\phi}(o_{t,i}|s_{t,i})\right] \tag{5.3.5}$$

where the expectation is with respect to the distribution of $(s_t, s_{t+1}) \sim p(s_t, s_{t+1}|o_{1:T}, a_{1:T})$.

However, the posterior distribution $p(s_t, s_{t+1}|o_{1:T}, a_{1:T})$ is usually intractable if we have very complex dynamics. As a result, we could instead try to learn an approximate posterior $q_{\psi}(s_t|o_{1:t}, a_{1;t})$, which is called an **encoder**. We could also learn $q_{\psi}(s_t, s_{t+1}|o_{1:t}, a_{1;t})$ and $q_{\psi}(s_t|o_t)$. Learning the distribution $q_{\psi}(s_t|o_t)$ is crude, but it is simple to implement. Let us focus on $q_{\psi}(s_t|o_t)$ for now, and then the expectation becomes:

$$\max_{\phi} \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \mathbb{E}\left[\log p_{\phi}(s_{t+1,i}|s_{t,i}, a_{t,i}) + \log p_{\phi}(o_{t,i}|s_{t,i})\right]$$

where the expectation is with respect to $s_t \sim q_{\psi}(s_t|o_t)$, $s_{t+1} \sim q_{\psi}(s_{t+1}|o_{t+1})$.

For now, let us further assume that $q(s_t|o_t)$ is deterministic (because the stochastic case requires variational inference, which will be covered in-depth in a later chapter). In deterministic case, we are training a neural net $g_{\psi}(o_t) = s_t$ using a Dirac-delta function such that $q_{\psi}(s_t|o_t) = \delta(s_t = g_{\psi}(o_t))$. Then the expectation can be simplified as

$$\max_{\phi} \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \log p_{\phi}(g_{\psi}(o_{t+1,i})|g_{\psi}(o_{t,i}), a_{t,i}) + \log p_{\phi}(o_{t,i}|g_{\psi}(o_{t,i})). \tag{5.3.6}$$

Now everything is differentiable, we can train using backpropagation.

Thus, we can slightly modify Alg. 25 so that we can deal with observations and latent space. We show the sketch of this slightly modified algorithm in Alg. 26. In line 3, we are respectively learning the dynamics, reward model, observation model, and encoder.

Algorithm 26 Model-based Reinforcement Learning with Latent States

Require: Some base policy for data collection π_0, hyperparameter N
1: Run base policy $\pi(a_t|s_t)$ (e.g. random policy) to collect $\mathcal{D} = \{(s, a, s')_i\}$
2: **while** True **do**
3: Learn dynamics model $p_{\phi}(s_{t+1}|s_t, a_t), p_{\phi}(r_t|s_t), p(o_t|s_t), g_{\psi}(o_t)$
4: **for** N times **do**
5: Plan through $f(s, a)$ to choose actions
6: Execute the first planned action, observe resulting state o' (MPC)
7: Append (o, a, o') to dataset $\mathcal{D}$

5.3.5 Further Reading

For more details on model-based RL, refer to Nagabandi et al. (2018), Chua et al. (2018), Feinberg et al. (2018) and Buckman et al. (2018).

For more details for latent space models, refer to Watter et al. (2015) and Zhang et al. (2019).

5.4 Model-Based Policy Learning

So far we have covered the basics of model-based RL that we first learn a model and use a model for control. We have seen that this approach does not work well in general because of the effect of distributional shift in model-based RL. We have also seen the method to quantify uncertainty in our model in order to alleviate this

issue. The methods we covered so far do not involve learning policies. In this chapter, we will cover model-based reinforcement learning of policies. Specifically, we will learn global policies and local policies, and combine local policies into global policies using guided policy search and policy distillation. We shall understand how and why we should use models to learn policies, global and local policy learning, and how local policies can be merged via supervised learning into a global policy.

We have seen the difference between a closed-loop and open-loop controller. We also discussed why an open-loop controller is suboptimal because we are rolling out a whole sequence of actions solely based on one state observation. Therefore, it would be more ideal if we could design a closed-loop controller where state feedbacks can help us correct the mistakes we make. Recall in a stochastic environment, we are optimizing over the policy as follows:

$$p(s_1, a_1, \ldots, s_T, a_T) = p(s_1) \prod_{t=1}^{T} \pi(a_t|s_t) p(s_{t+1}|s_t, a_t)$$

$$\pi = \arg\max_{\pi} \mathbb{E}_{\tau \sim p(\tau)} \left[\sum_t r(s_t, a_t) \right]$$

and π could take several forms: π can be a neural net, which we call a **global** policy, and it can also be a time-varying linear controller $K_t s_t + k_t$ as we saw in LQR, which we call a **local** policy.

5.5 Back-propagate into the Policy

Let us start with a simple solution for model-based policy learning. Ideally, we could build a computational graph in Tensorflow, and calculate the partial derivatives step by step so that we can backpropage into policy and optimize the policy, illustrated in Fig. 5.6.

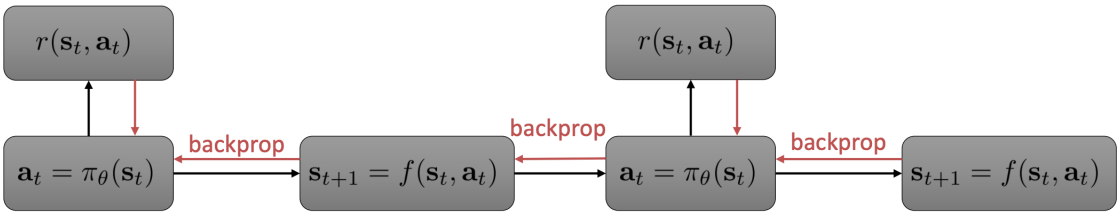


Figure 5.6: Back-propagate into policies

Then we can modify our model-based policy-free RL algorithm to accomodate this new policy learning process in Alg. 27.

Algorithm 27 Model-based Reinforcement Learning Version 1.5

Require: Some base policy for data collection π_0

1: Run base policy $\pi_0(a_t|s_t)$ (e.g. random policy) to collect $\mathcal{D} = \{(s, a, s')_i\}$
2: **while** True **do**
3: Learn dynamics model $f(s, a)$ to minimize $\sum_i ||f(s_i, a_i) - s'_i||^2$
4: Backpropagate through $f(s, a)$ into the policy to optimize $\pi_\theta(a_t|s_t)$
5: Run $\pi_\theta(a_t|s_t)$, appending the visited tuples (s, a, s') to $\mathcal{D}$.

5.5.1 Vanishing and Exploding Gradients

One problem with Alg. 27, or general gradient-based optimization is that as we progress into the time steps, we might encounter vanishing or exploding gradients. Because as we apply chain rule, the gradients get multiplied by each other, so we the product may get extremely big (exploding) or extremely small (vanishing), making optimization a lot harder. Furthermore, we have similar parameter sensitivity problems as shooting methods, but we no longer have convenient second order LQR-like method, because the policy function is extremely complicated and policy parameters couple all the time steps, so no dynamic programming.

So what can we do about it? First, we can use model-free RL algorithms with synthetic samples generated by the model. Essentially, we are using models to accelerate model-free RL. Second, we can use simpler policies than neural nets such as LQR, and train local policies to solve simple tasks, and then combine them into global policies via supervised learning.

5.6 Model-free Optimization with a Model

Model-free Optimization with a Model is yet to be written.

Recall the equation from policy gradients:

$$\nabla_\theta J(\theta) \simeq \frac{1}{N}\sum_{i=1}^{N}\sum_{t=1}^{T}\nabla_\theta \log \pi_\theta(a_{i,t}|s_{i,t})\hat{Q}^\pi_{i,t}$$

Note that we are not doing any backprop through time in policy gradient because we are calculating the gradient with respect to an expectation, so we can just take the derivative of the probability of the samples instead of the actual dynamics function.

Then we look at the regular backprop (pathwise) gradient, we see a more chain rule-like gradient:

$$\nabla_\theta J(\theta) = \sum_{t=1}^{T}\frac{dr_t}{ds_t}\prod_{t'=2}^{t}\frac{ds_{t'}}{da_{t'-1}}\frac{da_{t'-1}}{ds_{t'-1}}$$

The two gradients are different, because the policy gradient is for stochastic systems while the backprop policy is for deterministic systems. But using variational inference, we can prove that they are calculating the same gradient differently, thus having different tradeoffs. We will talk about variational inference more in-depth in the next chapter.

Actually, given more samples to reduce variance, policy gradient is more stable because it does not require multiplying many Jacobians. However, if our models are inaccurate, the samples we use from the wrong model will be incorrect, and the mistakes are likely to exacerbate as time goes on. So it would be nice to use such model-free optimizer and keep the rolled out samples' trajectory short. This is essentially what Dyna algorithm does.

5.6.1 Dyna

Dyna is an online Q-learning algorithm that performs model-free RL with a model.

Algorithm 28 Dyna

Require: Some exploration policy for data collection π_0

1: Given state s, pick action a using exploration policy
2: Observe s' and r, to get transition (s, a, s', r)
3: Update model $\hat{p}(s'|s, a)$ and $\hat{r}(s, a)$ using (s, a, s')
4: Q-update: $Q(s, a) \leftarrow Q(s, a) + \alpha\mathbb{E}_{s',r}\left[r + \max_{a'} Q(s', a') - Q(s, a)\right]$
5: **for** K times **do**
6: Sample $(s, a) \sim \mathcal{B}$ from buffer of past states and actions
7: Q-update: $Q(s, a) \leftarrow Q(s, a) + \alpha\mathbb{E}_{s',r}\left[r + \max_{a'} Q(s', a') - Q(s, a)\right]$

In step 3 of Alg. 28, we are updating the model and reward function using the observed transition. Then in step 6, we will sample some old state and action pairs and apply the model onto the sampled pair, so the s' in step 7 are synthetic next states. Intuitively, as the models get better, the expectation estimate in step 7 also gets more accurate. This algorithm seems arbitrary in many aspects, but the gist is to keep improving models and use models to improve Q-function estimation by taking expectations.

We can also generalize Dyna to see how this kind of general Dyna-style model-based RL algorithms work. The generalized algorithm is shown in Alg. 29.

As shown in Fig. 5.7, we choose some states (orange dots) from the buffer, simulate the next states using the learned model, and then train model-free RL with synthetic data (s, a, s', r) where s is from the experience

Algorithm 29 General Dyna

Require: Some exploration policy for data collection π_0
1: Collect some data, consisting of transitions (s, a, s', r)
2: Learn model $\hat{p}(s'|s, a)$ (and optionally, $\hat{r}(s, a)$)
3: **for** K times **do**
4: Sample $s \sim \mathcal{B}$ from buffer
5: Choose action a (from $\mathcal{B}$, from π, or random)
6: Simulate $s' \sim \hat{p}(s'|s, a)$ (and $r = \hat{r}(s, a)$)
7: Train on (s, a, s', r) with model-free RL
8: (optional) take N more model-based steps

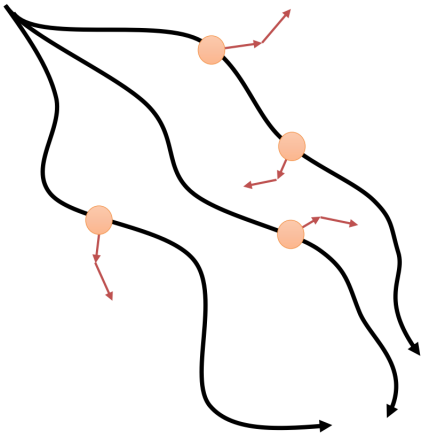


Figure 5.7: General Dyna training

buffer, s' is from the learned model. One could also take more than one step if one believes that the model is good enough for more steps.

This algorithm only requires very short (as few as one step) rollouts from model, so the mistakes will not exacerbate and accumulate much. Moreover, we explore well with a lot of samples because we still see diverse states.

5.7 Local and Global Models

Recall that in LQR, we can turn a constrianed optimization problem into an unconstrained problem:

$$\min_{u_1,\dots,u_T} c(x_1, u_1) + c(f(x_1, u_1), u_2) + \cdots + c(f(f(\dots)\dots), u_T)$$

Backpropagation is indeed a possible solution to solve this optimization problem, and we need $\frac{df}{dx_t}$, $\frac{df}{du_t}$, $\frac{dc}{dx_t}$, $\frac{dc}{du_t}$

5.7.1 Local Models

Since LQR gives us a state-feedback controller for a linear system, we can keep linearizing the system and iteratively apply LQR to generate local models. We fit $\frac{df}{dx_t}$, $\frac{df}{du_t}$ around the current trajectory or policy. Say the model is a Gaussian $p(x_{t+1}|x_t, u_t) = \mathcal{N}(f(x_t, u_t), \Sigma)$, then we can approximate the model as a linear function $f(x_t, u_t) \simeq A_t x_t + B_t u_t$, and we can use $\frac{df}{dx_t}$ as A_t, and $\frac{df}{du_t}$ as B_t.

Iterative LQR produces $\hat{x}_t, \hat{u}_t, K_t, k_t$, where $u_t = K_t(x_t - \hat{x}_t) + k_t + \hat{u}_t$. We can execute the controller using a Gaussian $p(u_t|x_t) = \mathcal{N}(K_t(x_t - \hat{x}_t) + k_t + \hat{u}_t, \Sigma_t)$ because we can add noise to the iLQR controller so that all samples do not look the same. Practically, we can set $\Sigma_t = Q_{u_t,u_t}^{-1}$. We can fit the model $p(s_{t+1}|s_t, a_t)$ using Bayesian linear regression, and use the global model as prior.

We also need to stay close to old controller if we go too far. If trajectory distribution is close, then dynamics will be close too. Close here means the KL-divergence is small $D_{KL}(p(\tau)||p(\bar{\tau})) \leq \epsilon$.

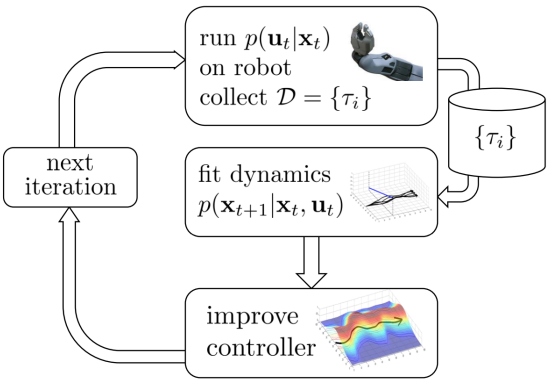


Figure 5.8: Local models fitting

Algorithm 30 Guided Policy Search

1: **while** True **do**
2: Optimize each local policy $\pi_{LQR,i}(u_t|x_t)$ on initial state $x_{0,i}$ with respect to $\tilde{c}_{k,i}(x_t, u_t)$
3: Use samples from the previous step to train $\pi_\theta(u_t|x_t)$ to mimic each $\pi_{LQR,i}(u_t|x_t)$
4: Update cost function $\tilde{c}_{k+1,i}(x_t, u_t) = c(x_t, u_t) + \lambda_{k+1} \log \pi_\theta(u_t|x_t)$

5.7.2 Guided Policy Search

The high level idea of guided policy search is to use some simpler local policy such as local LQR controller to help and guide the learning process of more complex global policy learner. Essentially, we would use the local models trajectories as the training data for a supervised learning neural net that can solve all the tasks.

However, one problem is that the local policies might not be able to be reproduced using a single neural net. Therefore, after training the global policy with supervised learning, we need to reoptimize the local policies using the global policy so that the policies are consistent with each other. The sketch of guided policy search is shown in Alg. 30. Note that the cost function $\tilde{c}_{k,i}$ is the modified cost function to keep π_{LQR} close to π_θ.

In Divide and Conquer RL, the idea is similar, except that we are replacing the local LQR controllers with local neural net.

5.7.3 Distillation

In RL, we borrow some ideas from supervised learning to achieve the task of learning a global policy from a bunch of local policies.

Recall in supervised learning, we use model ensemble to make our predictions more robust and accurate. However, keeping a lot of models is expensive during test time. Is there a way to train just one model that can behave as well as a meta-learner?

The idea, proposed by Hinton in **?**, is to train a model on the ensemble's predictions as "soft" targets using:

$$p_i = \frac{\exp(z_i/T)}{\sum_j (z_j/T)}$$

where T is called temperature. The new labels here can be intuitively explained using the example of MNIST dataset. For example, a handwritten digit "2" looks like a 2 and a backward 6. Therefore, the soft-labels that we use to train the distilled model is going to be "80% chance being 2 and 20% chance being 6".

In RL, to achieve multi-task global policy learning, we can use something similar called policy distillation. The idea is to train a global policy using a bunch of local tasks:

$$\mathcal{L} = \sum_a \pi_{E_i}(a|s) \log \pi_{AMN}(a|s)$$

where the meta-policy π_{AMN} can be trained in a supervised learning fashion.

- encode image observations using auto-encoder
- build predictive model on auto-encoder latent states
- use model error as exploration bonus

Schmidhuber (1991) and Schmidhuber (2010):

- exploration bonus for model errors
- exploration bonus for model gradients
- many other variations

6.4 Learning without reward functions

What if we want to recover diverse behavior without any reward function at all? Why would we want to do this? Perhaps by doing so, we can

- learn skills without supervision, then use them to accomplish new goals,
- learn sub-skills to use with hierarchical refinforcement learning,
- explore the space of many other possible behaviors.

In this subsection, we will discuss the following:

- Definitions & concepts from information theory,
- Learning without a reward function by reaching goals,
- A *state distribution-matching* formulation of reinforcement learning,
- Is coverage of valid states a *good* exploration objective?
- Beyond state covering: covering the *space of skills* .

6.4.1 Definitions & concepts from information theory

There are some useful identities in information theory that are related. The first two are

$$p(x)$$

which is the distribution of overvation x and the entropy of the distribution $p(x)$

$$\mathcal{H}(p(x)) = -\mathbb{E}_{x\sim p(x)}[\log p(x)]$$

which measures how "braod" $p(x)$ is. Another new concept is **mutual information**, which is defined as

$$\begin{aligned}\mathcal{I}(x;y) &= D_{\mathrm{KL}}(p(x,y)\|p(x)p(y)) \\ &= E_{(x,y)\sim p(x,y)}\left[\log \frac{p(x,y)}{p(x)p(y)}\right] \\ &= \mathcal{H}(p(y)) - \mathcal{H}(p(y|x)).\end{aligned}$$

Intuitively, $\mathcal{I}(x;y)$ measures how "entangled" x and y are. Related to RL, we are intersted in

- $\pi(s)$ - the state marginal distribution of policy π,
- $\mathcal{H}(\pi(s))$ - the state marginal entropy of policy π (which quantifies *coverage* of policy π),

- $\mathcal{I}(s_{t+1}; a_t) = \mathcal{H}(s_{t+1}) - \mathcal{H}(s_{t+1}|a_t)$ - "empowerment", which can be viewed as quantifying "control" in an information-theoretic sense.

Remark. How to understand $\mathcal{I}(s_{t+1}; a_t)$? For the first term, maximizing $\mathcal{H}(s_{t+1})$ means that we want more possible states to be covered in the next state s_{t+1}. For the second term, we want to have more control, i.e., to minimize the entropy of the next state given the action a_t.

6.4.2 Learning without a reward function by reaching goals

What if we don't have a concrete goal, but instead we want the agent to learn to reach a "proposed" goal? We can then propose goals $z_g \sim p(z)$, then train the agent to reach the proposed goal, which ultimately leads to the agent learning to reach any goal from the distribution of $p(z)$.

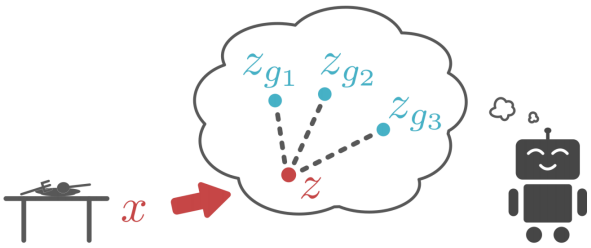


Figure 6.1: Learning to reach all goals in the distribution of $p(z)$.

To do this, we can adopt an algorithm similar to Alg. 32.

Why use a auto-encoder? How to understand $x_g \sim p_\theta(x_g|z_g)$? Why do we need $q_\phi(z_g|x_g)$?

Algorithm 32 Learning with proposed goals

1: **repeat**
2: Propose goal: $z_g \sim p(z)$ and $x_g \sim p_\theta(x_g|z_g)$
3: Attempt to reach goal using policy $\pi(a|x, x_g)$, and reach x'
4: Use data to update policy π
5: Use data to update $p_\theta(x_g|z_g)$ and $q_\phi(z_g|x_g)$
6: **until** some stopping criterion is met

Unfortunately, this algorithm might not work well, since the auto-encoder fashion of training would generate lots of similar goals, which is not what we want. Therefore, we may need to borrow some ideas from exploration. A simple solution is to reweight the state distribution $p(x)$ to make it diverse. Traditionally, in line 5, we adopt the following method:

$$\text{standard MLE: } \theta, \phi \leftarrow \arg\max_{\theta,\phi} \mathbb{E}[\log p(x')]$$

where we maximize the log-likelihood of the state x' that we actually observed. We can reweight the states to make them diverse:

$$\text{weighted MLE: } \theta, \phi \leftarrow \arg\max_{\theta,\phi} \mathbb{E}[w(x') \log p(x')]$$
$$w(x') = p_\theta(x')^\alpha$$
$$\text{key result: for any } \alpha \in [-1, 0), \mathcal{H}(p_\theta) \text{ actually increases}$$

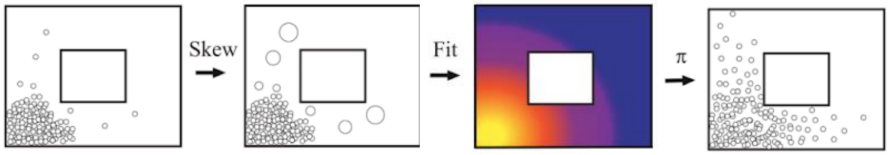


Figure 6.2: The Skew-Fit scheme.

The above analysis is pretty intuitive. Let's consider a more concrete example in RL. The objective to be optimized can be

$$\max \mathcal{H}(p(G)) - \mathcal{H}(p(G|S)) = \max \mathcal{I}(S; G) \tag{6.4.1}$$

where the first term $\mathcal{H}(p(G))$ measures the "diversity" of the goal distribution $p(G)$, and the second term $\mathcal{H}(p(G|S))$ measures the "control" of the agent in reaching the goal G given the current state S. Intuitively, as $\pi(a|S, G)$ gets better, the state S gets closer to G, which means $p(G|S)$ becomes more deterministic.

6.4.3 A state distribution-matching formulation

On one hand, the state marginal matching problem can be formulated as learning $\pi(a|s)$ so as to minimize $D_{KL}(p_\pi(s)\|p^*(s))$, where $p^*(s)$ is the target state distribution and $p_\pi(s)$ is the state distribution under policy π.

On the other hand, if a state is visited *often*, it is not *novel*. Therefore, we can add an exploration bonus to reward:

$$\tilde{r}(s) = \log p^*(s) - \log p_\pi(s).$$

We can then alternatively update π to maximize $\mathbb{E}[\tilde{r}(s)]$ and update the state distribution $p_\pi(s)$ to fit state marginal. However, this is **not** performing marginal matching, because each time we fit a policy to the states during one episode rather than the entire state distribution (Fig. 6.3). To actually perform marginal matching, we may consider averaging over all states so far by alternatively looping over step 1 and step 2:

1. learn policy π^k to maximize $\mathbb{E}[\tilde{r}(s)]$
2. update $p_{\pi^k}(s)$ to fit all states seen so far
3. return $\pi^*(a|s) = \sum_k \pi^k(a|s)$

Remark. $p_\pi(s) = p^*(s)$ is the Nash equilibrium for two player game between π^k and p_{π^k}.

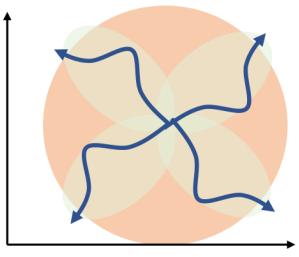


Figure 6.3: The outer orange circle is the target state distribution. The four lighter green ellipses are the state distributions observed in each episode. Darker blue lines are the policies learned in each episode.

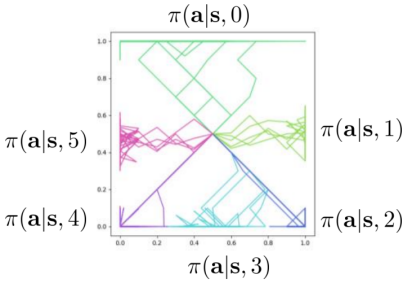


Figure 6.4: Performing diverse tasks is more like reaching different regions. Performing different skills is more like reaching the same region following different paths with the same color.

For more details, take a look at Lee et al. (2019) and Hazan et al. (2019).

Is state entropy really a good objective?

Is state entropy really a good objective?

Recall that the objective for the Skew-Fit method is $\max \mathcal{H}(p(G)) - \mathcal{H}(p(G|S)) = \max \mathcal{I}(S;G)$ and the objective for the state marginal matching problem is $\max D_{KL}(p_\pi(s) \| p^*(s))$. But when is this a good idea? Check out Gupta et al. (2018) and Hazan et al. (2019) for more details.

6.4.4 Learning diverse skills

Generally speaking, reaching diverse goals is not the same as performing diverse tasks. Intuitively, different skills should visit different state-space regions (Fig. 6.4).

Take a look at Gregor et al. (2016) and Eysenbach et al. (2018) for further details.

Learning diverse skills

6.5 Improving RL with Imitation

This section is covered in CS285 after Fall 2021 and is directly copied from the notes taken by harryhzhangOG.

Imitation learning is simple, stable supervised learning, but it requires demonstrations, and it must address distributional shift. Furthermore, the state of the art imitation learning can only be as good as the demo. In contrast, reinforcement learning can become arbitrarily good, but it requires reward function, and must address exploration. Also, it often does not have convergence guarantees.

6.5.1 Pretrain and Finetune

Can we somehow combine the two such that we have both demonstrations data and rewards? The answer is yes. The simplest idea that is practical and works is to pretrain with imitation learning and fine tune with RL. This idea is shown in Alg. 33.

Algorithm 33 Pretrain and Fine Tune

Require: Demonstrations data $\mathcal{D}$
1: Collect demonstration data $\mathcal{D} = \{(s_i, a_i)\}$
2: Initialize π_θ as $\max_\theta \sum_i \log \pi_\theta(a_i|s_i)$
3: **while** not done **do**
4: Run π_θ to collect experience
5: Improve π_θ with any RL algorithm

The problem with Alg. 33 is that in step 4, we might collect very bad experience due to distribution shift, so the first batch of data might be bad, thus destroying the initialization.

6.5.2 Off-policy RL

One way to mitigate the issue of forgetting the demonstrations is to use off-policy RL because off-policy RL can use any data, and if we let it use demonstrations as off-policy samples, since demonstrations are provided as data in every iteration, they are never forgotten. Furthermore, the policy can still become better than the demos, since it is not forced to mimic them. To achieve this, we could use off-policy policy gradients and off-policy Q-learning.

Recall policy gradients with importance sampling:

$$\nabla_\theta J(\theta) = \sum_{\tau \in \mathcal{D}} \left[\sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) \left(\prod_{t'=1}^{t} \frac{\pi_\theta(a_{t'}|s_{t'})}{q(a_{t'}|s_{t'})} \right) \left(\sum_{t'=t}^{T} r(s_{t'}, a_{t'}) \right) \right]$$

The trick here is when we collect the sum of samples, in our samples, we include both our experience and demonstration. However, it seems a little weird because in policy gradients we actually want on-policy data, so why are we including off-policy data. To answer this question, let us build up some intuition by looking at the

Algorithm 34 Q-Learning with Replay Buffer and Target Network

Require: Some base policy for data collection; hyperparameter K and N

while true **do**
  Save target network parameters: $\phi' \leftarrow \phi$
  for N times **do**
    Collect dataset $\{(s_i, a_i, s_i', r_i)\}$ using some policy, add it to replay buffer $\mathcal{B}$
    for K times **do**
      Sample a batch (s_i, a_i, s_i', r_i) from $\mathcal{B}$
      Set $y_i \leftarrow r(s_i, a_i) + \gamma \max_{a_i'} Q_{\phi'}(s_i', a_i')$
      Set $\phi \leftarrow \phi - \alpha \Sigma_i \frac{dQ_\phi}{d\phi}(s_i, a_i)(Q_\phi(s_i, a_i) - y_i)$

optimal importance sampling distribution. Say we want to estimate $\mathbb{E}_{p(x)}[f(x)]$ using importance sampling:

$$\mathbb{E}_{p(x)}[f(x)] \simeq \frac{1}{N} \sum_i \frac{p(x_i)}{q(x_i)} f(x_i)$$

and it can be proven that

$$q \propto p(x)|f(x)|$$

gives us the smallest variance. Therefore, by taking off-policy demonstration samples, we are motivating importance sampling to use distributions that have higher reward than current policy in order to get closer to the optimal distribution. To construct the sampling distribution, first we need to figure out which distribution the demonstrations come from. First, we could use supervised behavioral cloning to learn π_{demo}. If the demonstration is from multiple distribution, we could instead use **fusion distribution** by

$$q(x) = \frac{1}{M} \sum_i q_i(x)$$

6.5.3 Q-learning with Demonstrations

Since Q-learning is already off-policy, there is actually no need to bother with importance weights like we did in policy gradients. Therefore, one simple solution is just drop demonstrations into the replay buffer.

We can modify Alg. 34 slightly such that we initialize $\mathcal{B}$ with some demonstrations data, and then do the same things as before.

6.5.4 Imitation as an Auxiliary Loss Function

Recall the imitation learning maximum likelihood training objective is

$$\sum_{(s,a) \sim \mathcal{D}_{demo}} \log \pi_\theta(a|s)$$

and the RL objective is

$$\mathbb{E}_{\pi_\theta}[r(s, a)]$$

to combine the two, we can come up with a hybrid objective:

$$\mathbb{E}_{\pi_\theta}[r(s, a)] + \lambda \sum_{(s,a) \sim \mathcal{D}_{demo}} \log \pi_\theta(a|s)$$

6.6 Further Readings

Chapter 11 Inverse Reinforcement Learning

So far in our RL algorithms, we have been assuming that the reward function is known a priori, or it is manually designed to define a task. What if we want to learn the reward function from observing an expert, and then use reinforcement learning? This is the idea of **inverse RL** , where we first figure out the reward function and then apply RL.

Why should we worry about learning rewards at all? From the imitation learning perspective, the agent learns via imitation by copying the *actions* performed by the expert, without any reasoning about outcomes of actions. However, the natural way that human learn through imitation is that human copy the *intent* of the expert, and thus might take very different actions. In RL, it is often the case that the reward function is ambiguous in the environment. For example, it is hard to hand-design a reward function for autonomous driving.

The inverse RL problem definition is as follows: we try to infer the *reward functions* from demonstrations, and then learn to maximize the inferred reward using any RL algorithm available. Formally, in inverse RL, we learn $r_\psi(s,a)$, and then use it to learn $\pi^*(a|s)$. However, this is an underspecified problem, because there are infinitely many reward function can explain the same behavior. One potential form is the **linear reward function**, which is a weighted sum of features:

$$r_\psi(s,a) = \sum_i \psi_i f_i(s,a) = \psi^\top \mathbf{f}(s,a), \tag{11.0.1}$$

or it could be a neural net with parameters ψ.

11.1 Feature Matching Inverse RL

For now, let us focus on the linear reward function design. Since it is a weighted sum of features, one natural interpretation to match the features is to match the **expectation** of important features. Let π^{r_ψ} be the optimal policy for reward function r_ψ, then to design the reward, we are picking ψ such that

$$\mathbb{E}_{\pi^{r_\psi}}[\mathbf{f}(s,a)] = \mathbb{E}_{\pi^*}[\mathbf{f}(s,a)] \tag{11.1.1}$$

The expectation of features under the unknown optimal policy right hand side expectation can be estimated using samples from expert, i.e., take N samples of features, and get the average. The left hand side expectation is a little involved. One way to do it is to use any RL algorithm to maximize r_ψ, which is defined using the right hand side samples, and then produce π^{r_ψ}, and then we can use this policy to generate more samples. However, this is still ambiguous because there could be multiple reward functions r_ψ that results in the same optimal policy π^{r_ψ}. One way to address this is using the **maximum margin principle**:

$$\max_{\psi,m} m \quad \text{s.t.} \quad \psi^\top \mathbb{E}_{\pi^*}[\mathbf{f}(s,a)] \geq \max_{\pi\in\Pi} \psi^\top \mathbb{E}_{\pi}[\mathbf{f}(s,a)] + m. \tag{11.1.2}$$

However, this is problematic when the space of policies is large and continuous, since there could be very similar policies. Hence, we need to "weight" the similarity between π and π^* so that similar policies do not need to abide by the m margin requirement. Using the "SVM trick" (with the use of Lagrangian dual), we can transform the above optimization into the following which also contains a function $D(\pi,\pi^*)$ that measures the similarity between policies:

$$\min_\psi \frac{1}{2}||\psi||^2 \quad \text{s.t.} \quad \psi^\top \mathbb{E}_{\pi^*}[\mathbf{f}(s,a)] \geq \max_{\pi\in\Pi} \psi^\top \mathbb{E}_{\pi}[\mathbf{f}(s,a)] + D(\pi,\pi^*). \tag{11.1.3}$$

However, such approaches have some issues. First, maximizing the margin is a bit arbitrary. Second, there is no clear model of expert suboptimality (can add slack variables), i.e., there is no reason why the expert is

sometimes not optimal. Furthermore, now we have a messy constrained optimization problem, which is not great for deep learning. For more details, see Ratliff et al. (2006) and Abbeel and Ng (2004).

11.2 Learning the Optimality Variable

Recall that in last chapter, we introduced the optimality variable $\mathcal{O}_t$ to indicate if the agent is acting optimally. It turns out that as we learn the reward function, we are also learning the optimality variable. The optimality variable is defined as $p(\mathcal{O}_t|s_t, a_t) = \exp(r_\psi(s_t, a_t))$. Since the reward parameter ψ is unknown, the optimality distribution should also depend on ψ: $p(\mathcal{O}_t|s_t, a_t, \psi)$. Recall that

$$p(\tau|\mathcal{O}_{1:T}, \psi) \propto p(\tau) \exp\left(\sum_t r_\psi(s_t, a_t)\right) \tag{11.2.1}$$

Note that we can ignore $p(\tau)$ in our optimiztion since it does not depend on ψ. We are given sample trajectories $\{\tau_i\}$ sampled from expert policy $\pi^*(\tau)$, we can maximize the log-likelihood of the observed trajectories using the maximum likelihood principle:

$$\max_\psi \frac{1}{N}\sum_{i=1}^N \log p(\tau_i|\mathcal{O}_{1:T}, \psi) \propto \max_\psi \frac{1}{N}\sum_{i=1}^N r_\psi(\tau_i) - \log Z \tag{11.2.2}$$

where Z is the **partition function** needed to normalize of probability with respect to τ.

11.2.1 Inverse RL Partition Function

In our maximum likelihood training, to make the probability with respect to τ sum to 1, we introduced the IRL partition function Z. Mathematically, Z is the integral of all possible trajectories:

$$Z = \int p(\tau) \exp(r_\psi(\tau)) d\tau$$

Then we take the gradient of the likelihood with respect to ψ after plugging in Z:

$$\begin{aligned} \nabla_\psi \mathcal{L} &= \frac{1}{N}\sum_{i=1}^N \nabla_\psi r_\psi(\tau_i) - \frac{1}{Z}\int p(\tau)\exp(r_\psi(\tau))\nabla_\psi r_\psi(\tau) d\tau \\ &\approx \mathbb{E}_{\tau\sim\pi^*(\tau)}[\nabla_\psi r_\psi(\tau_i)] - \mathbb{E}_{\tau\sim p(\tau|\mathcal{O}_{1:T},\psi)}[\nabla_\psi r_\psi(\tau)] \end{aligned} \tag{11.2.3}$$

The first expectation is estimated with **expert samples**, and the second expectation is estimated under the **soft optimal policy under current reward**. To increase the gradient, we want more expert trajectory and less current agent trajectory.

11.2.2 Estimating the Expectation

In the above derivation, the first expectation is easy to calculate, but the second one is hard. To calculate the second expectation, we need to do some "messaging":

$$\begin{aligned} \mathbb{E}_{\tau\sim p(\tau|\mathcal{O}_{1:T},\psi)}[\nabla_\psi r_\psi(\tau)] &= \mathbb{E}_{\tau\sim p(\tau|\mathcal{O}_{1:T},\psi)}\left[\nabla_\psi \sum_{t=1}^T r_\psi(s_t, a_t)\right] \\ &= \sum_{t=1}^T \mathbb{E}_{(s_t,a_t)\sim p(s_t,a_t|\mathcal{O}_{1:T},\psi)}[\nabla_\psi r_\psi(s_t, a_t)]. \end{aligned} \tag{11.2.4}$$

Note that the distribution $p(s_t, a_t|\mathcal{O}_{1:T}, \psi)$ can be rewritten using chain rule as:

$$p(s_t, a_t|\mathcal{O}_{1:T}, \psi) = p(a_t|s_t, \mathcal{O}_{1:T}, \psi)p(s_t|\mathcal{O}_{1:T}, \psi), \tag{11.2.5}$$

where

$$\begin{aligned} p(a_t|s_t, \mathcal{O}_{1:T}, \psi) &= \frac{\beta(s_t, a_t)}{\beta(s_t)} \\ p(s_t|\mathcal{O}_{1:T}, \psi) &\propto \alpha(s_t)\beta(s_t). \end{aligned}$$

Therefore, the distribution is directly proportional to the product of the backward message $\beta(s_t, a_t)$ and the forward message $\alpha(s_t)$:

$$p(a_t|s_t, \mathcal{O}_{1:T}, \psi)p(s_t|\mathcal{O}_{1:T}, \psi) \propto \beta(s_t, a_t)\alpha(s_t). \tag{11.2.6}$$

If we let $\mu_t(s_t, a_t) \propto \beta(s_t, a_t)\alpha(s_t)$, then the second expectation can be written as:

$$\begin{aligned}\mathbb{E}_{\tau\sim p(\tau|\mathcal{O}_{1:T},\psi)}[\nabla_\psi r_\psi(\tau)] &= \sum_{t=1}^{T} \int\int \mu_t(s_t, a_t)\nabla_\psi r_\psi(s_t, a_t) ds_t da_t \\ &= \sum_{t=1}^{T} \vec{\mu}_t^\top \nabla_\psi \vec{r}_\psi\end{aligned} \tag{11.2.7}$$

where $\vec{\mu}_t$ is the state-action visitation probability for each state-action tuple (s_t, a_t).

Remark. This is only feasible for small and discrete state and action spaces. Also, we need to know the dynamics to compute the forward and backward messages.

Now we are ready to sketch out our MaxEnt Inverse RL algorithm in Alg. 37 (Ziebart et al., 2008). We can use this to learn the reward function.

Algorithm 37 MaxEnt Inverse RL

Require: Some random reward parameter ψ

1: **while** True **do**
2: Given ψ, compute backward message $\beta(s_t, a_t)$
3: Given ψ, compute forward message $\alpha(s_t)$
4: Compute $\mu_t(s_t, a_t) \propto \beta(s_t, a_t)\alpha(s_t)$
5: Evaluate $\nabla_\psi \mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}\sum_{t=1}^{T}\nabla_\psi r_\psi(s_{i,t}, a_{i,t}) - \sum_{t=1}^{T}\int\int \mu_t(s_t, a_t)\nabla_\psi r_\psi(\tau) ds_t da_t$
6: $\psi \leftarrow \psi + \eta\nabla_\psi \mathcal{L}$

Why is it called maximum entropy (MaxEnt)? Because in cases where $r_\psi(s_t, a_t) = \psi^\top \mathbf{f}(s_t, a_t)$, we can show that Alg. 37 oprimizes

$$\max_\psi \mathcal{H}(\pi^{r_\psi}) \text{ s.t. } \mathbb{E}_{\pi^{r_\psi}}[\mathbf{f}] = \mathbb{E}_{\pi^*}[\mathbf{f}].$$

Insight. The MaxEnt algorithm makes the reward function as diverse as possible, which says that you should not make any undue assumptions about the reward function that are not supported by the data.

11.3 Approximations in High Dimensions

There is some drawbacks for the MaxEnt algorithm. So far, MaxEnt Inverse RL requires

- Solving for soft optimal policy in the inner loop
- Enumerating all state-action tuples for visitation frequency and gradient

Besides, to apply this in practical problem settings, we need to handle

- Large and continuous state and action spaces
- States obtained via sampling only
- Unknown dynamics

Recall the gradient of likelihood is calculated as

$$\nabla_\psi \mathcal{L} = \mathbb{E}_{\tau\sim\pi^*(\tau)}[\nabla_\psi r_\psi(\tau)] - \mathbb{E}_{\tau\sim p(\tau|\mathcal{O}_{1:T},\psi)}[\nabla_\psi r_\psi(\tau)]$$

We know that the first expectation is easy to calculate by sampling expert data, but the second expectation is hard to calculate. One idea to calculate it is to **learn** the entire soft optimal policy $p(a_t|s_t, \mathcal{O}_{1:T}, \psi)$ using any max-ent RL algorithm and then run this policy to sample $\{\tau_j\}$ such that:

$$\nabla_\psi \mathcal{L} = \frac{1}{N}\sum_{i=1}^{N} \nabla_\psi r_\psi(\tau_i) - \frac{1}{M}\sum_{j=1}^{M} \nabla_\psi r_\psi(\tau_j) \tag{11.3.1}$$

where we estimate the second expectation using the current policy samples. To learn the entire soft optimal policy $p(a_t|s_t, \mathcal{O}_{1:T}, \psi)$, we have to optimize the following objective

$$J(\theta) = \sum_t E_{\pi(s_t,a_t)}\left[r_\psi\left(s_t, a_t\right)\right] + E_{\pi(s_t)}\left[\mathcal{H}\left(\pi\left(a|s_t\right)\right)\right] \tag{11.3.2}$$

which is highly impractical because this requires us to run an RL algorithm to convergence in every gradient step.

11.3.1 More Efficient Updates

Instead of learning the policy at each time step, we could improve the policy a little in each time step such that if the policy keeps getting better, we can generate good samples eventually. Now sampling from this improved distribution is not actually sampling from the distribution we want, which is $p(\tau|\mathcal{O}_{1:T}, \psi)$. Actually, we are getting a *biased* estimation of the distribution. Therefore, to resolve this issue, we use **importance sampling**:

$$\begin{aligned} \nabla_\psi \mathcal{L} &\approx \frac{1}{N}\sum_{i=1}^{N} \nabla_\psi r_\psi(\tau_i) - \frac{1}{\sum_j w_j}\sum_{j=1}^{M} w_j \nabla_\psi r_\psi(\tau_j) \\ w_j &= \frac{p(\tau)\exp(r_\psi(\tau_j))}{\pi(\tau_j)}. \end{aligned} \tag{11.3.3}$$

And if we take a closer look at the importance ratio w_j:

$$\begin{aligned} w_j &= \frac{p(\tau)\exp(r_\psi(\tau_j))}{\pi(\tau_j)} \\ &= \frac{p(s_1)\prod_t p(s_{t+1}|s_t,a_t)\exp(r_\psi(s_t,a_t))}{p(s_1)\prod_t p(s_{t+1}|s_t,a_t)\pi(a_t|s_t)} \\ &= \frac{\exp(\sum_t r_\psi(s_t,a_t))}{\prod_t \pi(a_t|s_t)} \end{aligned}$$

With the importance ratio, each policy update with respect to r_ψ brings us closer to the target distribution.

11.4 Inverse RL as a Generative Adversarial Network

The idea of inverse RL looks like a game. Specifically, we have an initial policy π_θ, and expert demonstrations π^*. We sample trajectories τ_j from the initial policy, and τ_i from the expert policy. Then our gradient step looks like:

$$\nabla_\psi \mathcal{L} \simeq \frac{1}{N}\sum_{i=1}^{N} \nabla_\psi r_\psi(\tau_i) - \frac{1}{\sum_j w_j}\sum_{j=1}^{M} w_j \nabla_\psi r_\psi(\tau_j) \tag{11.4.1}$$

where demos are made more likely and samples are made less likely. Then we update the initial policy π_θ with respect to r_ψ:

$$\nabla_\theta \mathcal{L} \approx \frac{1}{M}\sum_{j=1}^{M} \nabla_\theta \log \pi_\theta(\tau_j) r_\psi(\tau_j) \tag{11.4.2}$$

which in turn changes the policy to make it harder to distinguish from demos.

This looks a lot like a Generative Adversarial Network (GAN). In a GAN, we have a generator that takes in some noise z, and outputs a distribution $p_\theta(x|z)$. We sample from the generator distribution $p_\theta(x)$. There is also demonstration data, for example, the real images, which we sample from its distribution $p^*(x)$. There is a discriminator parameterized by ψ that determines if the data is real: $D(x) = p_\psi(\text{real}|x)$. We update the discriminator parameter by maximizing the binary log likelihood:

$$\psi = \arg\max_\psi \frac{1}{N} \sum_{x \sim p^*} \log D_\psi(x) + \frac{1}{M} \sum_{x \sim p_\theta} \log(1 - D_\psi(x)) \tag{11.4.3}$$

where the log likelihood of the data is from demonstration is maximized and that of the data is from generator is minimized. We also update the generator parameter θ:

$$\theta \leftarrow \arg\max_\theta \mathbb{E}_{x \sim p_\theta} \log D_\psi(x) \tag{11.4.4}$$

so as to make it harder to distinguish from demos. Therefore, interestingly, we can frame the IRL problem as a GAN. In a GAN, the optimal discriminator can be showed to be:

$$D^*(x) = \frac{p^*(x)}{p_\theta(x) + p^*(x)} \tag{11.4.5}$$

For inverse RL, the optimal policy approaches $\pi_\theta(\tau) \propto p(\tau) \exp(r_\psi(\tau))$. Choosing the above optimal parameterization of the discriminator:

$$\begin{aligned} D_\psi(\tau) &= \frac{p(\tau) \frac{1}{Z} \exp(r(\tau))}{p_\theta(\tau) + p(\tau) \frac{1}{Z} \exp(r(\tau))} \\ &= \frac{p(\tau) \frac{1}{Z} \exp(r(\tau))}{p(\tau) \prod_t \pi_\theta(a_t|s_t) + p(\tau) \frac{1}{Z} \exp(r(\tau))} \\ &= \frac{\frac{1}{Z} \exp(r(\tau))}{\prod_t \pi_\theta(a_t|s_t) + \frac{1}{Z} \exp(r(\tau))} \end{aligned} \tag{11.4.6}$$

then we optimize the discriminator with respect to ψ such that:

$$\psi = \arg\max_\psi \mathbb{E}_{\tau \sim p^*}[\log D_\psi(\tau)] + \mathbb{E}_{\tau \sim \pi_\theta}[\log(1 - D_\psi(\tau))]$$

Now we don't need the importance ratio anymore, because it is subsumed into Z.

Can we just use a regular discriminator?

What if we don't want to recover the reward function? We could also use a general discriminator, i.e., D_ψ is just a normal binary neural net classifier. It is often simpler to set up optimization because we have fewer moving parts. However, the discriminator knows nothing at convergence, and generally the reward cannot be re-optimized.

Figure 11.1: IRL as adversarial optimization.

11.5 Further readings

For classic papers, Abbeel and Ng (2004) is a good introduction to inversed reinforcement learning. Ziebart et al. (2008) is an introduction to probabilistic method for inverse reinforcement learning. For modern papers, Finn et al. (2016) and Wulfmeier et al. (2015) talked about sampling-based method for MaxEnt IRL that handles unknown dynamics and deep reward functions. Ho and Ermon (2016) talked about IRL using GANs. Ho and Ermon (2016) further proposed learning robust reward functions with adversarial IRL.